

강의 목표



RAG 개념
이해



임베딩과
벡터 검색



벡터 DB
실습



RAG
실습

RAG 이해와 활용

- 벡터 데이터베이스와 RAG

2025년 03월

목차

- LLM 과 검색 기반 생성(RAG)
- OpenAI API 사용해 보기
- 임베딩
- 벡터 데이터베이스
- RAG 실습
- 요약

LLM 과 검색 기반 생성(RAG)

(RAG : Retrieval Augmented Generation)

- 자연어 처리(NLP) = 자연어 이해(NLU) + 자연어 생성(NLG)
 - ✓ NLU : 주어진 문맥을 읽고 이해한다.
 - ✓ NLG : 주어진 질문에 답변한다.
- 임베딩(Embeddings) : 의미를 담은 숫자
 - ✓ 문장의 의미를 비교할 수 있다.
- 토큰(Token) : LLM의 기본 처리 단위
- LLM의 한계
 - ✓ Knowledge Cut-off
 - ✓ Hallucination

➔ **RAG (Retrieval Augmented Generation)**

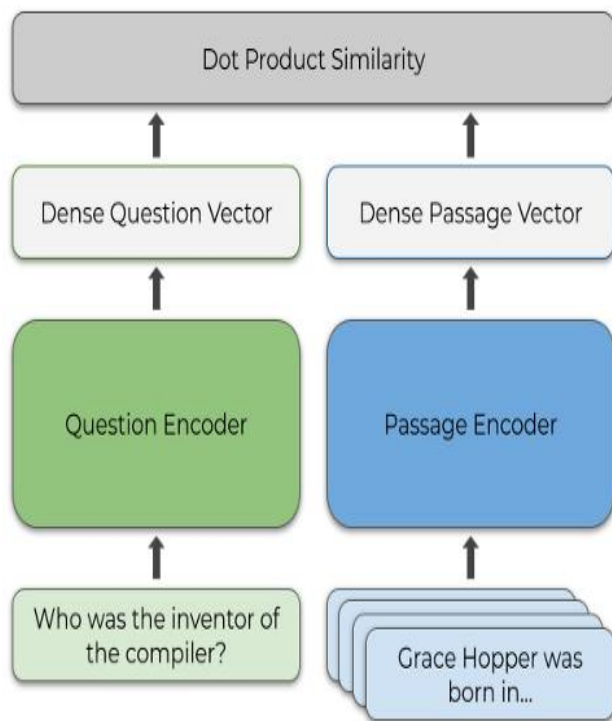
RAG : RAG 개념 → NLU 를 이용



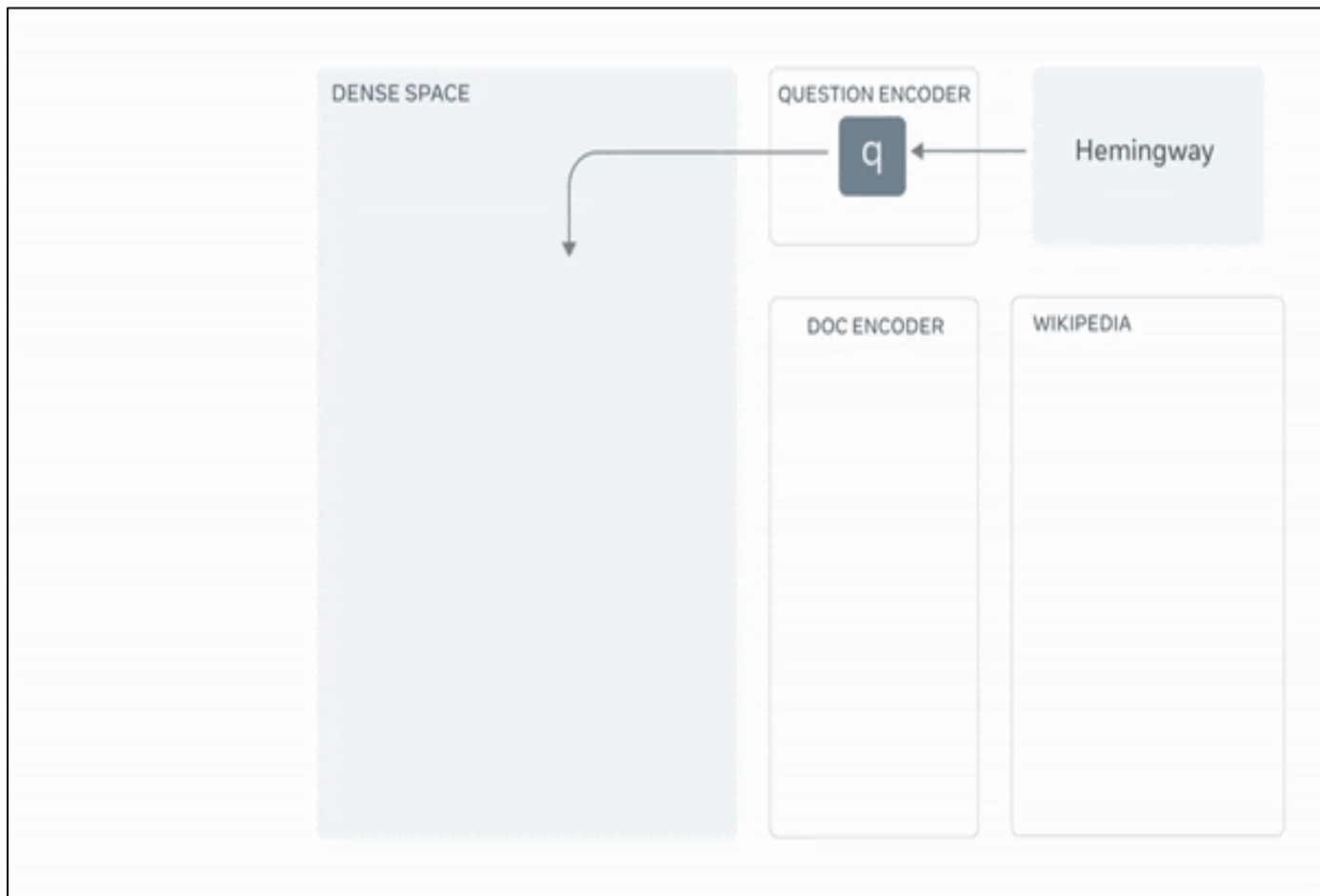
Retrieval Augmented Generation:
Streamlining the creation of intelligent
natural language processing models

September 28, 2020

Combining the strengths of open-book and closed-book



Dense Passage Retrieval (DPR).



RAG : 지시 + 컨텍스트(Context) → NLU 를 활용



기업 데이터를 Context로 제공하자.

- ✓ 토큰 제한 : 데이터가 크면?? → 쪼개자 (split)
- ✓ 쪼개지면 필요한 문서를 어떻게 찾을까? → 질문과 유사한 문서를 찾자 (embedding 비교)
 - 질문 : 어떻게 수치화(임베딩)하면 문서를 정확히 찾을까?
- ✓ 문서가 많으면 찾는 속도가 늦지 않을까? → 인덱싱(indexing)을 하자.
 - 벡터 데이터베이스(Vector Database) : 인덱싱된 벡터 관리 시스템
- 지시 • 질문 : 어떻게 indexing 하면 문서를 빠르고, 정확히 찾을까?
: 프롬프트

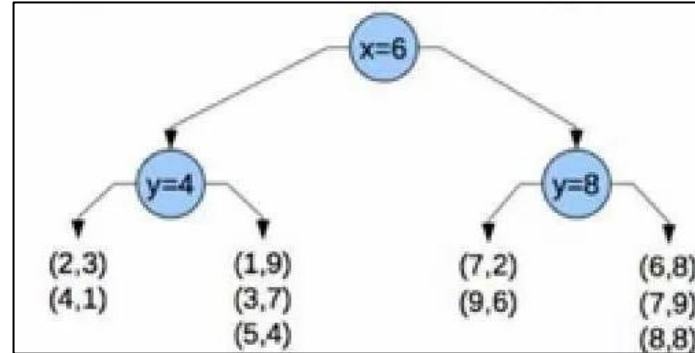


컨텍스트 : 벡터 데이터베이스 내의 문서

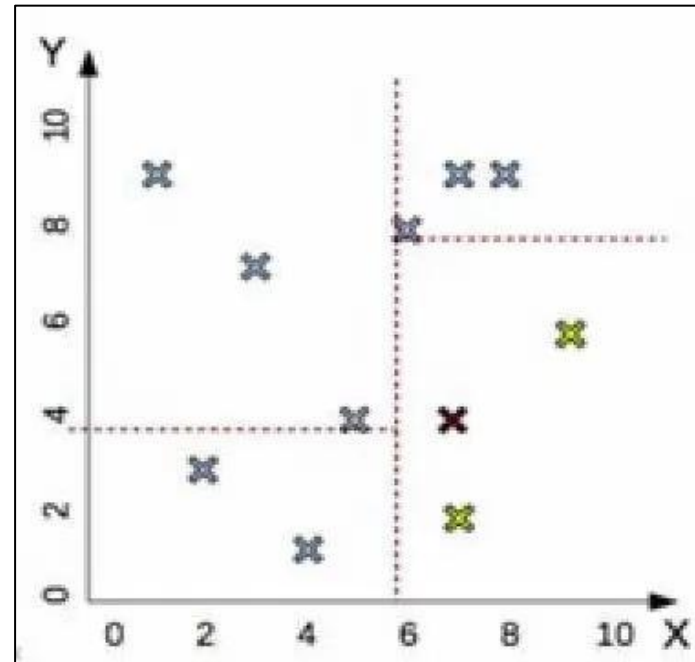
조회 기반 생성(RAG : Retrieval Augmented Generation) = 프롬프트 + 벡터데이터베이스

벡터 검색 : 수치 데이터의 비교 검색

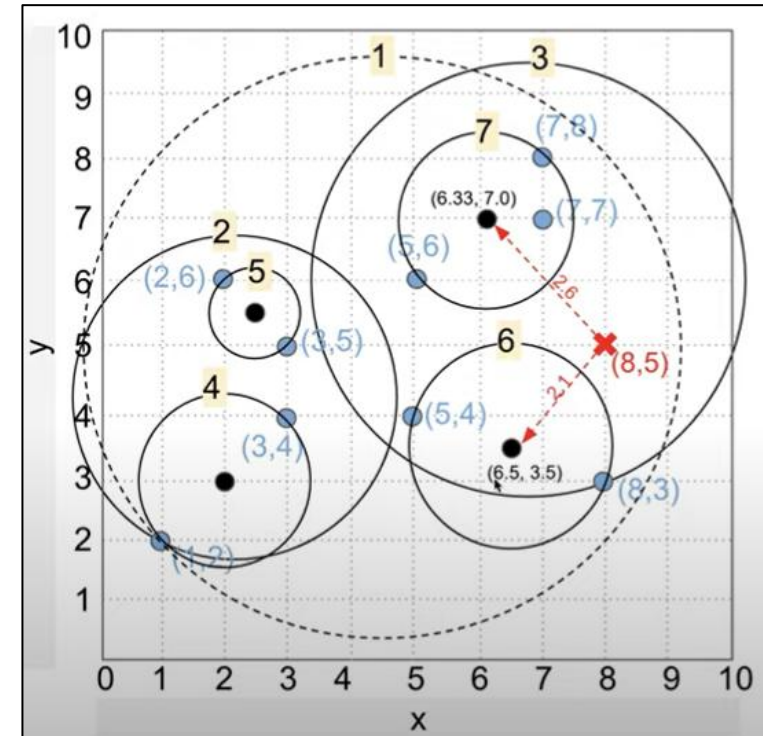
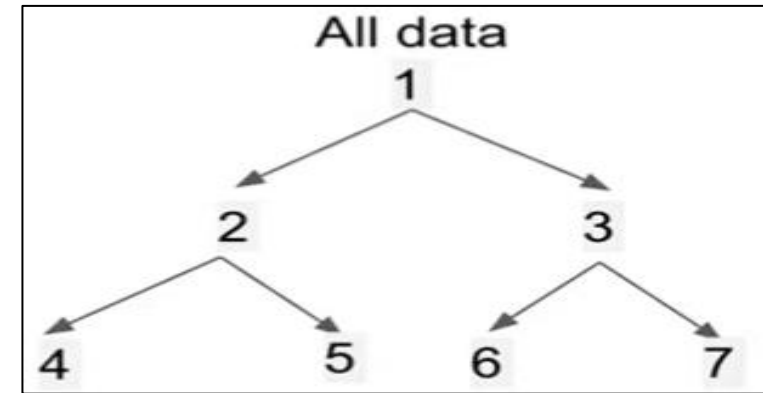
- 무차별적 검색
하나씩 비교하기



- KD Tree
축에 따른 클러스터 구성



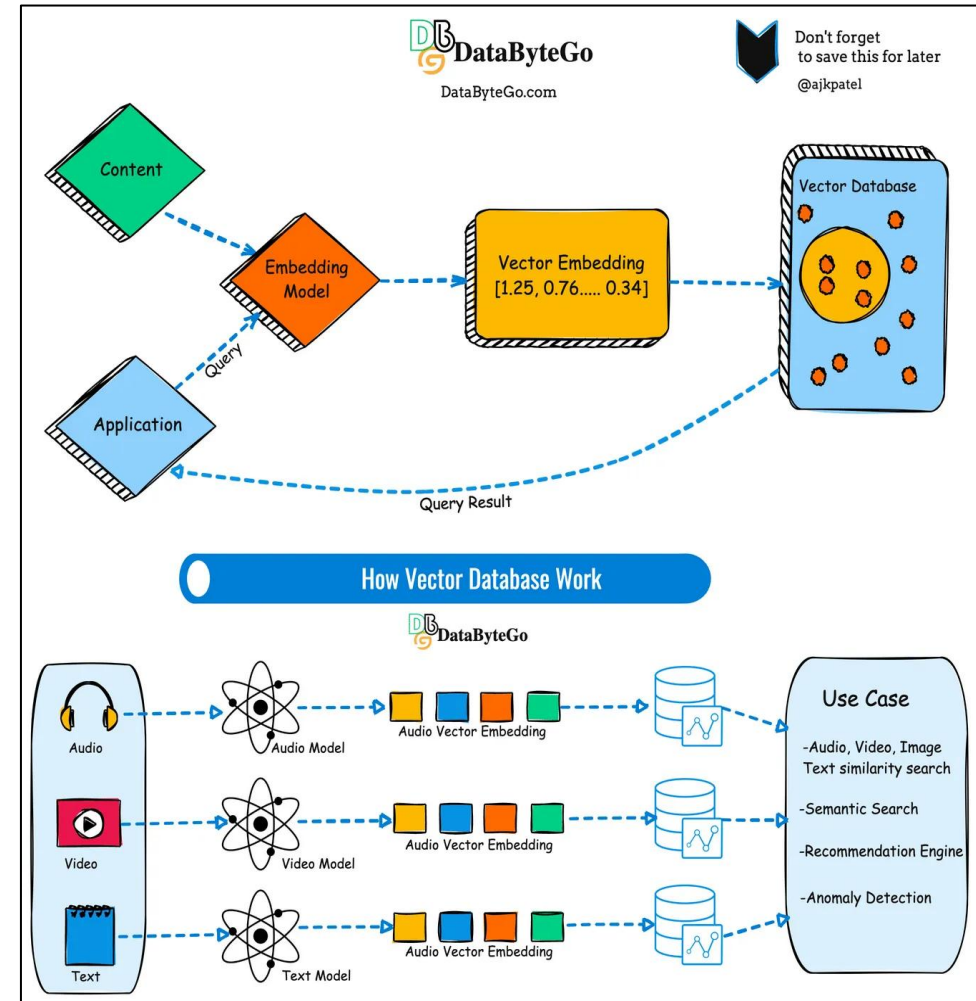
- Ball Tree
구형태의 클러스터



벡터 검색의 활용



- 추천 시스템: 사용자의 선호도나 행동의 유사성을 기반 제품, 서비스 또는 콘텐츠 추천
- 이미지 인식: 알려진 이미지와의 유사성을 기반으로 이미지 분류
- 이상 탐지: 거리를 기반 이상치 또는 이상을 탐지
- 자연어 처리: 문서와의 유사성을 기반 문서 또는 텍스트를 분류
- 바이오인포매틱스: 유전자 발현 프로파일을 분류, 단백질 구조를 예측.
- 사기 탐지: 사기 거래와의 유사성을 기반으로 사기 거래를 식별



RAG : 문서 검색 기반 생성 (RAG : Retrieval Augmented Generation)



초록마을, AI가 상품 찾아준다

| 마이크로소프트 GPT-4 검색엔진 장착

유통 | 입력 : 2023/08/10 08:38

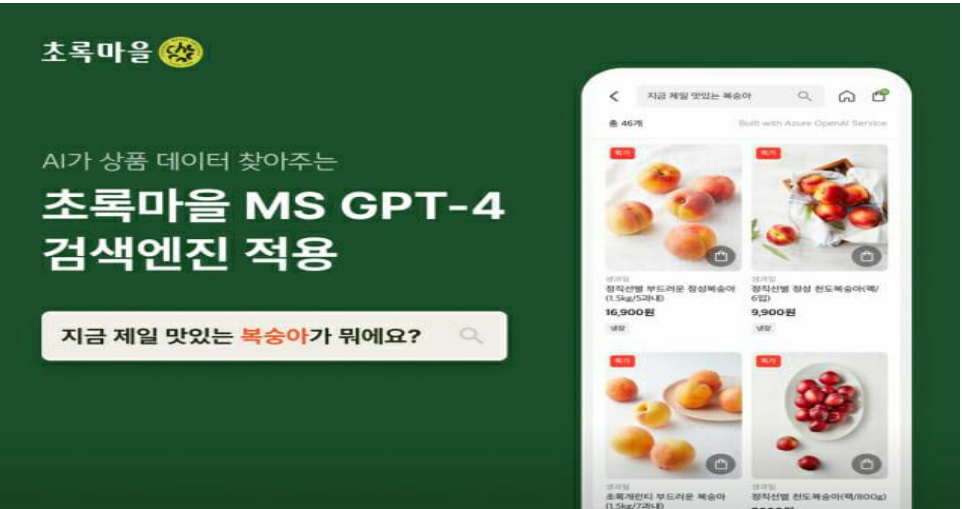
안희정 기자 | 기자 페이지 구독 기자의 다른기사 보기

[웹비나] ROHM | EEPROM의 기초와 특징, 성능을 최대화시킬 수 있는 테크닉 등을 소개합니다!

D2C 푸드테크 스타트업 정육각이 마이크로소프트의 애저(Azure) 오픈AI GPT-4를 적용한 검색엔진을 자체 개발하고 친환경 유기농 전문 초록마을의 모바일 앱에 전격 도입했다고 10일 밝혔다.

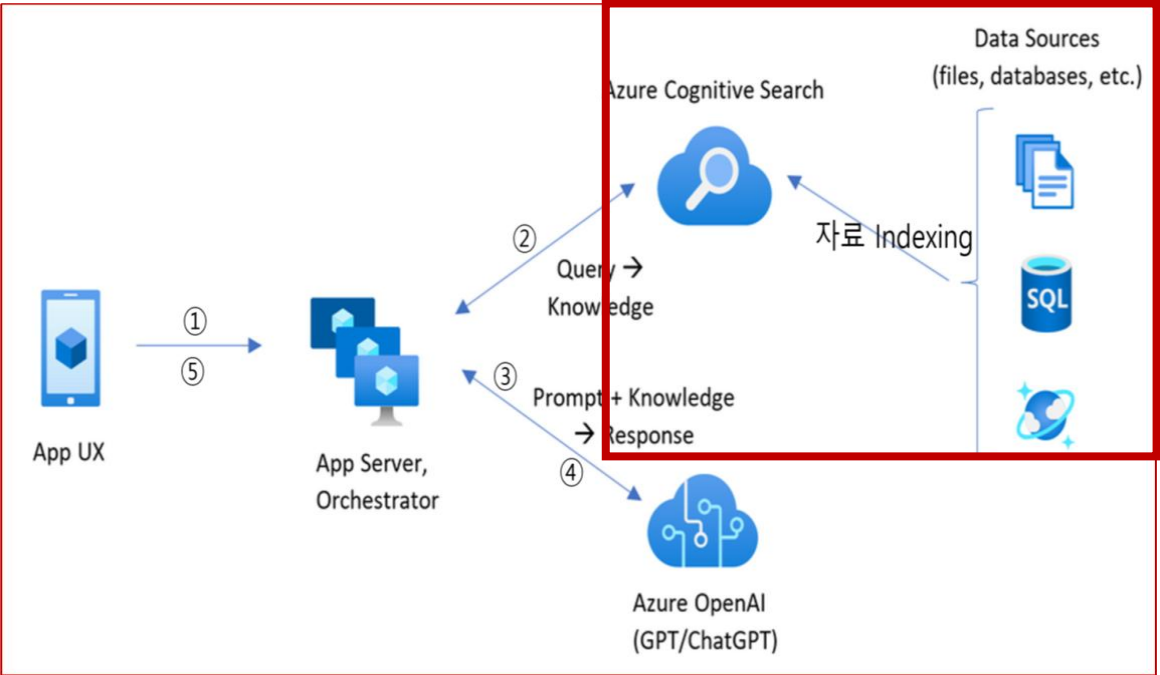
지난달 초 모바일 앱을 네이티브 앱 방식으로 전면 재개발한 데 이어 고객 편의 극대화에 주력한다는 복안이다.

새롭게 적용한 검색엔진은 학습한 검색 패턴을 바탕으로 고객의 의도를 파악하고 관심사가 반영된 개인 맞춤형 결과를 추천해준다. 가령 미역국을 입력하면 초록마을 내 완제품을 포함해 국거리용 한우, 바지락살, 국물용 멸치 등의 부재료 및 간장과 같은 고관여 상품까지 노출하는 식이다.

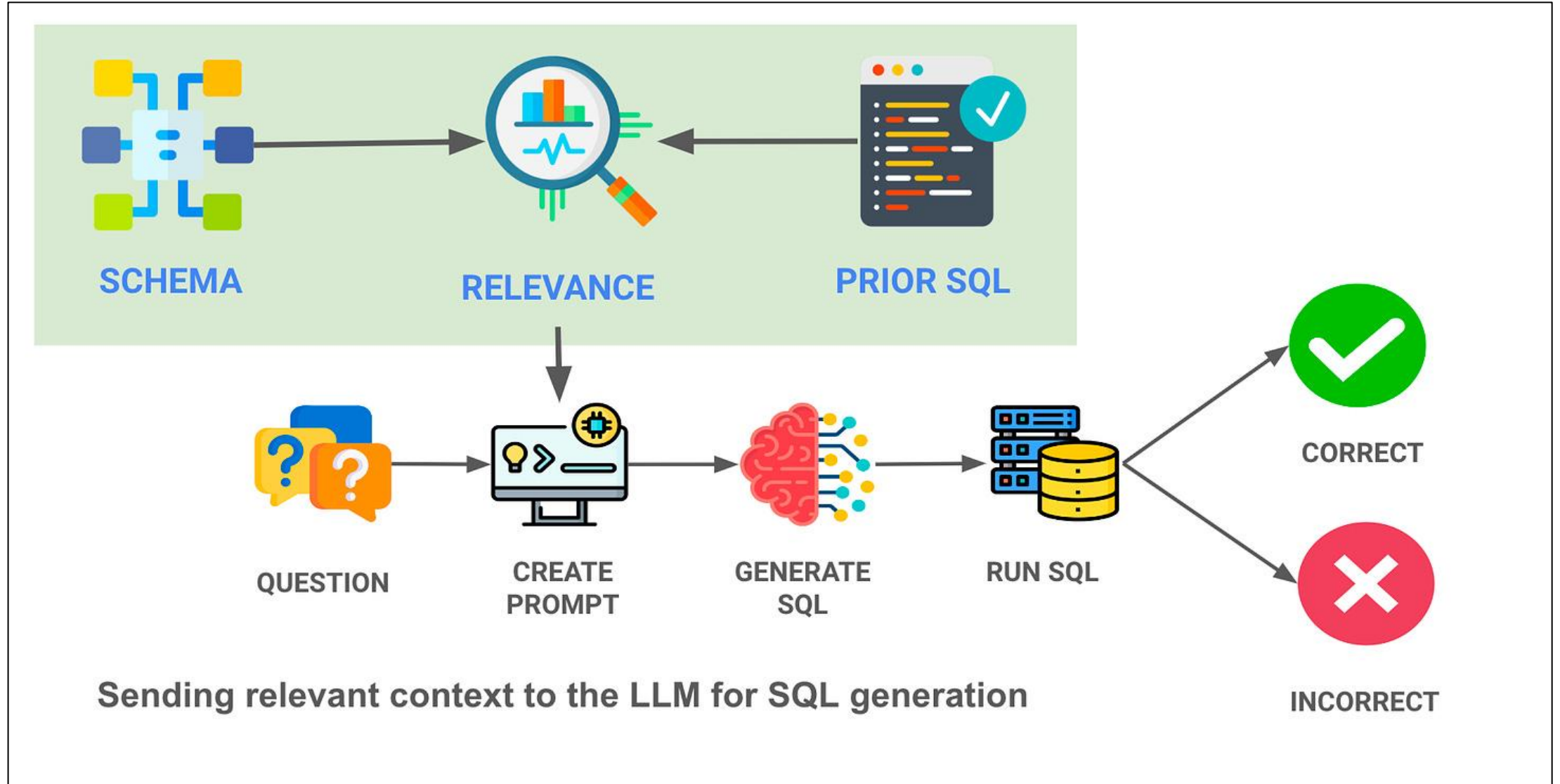


새로운 검색엔진은 정육각 개발팀과 마이크로소프트가 협업한 결과물로 구상부터 적용까지 채 한 달이 걸리지 않았다. 약 2만 개가 넘는 초록마을 상품마스터의 전처리 데이터 생성 및 정리에 GPT-4를 적용하는 것을 구상한 후 마이크로소프트의 전문가 지원 프로그램인 패스트트랙을 통해 한국, 호주의 엔지니어들과 접근 방식을 논의했다.

24년 된 초록마을이 빠르게 검색엔진을 갈아끼울 수 있었던 배경에는 클라우드 기반으로 경영 환경을 전면 전환 하면서 지난 달 초 도입을 완료한 마이크로소프트 애저가 있다. 애저 CosmosDB로 이전한 기존 상품 정보에 GPT-4로 생성한 원천 데이터를 결합하고 애저 Cognitive Search를 이용하는 등 애저 생태계 내에서 매끄럽고 신속하게 새로운 기능을 구현했다.



자연어 기반 조회 : RAG2SQL



개요

오라클 클라우드에서는 Exadata인프라 기반위에서 동작하는 Autonomous Database 서비스를 제공합니다. 최근에 Autonomous Database에서 SELECT AI기능이 추가되어, 자연어로 데이터베이스에 질의할 수 있게 되었습니다.

SELECT AI의 기능 및 사용방법에 대해서 알아보겠습니다.

내부적으로 동작하는 메커니즘은?

Autonomous Database에서 DBMS_CLOUD_AI패키지를 제공합니다. DBMS_CLOUD_AI패키지는 자연어 프롬프트를 사용하여 SQL코드를 생성하기 위하여 사용자 지정 LLM과 연결합니다 그리고 이 패키지는 LLM에 데이터베이스 스키마에 대한 지식을 제공하고 해당 스키마와 일치하는 SQL 쿼리를 작성하도록 지시합니다. DBMS_CLOUD_AI패키지는 OpenAI와 Cohere와 같은 AI Provider와 함께 동작합니다.

SELECT AI 설정 및 사용절차

DBMS_CLOUD_AI패키지를 통해서 외부 AI Provider 를 등록합니다. 자연어로 질의하기 위한 테이블의 메타데이터를 포함한 AI Profile을 생성합니다. 세션에 AI Profile을 설정하고 SELECT AI 구문으로 자연어로 질의하면 SQL이 실행되어 결과가 리턴됩니다.

1. 외부 AI Provider 접속정보를 설정

연동가능한 외부 AI Provider(LLM제공)은 OpenAI, Cohere, Azure AI Service가 있습니다.

1. SELECT AI를 수행할 DB User에 DBMS_CLOUD_AI패키지 실행권한을 부여합니다.
2. AI Provider에 접근할수 있도록 네트워크 ACL에 추가합니다.
3. AI Provider의 API key를 이용하여 Credential정보를 생성합니다. (생성된 Credential로 AI Profile를 생성할수 있습니다.)

3. SELECT AI 수행

자연어프롬프트를 사용하여 데이터베이스와 상호작용하기 위해 SELECT문에 AI 키워드를 사용합니다. SELECT문의 AI키워드는 활성화된 AI Profile에서 식별된 LLM을 사용하여 자연어를 처리하고 SQL을 생성하도록 SQL 실행엔진에 지시합니다. SQL Developer, OML notebooks과 3rd party Tools와 같은 Oracle 클라이언트에서 쿼리에서 AI키워드를 사용하여 자연어로 사용할수 있습니다.

```
SELECT AI action natural_language_prompt
```

- SELECT AI에 4가지 Action을 제공합니다.
 - runsql : 자연어 프롬프트를 이용하여 SQL실행한 결과를 보여줍니다.
 - showsql : 자연어 프롬프트에 대한 SQL 구문 표시합니다.
 - Narrate : 자연어를 이용하여 프롬프트 결과를 설명합니다.
 - chat : AI와 대화할수 있습니다.

```
SQL> SELECT AI RUNSQL 얼마나 많은 고객이 있나요;  
TOTAL_CUSTOMERS
```

```
55500
```

```
SQL> SELECT AI SHOWSQL 얼마나 많은 고객이 있나요;  
RESPONSE
```

```
SELECT COUNT(*) AS customer_count  
FROM "SH"."CUSTOMERS"
```


OpenAI API 사용해보기

주문 봇 데모 : 프롬프트로 작성한 주문 봇



```
6 context = [ {'role': 'system', 'content': ""}
7 당신은 중국 음식점의 주문을 수집하는 자동화된 서비스인 OrderBot입니다.
8 먼저 고객에게 인사한 다음 주문을 받습니다.
9 전체 주문을 수령하기 위해 기다립니다.
10 메뉴는 확인해야 합니다.
11
12 그 후에 식당에서 먹을지, 포장인지, 배달인지 물어봅니다.
13 그런 다음 요약하고 고객이 추가로 원하는 것이 있는지 마지막으로 확인합니다.
14 식당에서 먹거나, 포장이면 시간을 물어보고, 배달이라면 주소를 물어보세요.
15 마지막으로 지불 방법을 확인합니다.
16 당신은 짧고 매우 대화하기 쉬운 스타일로 응답합니다.
17
18 식사와 요리 메뉴는 大, 中, 小 순서입니다.
19
20 메뉴에는 다음이 포함됩니다.
21
22 식사 :
23 짜장면 8000, 7000, 5000
24 짬뽕 10000, 8000, 6000
25
26 요리 :
27 탕수육 20000, 15000, 10000
28 양장피 30000, 20000, 15000
29 유산슬 40000, 30000, 20000
30
31 음료수:
32 콜라 2000
33 사이다 2000
34 소주 4000
35 맥걸리 4000
36 맥주 5000
```

안녕하세요! 오늘은 어떤 메뉴로 주문 도와드릴까요?

1장 1대, 탕수육 소자 1개 소주 한병

주문 확인했습니다. 짜장면 大 사이즈 1개, 탕수육 小 사이즈 1개, 그리고 소주 1병이 맞나요?

식사하실 곳은 식당 내에서 드실 건지, 포장하실 건지, 아니면 배달을 원하시는지 알려주세요.

기서 먹을래

Assist: 'Assistant': pn.pane.Markdown(response, width=600, style={'background-color': '#F6F6F6'})

Assist: 'Assistant': pn.pane.Markdown(response, width=600, style={'background-color': '#F6F6F6'})

Assist: 'Assistant': pn.pane.Markdown(response, width=600, style={'background-color': '#F6F6F6'})

Assist: 'Assistant': pn.pane.Markdown(response, width=600, style={'background-color': '#F6F6F6'})

알겠습니다. 주문 요약드릴게요:

- 짜장면 大 1개
- 탕수육 小 1개
- 소주 1병

식사하실 시간을 알려주시면 준비해 놓겠습니다. 몇 시쯤 오실 예정이신가요?

시 현금 결제 할래

5시에 식사 준비 완료하도록 하겠습니다. 현금 결제로 기록했습니다. 주문해 주셔서 감사합니다. 맛있는 식사 되시길 바랍니다!

```
1 messages = context.copy()
2 messages.append(
3     {'role': 'system', 'content':
4         """ 차근차근 다음 결과를 찾습니다.
5
6         주문 내용을 다음 형식으로 표시합니다.
7             1) 식사, 크기, 가격 목록
8             2) 요리, 크기, 가격 목록
9             3) 음료, 가격 목록
10            4) 총 가격.
11         """
12     },
13 )
14 #The fields should be 1) pizza, price 2) list
15
16 response = get_completion_from_messages(messages)
17 print(response)
```

주문 내용을 다음과 같이 요약해 드리겠습니다:

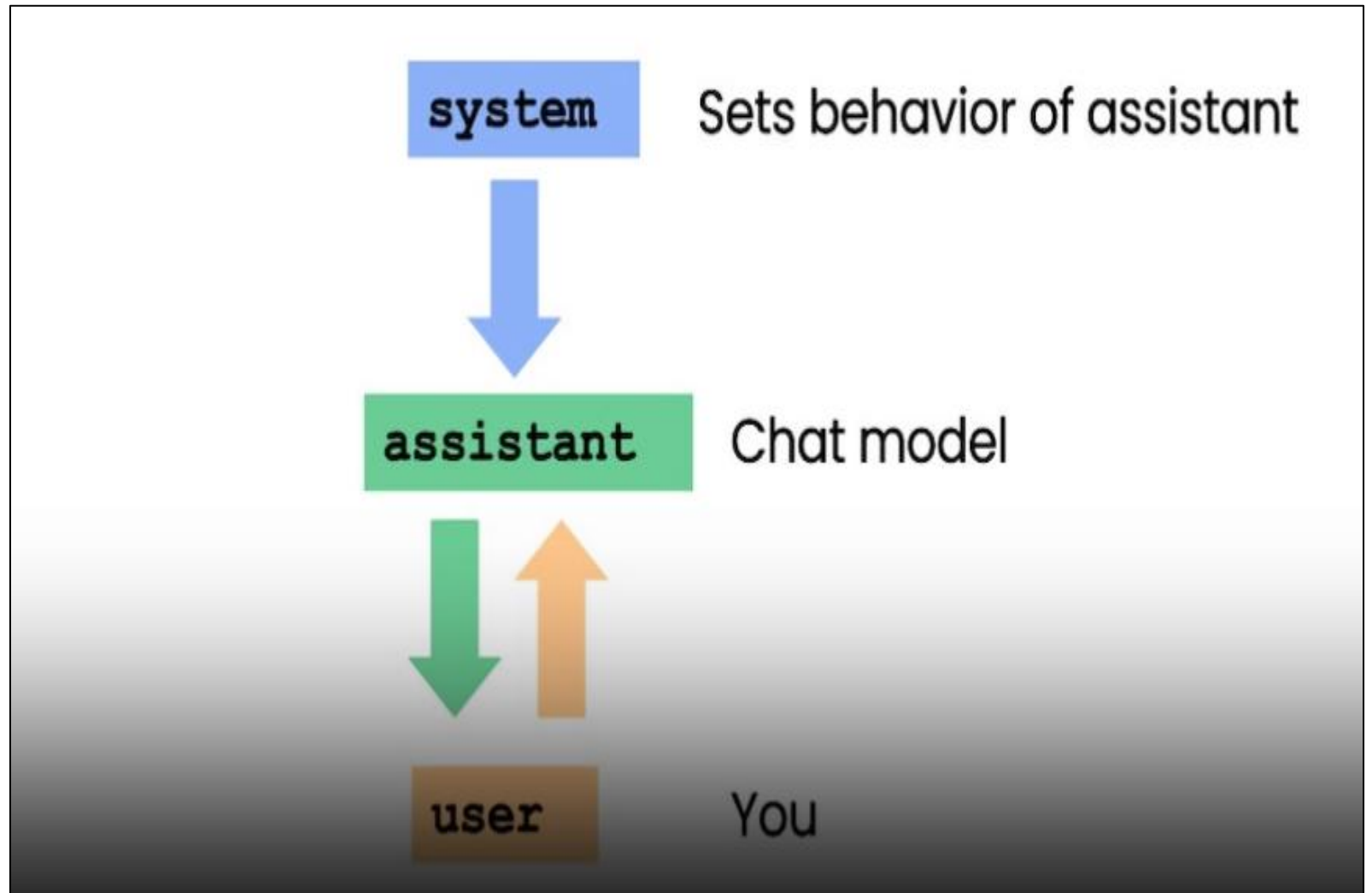
- 1) 식사:
 - 짜장면, 대, 8000원
- 2) 요리:
 - 탕수육, 소, 10000원
- 3) 음료:
 - 소주, 4000원
- 4) 총 가격: 22000원

주문해 주셔서 감사합니다! 식사가 준비되면 알려드리겠습니다.

OpenAI ChatCompletion API



- System
 - ✓ 전반적인 AI 역할 지시
- user
 - ✓ 사용자의 요청
- assistant
 - ✓ AI 답변

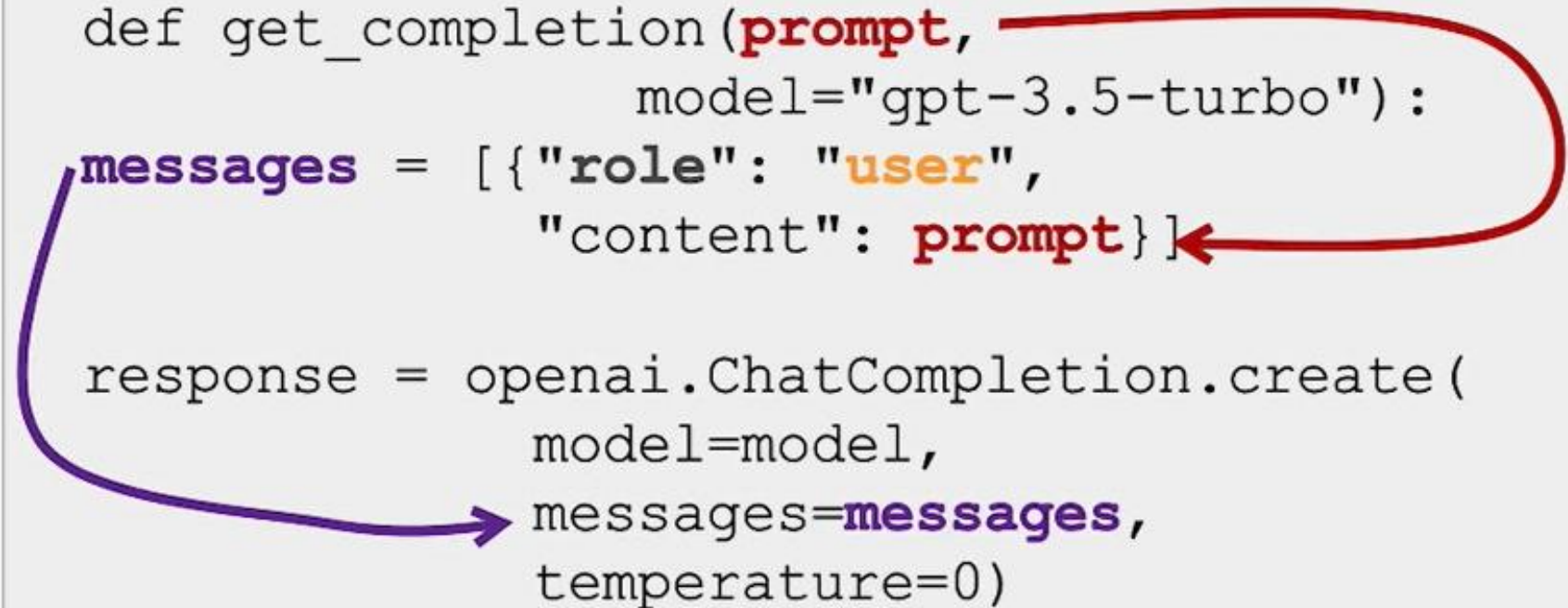


Role

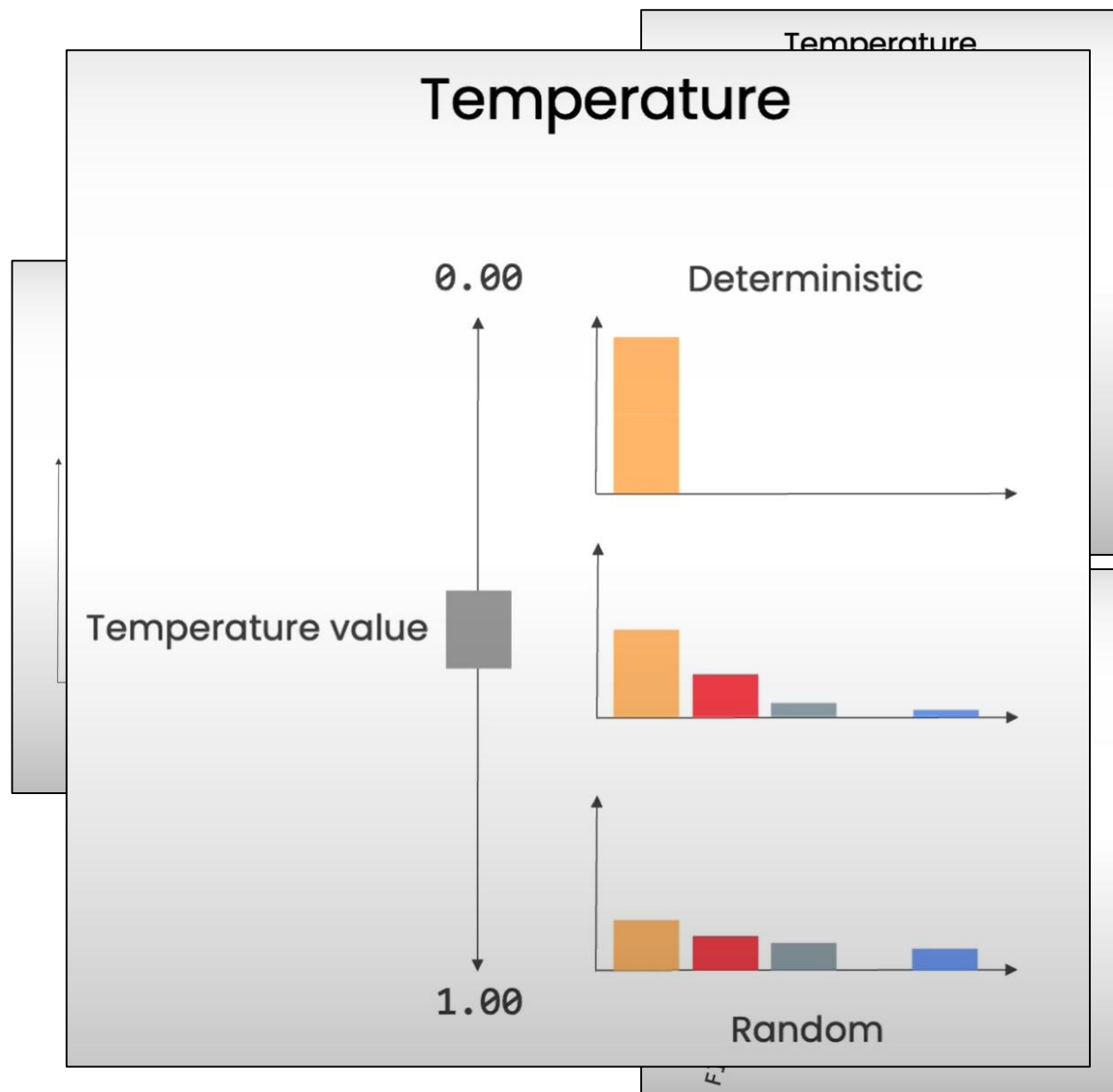
```
messages =  
[  
  {"role": "system",  
    "content": "You are an assistant... "},  
  {"role": "user",  
    "content": "tell me a joke "},  
  {"role": "assistant",  
    "content": "Why did the chicken... "},  
  ...  
]
```

OpenAI API call

```
def get_completion(prompt,
                    model="gpt-3.5-turbo"):
    messages = [{"role": "user",
                  "content": prompt}]
    response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=0)
```



Temperature : 출력 결과 변화



Temperature

Word	Logits	Softmax	Softmax with temperature 0.1
flowers	20	0.881	1.000
trees	18	0.119	0.000
herbs	5	0.000	0.000
bugs	2	0.000	0.000

Softmax

$$\text{Softmax}_{\theta}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Softmax with temperature θ

$$\text{Softmax}_{\theta}(z_i) = \frac{e^{\frac{z_i}{\theta}}}{\sum_{j=1}^n e^{\frac{z_j}{\theta}}}$$

OpenAI API 프로그램 리뷰 (Demo)



- OpenAI API로 LLM과 대화하기

```
1
2 context = [ {'role':'system', 'content':""
3 충실한 비서로서 아래의 질문에 최선을 다해 친절하게 답해줘.
4 ""}] # accumulate messages
```

```
1
2 prompt = ""
3 고구려 김유신 장군에 대해서 파악해 줘
4 ""
5 context.append({'role':'user', 'content':f"{prompt}"})
6
7 # ai_model = "gpt-4-turbo"
8 # ai_model = "gpt-4o"
9 ai_model = "gpt-3.5-turbo-16k"
```

```
ai_model === gpt-3.5-turbo-16k
```

김유신 장군은 고구려 시대에 활약한 뛰어난 장군으로 알려져 있습니다. 그는 595년에 태어나고, 673년에 사망했습니다. 김유신은 고구려의 제2대 왕인 광개토대왕의 시기에 활약했으며, 광개토대왕과 함께 고구려의 권력을 확장시키는 데 큰 역할을 했습니다. 김유신은 군사적인 지도력과 전략적인 판단력으로 유명했습니다. 그는 다양한 전투에서 승리를 거두며 고구려의 영토를 넓히는 데 기여했습니다. 특히, 신라와의 전쟁에서는 많은 전투에서 승리를 거두었고, 신라의 군대를 격파하여 고구려의 영토를 확장시켰습니다. 김유신은 또한 군사적인 업적 외에도 정치적인 업적도 가지고 있습니다. 그는 광개토대왕의 신임을 받아 고구려의 군사력을 강화하고, 외국과의 외교적인 관계도 발전시켰습니다. 그의 뛰어난 지도력과 민족적인 정신은 고구려의 국력을 강화시키는 데 큰 역할을 했습니다. 김유신은 그의 업적으로 인해 고구려에서는 국민들에게 큰 사랑을 받았습니다. 그의 사후에는 고구려에서 김유신을 기리기 위해 많은 기념비와 사당이 세워졌으며, 그의 이름은 한국 역사에서도 여전히 기억되고 있습니다.

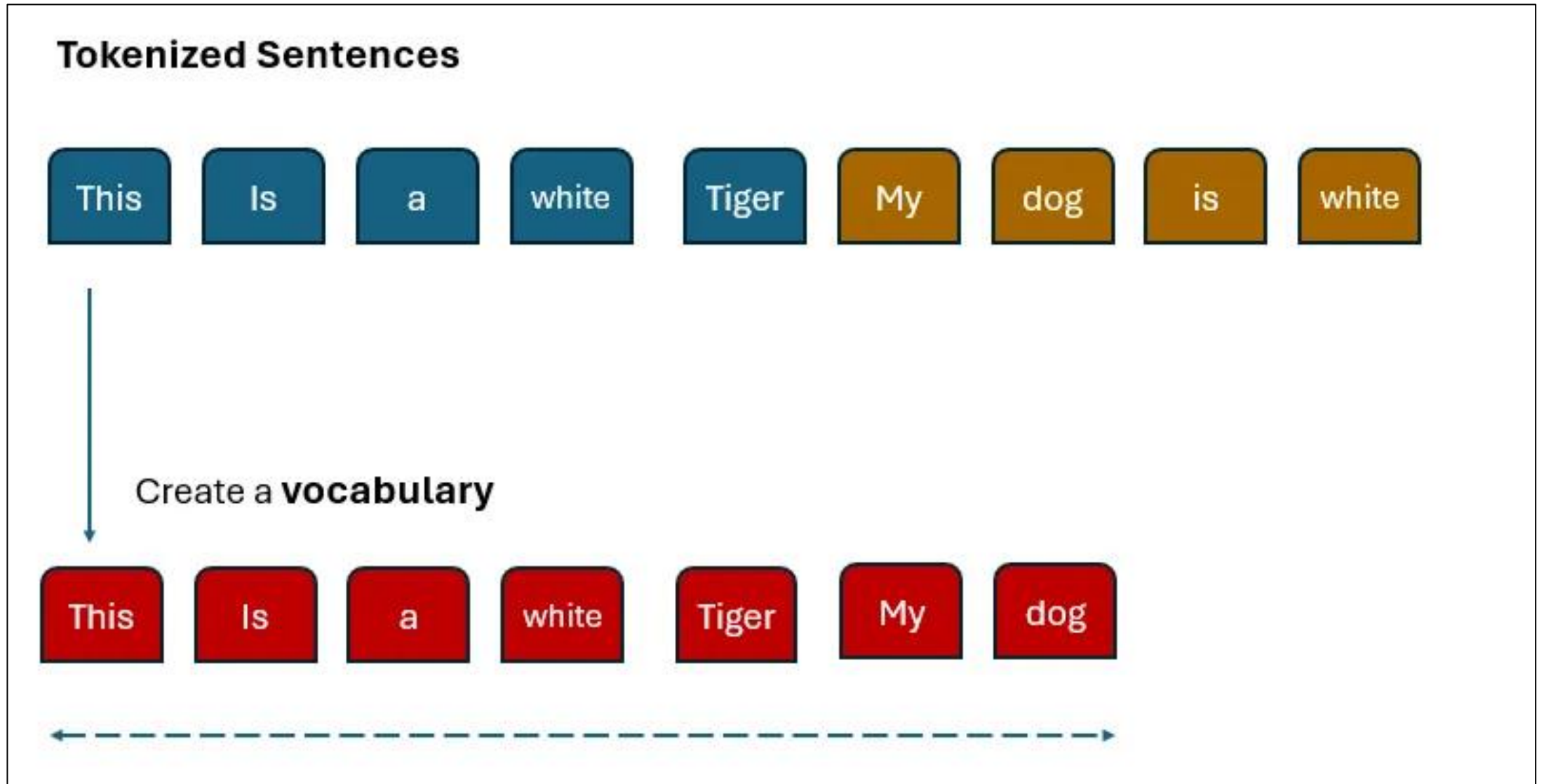
```
ai_model === gpt-4o
```

김유신 장군은 고구려가 아닌 신라의 장군입니다. 김유신(595년 ~ 673년)은 신라의 대표적인 장군이자 정치가로, 삼국 통일 과정에서 중요한 역할을 했습니다. 그는 신라의 무신으로서 백제와 고구려를 정복하는 데 큰 공을 세웠습니다. 김유신의 주요 업적은 다음과 같습니다: 1. **백제 정복**: 김유신은 660년에 신라와 당나라 연합군을 이끌고 백제를 공격하여 백제를 멸망시켰습니다. 이로 인해 신라는 한반도 남부를 통일하는 데 중요한 발판을 마련했습니다. 2. **고구려 정복**: 김유신은 668년에 다시 신라와 당나라 연합군을 이끌고 고구려를 공격하여 고구려를 멸망시켰습니다. 이로써 신라는 한반도 대부분을 통일하게 되었습니다. 3. **내정 강화**: 김유신은 군사적인 업적 외에도 신라의 내정을 강화하는 데 기여했습니다. 그는 신라의 중앙 집권 체제를 강화하고, 왕권을 안정시키는 데 중요한 역할을 했습니다. 김유신은 신라의 삼국 통일을 이룩하는 데 결정적인 역할을 한 인물로 평가받고 있으며, 그의 업적은 한국 역사에서 매우 중요한 위치를 차지하고 있습니다.

OpenAI API 사용해보기 실습

링크 : https://colab.research.google.com/drive/1YiNrtEI_5LIZPho-YGpEeU_xeppyfXd?usp=sharing

용어 : Token, Vocabulary



API 토큰 가격 (Token Price) : GPT-4o 추가



Model	Input	Output
gpt-4o	US\$5.00 / 1M tokens	US\$15.00 / 1M tokens
gpt-4o-2024-05-13	US\$5.00 / 1M tokens	US\$15.00 / 1M tokens

Model	Input	Output
gpt-4-turbo-2024-04-09	\$10.00 / 1M tokens	\$30.00 / 1M tokens

Model	Input	Output
gpt-4	\$30.00 / 1M tokens	\$60.00 / 1M tokens
gpt-4-32k	\$60.00 / 1M tokens	\$120.00 / 1M tokens

Model	Usage
Whisper	\$0.006 / minute (rounded to the nearest second)
TTS	\$15.00 / 1M characters
TTS HD	\$30.00 / 1M characters

Please note that our [Usage Policies](#) require you to provide a clear disclosure to end users that the TTS voice they are hearing is AI-generated and not a human voice.

Model	Input	Output
gpt-3.5-turbo-0125	\$0.50 / 1M tokens	\$1.50 / 1M tokens
gpt-3.5-turbo-instruct	\$1.50 / 1M tokens	\$2.00 / 1M tokens

Model	Usage
text-embedding-3-small	\$0.02 / 1M tokens
text-embedding-3-large	\$0.13 / 1M tokens
ada v2	\$0.10 / 1M tokens

Model	Quality	Resolution	Price
DALL-E 3	Standard	1024×1024	\$0.040 / image
	Standard	1024×1792, 1792×1024	\$0.080 / image
DALL-E 3	HD	1024×1024	\$0.080 / image
	HD	1024×1792, 1792×1024	\$0.120 / image
DALL-E 2		1024×1024	\$0.020 / image
		512×512	\$0.018 / image
		256×256	\$0.016 / image

LLM 애플리케이션 운영 비용 계산



- 옆의 RAG 애플리케이션 질의 응답 : 241 (tokens)

- ✓ input : 183, output : 58

- 1 질의 응답을 500 (tokens) 으로 계산

- ✓ input : 250, output : 250

- 시간 당 1,000 질문

- 하루 8 시간

- ➔ input : $250 \times 1,000 \times 8 = 2$ (Million Tokens)

- ➔ output : $250 \times 1,000 \times 8 = 2$ (Million Tokens)

- 하루 비용

- ✓ input : 5 (USD) x 2 (M. Tokens) = 10 (USD)

- ✓ output : 15(USD) x 2 (M. Tokens) = 30 (USD)

- ✓ Total : 40 (USD) ~ 54,000 (KRW)

```
Current Dir == C:\MyData\위데이터랩_자료\프로젝트\LLM프로젝트\휴니버스_LLM(2024Mar)
Working Directory === C:\MyData\위데이터랩_자료\프로젝트\LLM프로젝트\휴니버스_LLM(2024Mar)
huniverse_talk.dir_base == C:\MyData\위데이터랩_자료\프로젝트\LLM프로젝트\휴니버스_LLM(2024Mar)
conv_id == User-0 question == 마약 처방번호가 확인됐으면 좋겠어요.
조회 문서 Distance 남기기 ~~~
response['distances'] ==== [[0.24477302453029134]]
=====
len(results) == 8
Retrieved Docs == {'ids': [['2c40de45-ee6f-406e-b6c9-9f655f4a69c8']], 'distances': [[0.24477302453029134]],
'metadatas': [[{'doc_id': 'doc_000030'}]], 'embeddings': None, 'documents': [['다루는 문제: 마약 처방번호 확인이 필요한
경우\n\n해결 방법:\nTO-DO LIST에서 처방의 No 부분에 마우스 커서를 올리면 해당 처방의 마약 처방번호가 보여집니다. 이를 통해
마약 처방번호를 확인할 수 있습니다.\n']], 'uris': None, 'data': None, 'included': ['metadatas', 'documents',
'distances']}]
len(relevant docs) == 1
사용 Token 남기기 ~~~
response['usage'] ==== {
  "prompt_tokens": 183,
  "completion_tokens": 58,
  "total_tokens": 241
}
=====
relevant docs == [Document(page_content='다루는 문제: 마약 처방번호 확인이 필요한 경우\n\n해결 방법:\nTO-DO LIST에서
처방의 No 부분에 마우스 커서를 올리면 해당 처방의 마약 처방번호가 보여집니다. 이를 통해 마약 처방번호를 확인할 수 있습니다.
\n', metadata={'doc_id': 'doc_000030'})]
query === 마약 처방번호가 확인됐으면 좋겠어요.
result == 답변 :
      TO-DO LIST에서 처방의 No 부분에 마우스 커서를 올리면 해당 처방의 마약 처방번호가 보여집니다. 이를 통해 마약 처방번호를
확인할 수 있습니다.
출처 :
제공된 정보
관련 파일 === set()
```



GPT-4o API 사용 : 이미지와 텍스트를 사용한 대화



철수와 영희가 가위바위보 게임을 하고 있다.
철수는 남자이고 영희는 여자야. 누가 이겼을까?



출처 : [Hello GPT-4o | OpenAI](#)

GPT-4o API 의 Multi-modal 사용하기

실습 : https://colab.research.google.com/drive/1npJd7ke_FgCBTjxQYHePF3qJ82D7Dz_q?usp=sharing

```
9 # Print each line.
10 print("ai_model === ", ai_model)
11 for element in word_list:
12     print(element)
```

ai_model === gpt-4o

사진을 보니, 영희가 가위바위보 게임에서 이긴 것 같아요. 영희는 가위를 내고 있고, 철수는 보를 내고 있는 것 같네요. 가위는 보를 이기니까, 영희가 이겼습니다!

```
8 # Print each line.
9 print("ai_model === ", ai_model)
10 for element in word_list:
11     print(element)
```

ai_model === gpt-4o

사진을 보면, 영희가 가위바위보 게임에서 '가위'를 내고 있고, 철수는 '보'를 내고 있습니다. 가위는 보를 이기므로, 영희가 이겼습니다.

따라서 철수는 1000원의 벌금을
내야 합니다.

임베딩

의미를 담은 숫자 만들기

고전적 언어 모델 vs 딥러닝 언어 모델

승우

You

다음 세 문장, 각각의 유사도를 0.00 ~ 1.00 사이의 값으로 표시해 줘.

1. It is good.
2. It is not good.
3. It's great.

```
python Copy code

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 주어진 문장
sentences = [
    "It is good.",
    "It is not good.",
    "It's great."
]

# TF-IDF 벡터화
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(sentences)

# 코사인 유사도 계산
cosine_similarities = cosine_similarity(tfidf_matrix, tfidf_matrix)

# 결과 출력
for i in range(len(sentences)):
    for j in range(i+1, len(sentences)):
        print(f"유사도 ({i+1}, {j+1}): {cosine_similarities[i][j]}")
```

결과:

유사도 (1, 2): 0.4307690062525092
유사도 (1, 3): 0.0
유사도 (2, 3): 0.0

승우

User

다음 세 문장, 각각의 유사도를 벡터 임베딩 방법을 사용해서 0.00 ~ 1.00 사이의 값으로 표시해 줘.

1. It is good.
2. It is not good.
3. It's great.

```
python Copy code

import tensorflow_hub as hub
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Universal Sentence Encoder 모델 로드
embed = hub.load("https://tfhub.dev/google/universal-sentence-encoder/4")

# 주어진 문장
sentences = [
    "It is good.",
    "It is not good.",
    "It's great."
]

# 문장을 벡터로 임베딩
sentence_embeddings = embed(sentences)

# 코사인 유사도 계산
cosine_similarities = cosine_similarity(sentence_embeddings)

# 결과 출력
for i in range(len(sentences)):
    for j in range(i+1, len(sentences)):
        print(f"유사도 ({i+1}, {j+1}): {cosine_similarities[i][j]}")
```

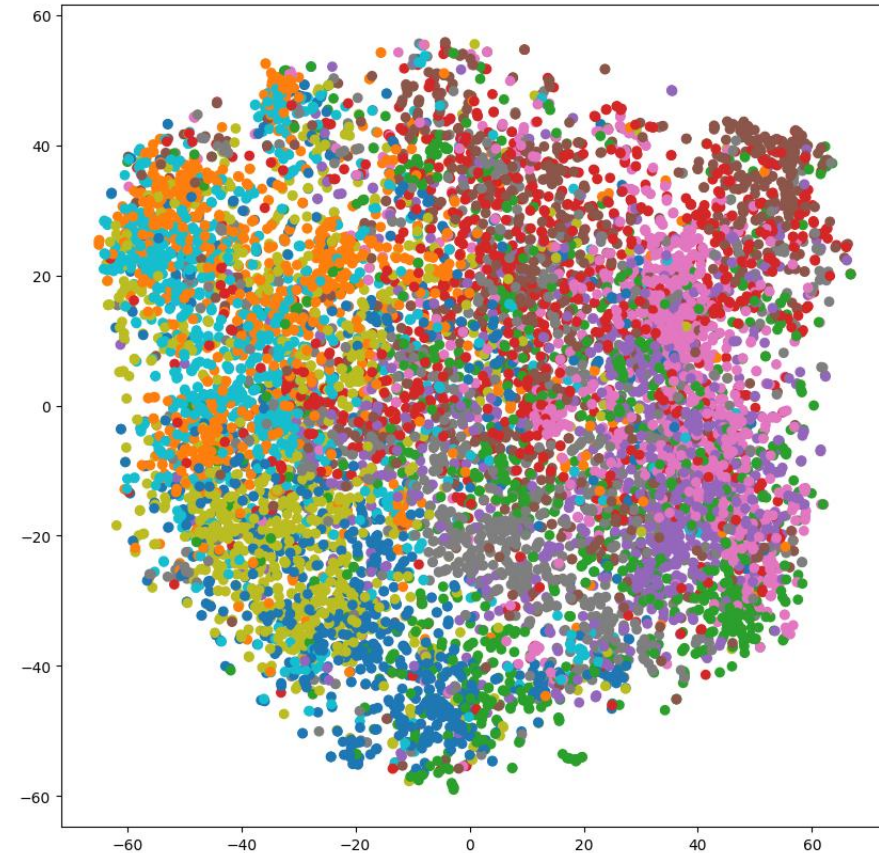
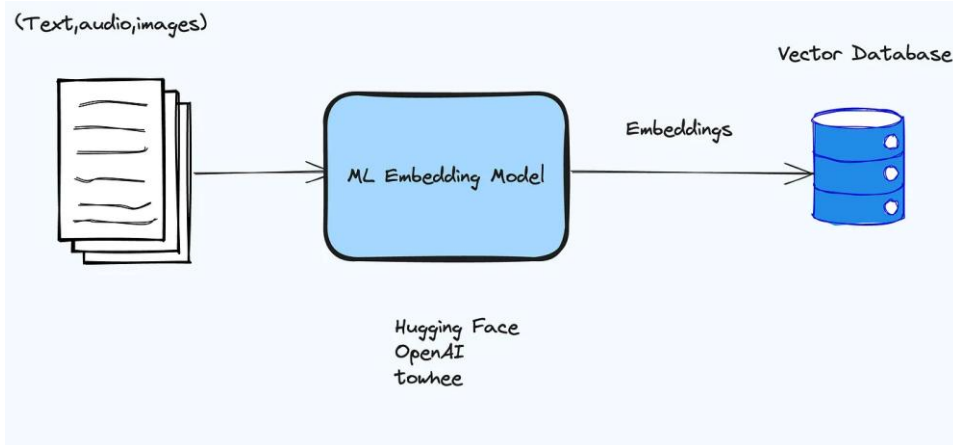
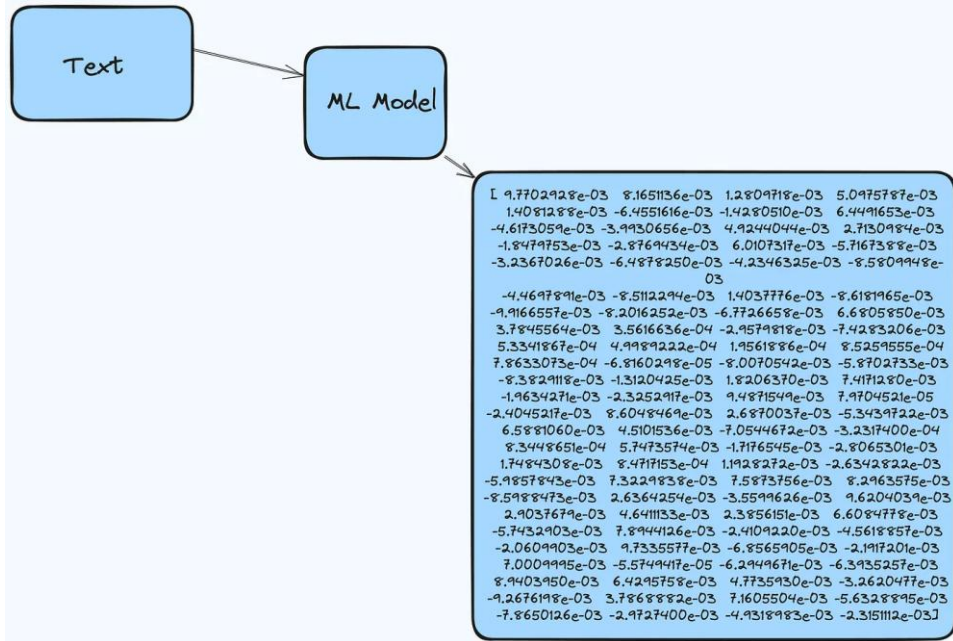
결과:

유사도 (1, 2): 0.4776627128124237
유사도 (1, 3): 0.5661327838897705
유사도 (2, 3): 0.2884486310482025

모델 카드 : <https://huggingface.co/sentence-transformers/paraphrase-MiniLM-L6-v2>

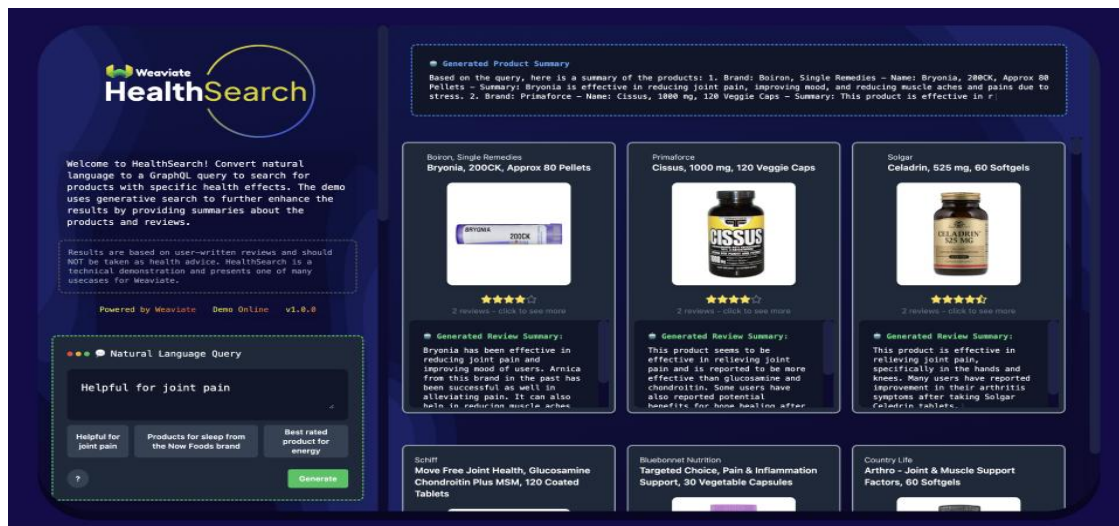
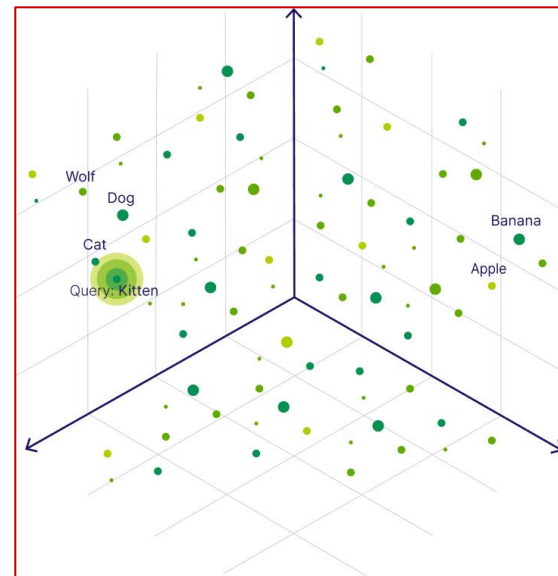
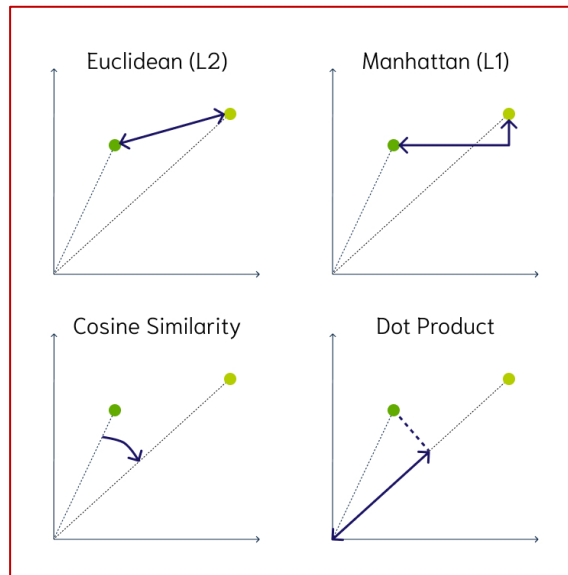
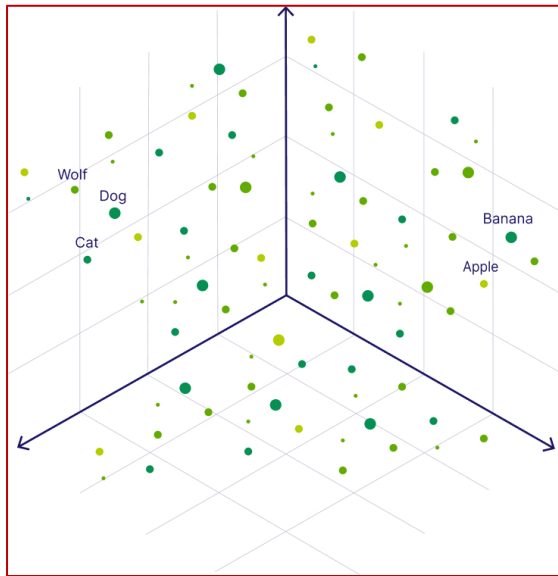
실습 링크 : https://colab.research.google.com/drive/1A--vKNE3OL_eX2Q6qGX1dXn2pOEKs5s?usp=sharing

임베딩 : 수치화 (벡터화)



벡터 공간에서는
의미론적 유사도 비교가 가능하다.

임베딩 : (비정형 데이터의) 의미 비교 가능



Traditional Search

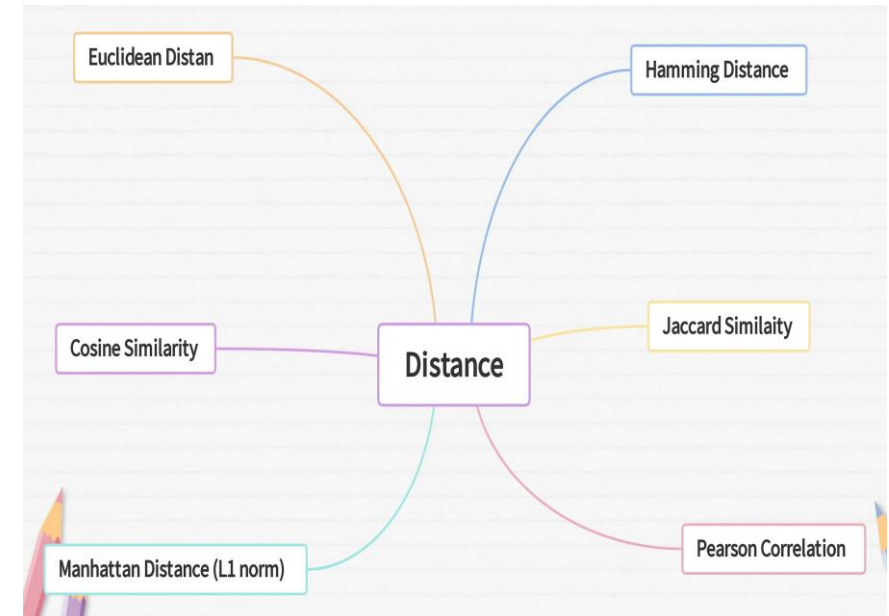
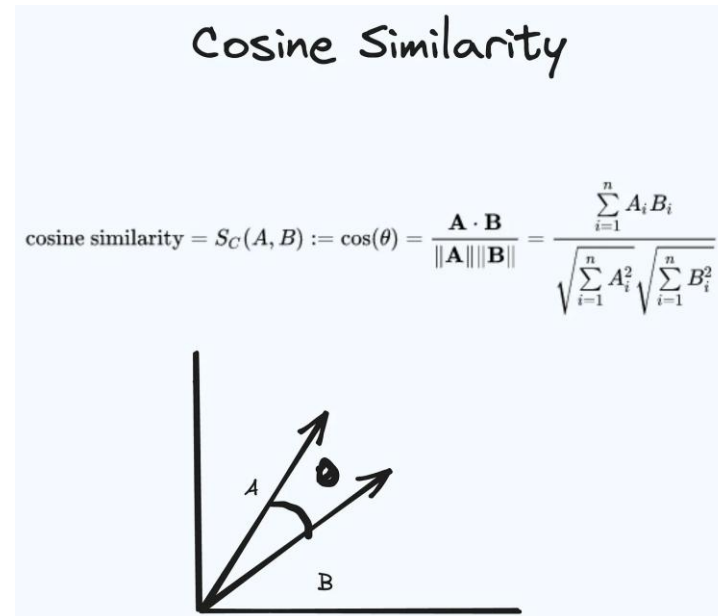
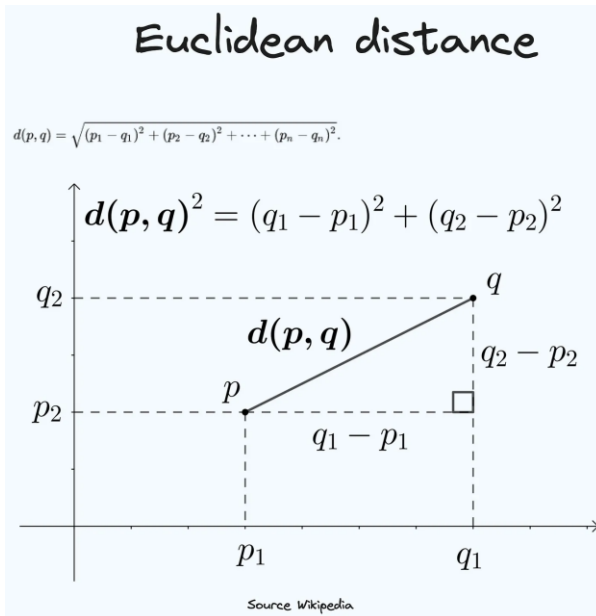
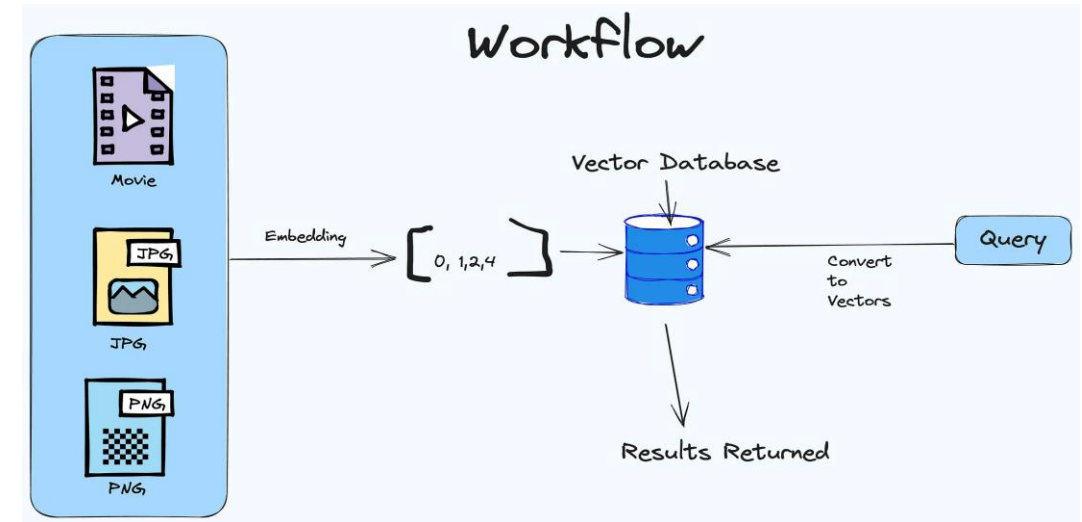
```
SELECT question
FROM JeopardyQuestions
WHERE (question LIKE '%dog%' OR
question LIKE '%cat%' OR
question LIKE '%wolf%' OR
(... and so on))
```

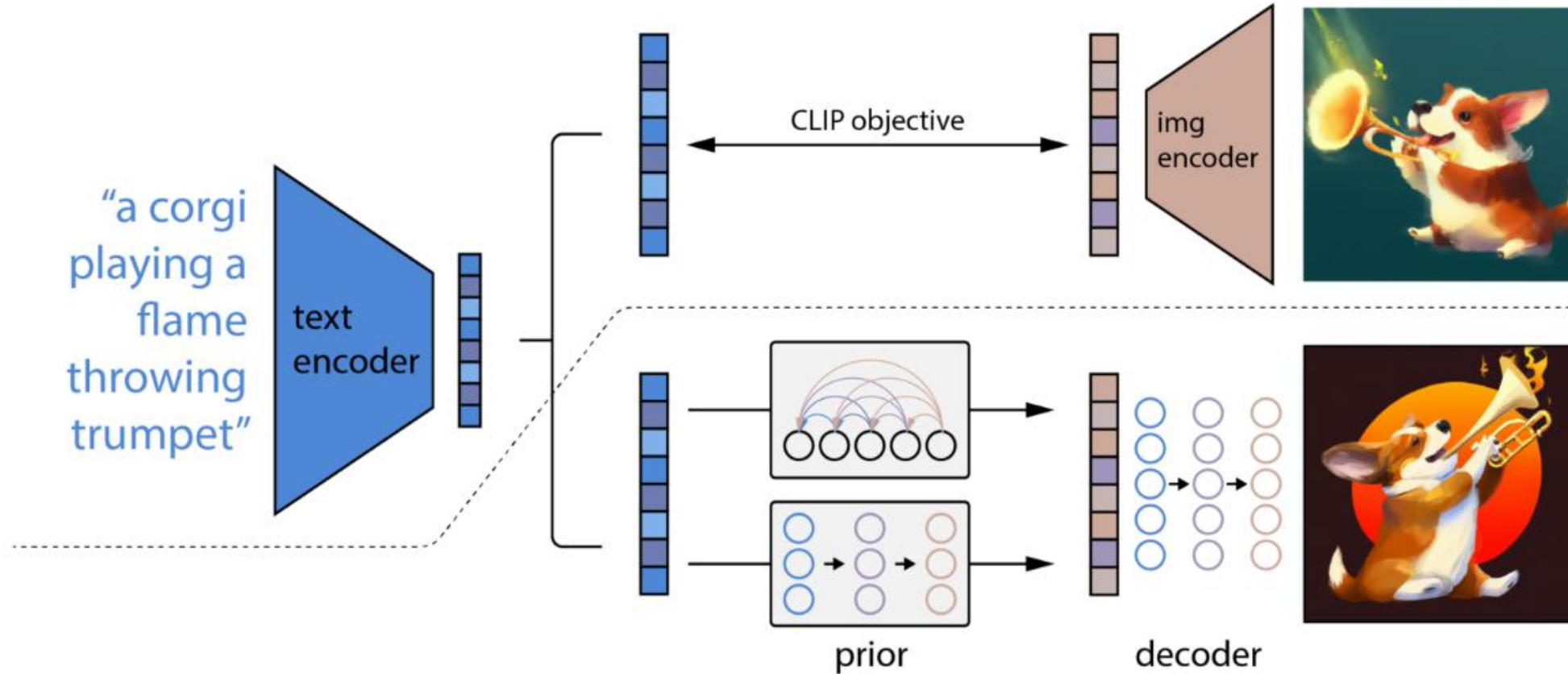
VS

Semantic Search

```
{
  Get {
    JeopardyQuestions {
      nearText: { concepts: ["animals"] }
    } { question }
  }
}
```

임베딩 : 유사도 비교 가능 (Q: 무엇의 유사도?, 스타일? 콘텐츠?)





- CLIP : Contrastive Language-Image Pre-training
- GLIDE : Guided Language to Image Diffusion for Generation and Editing

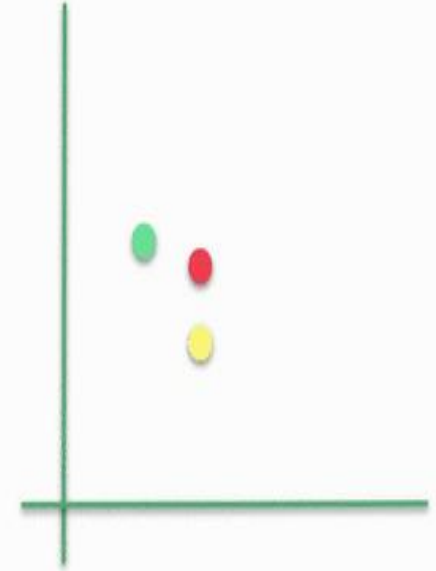
Multi-Modal 임베딩 : CLIP



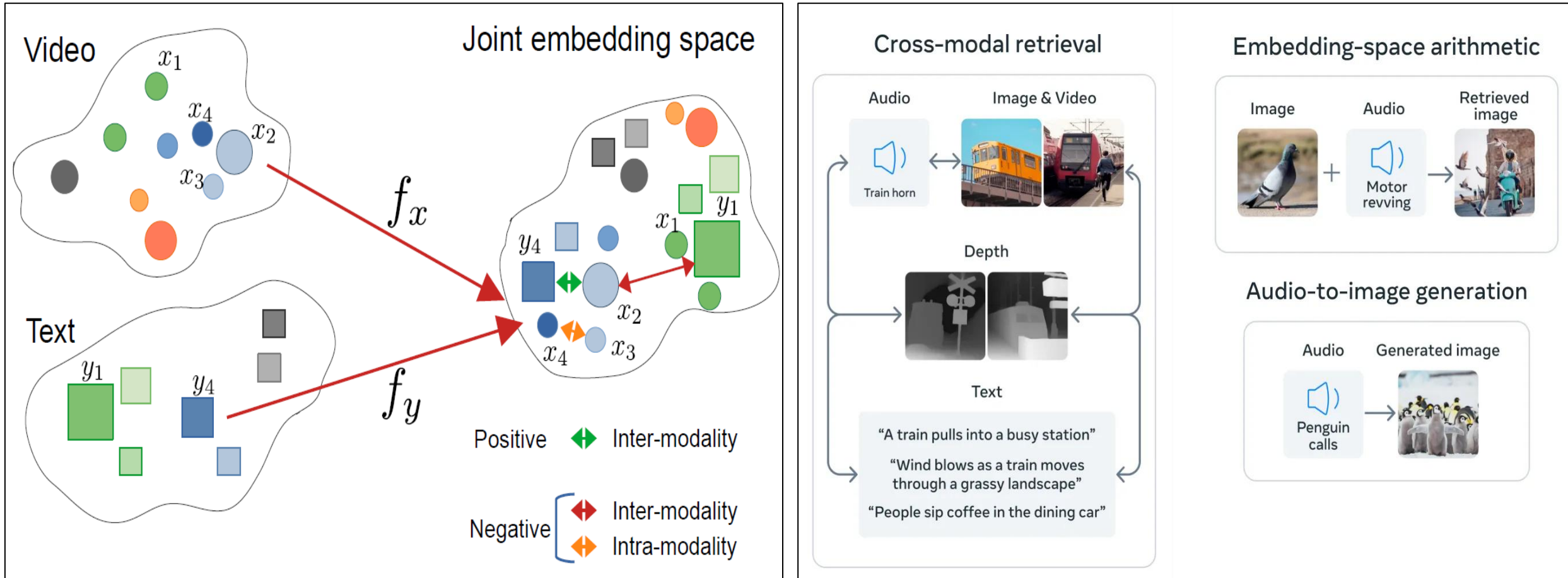
Two people on the mountain



Two people on the mountain



Multi-Modal 임베딩 : Joint embedding 공간 활용

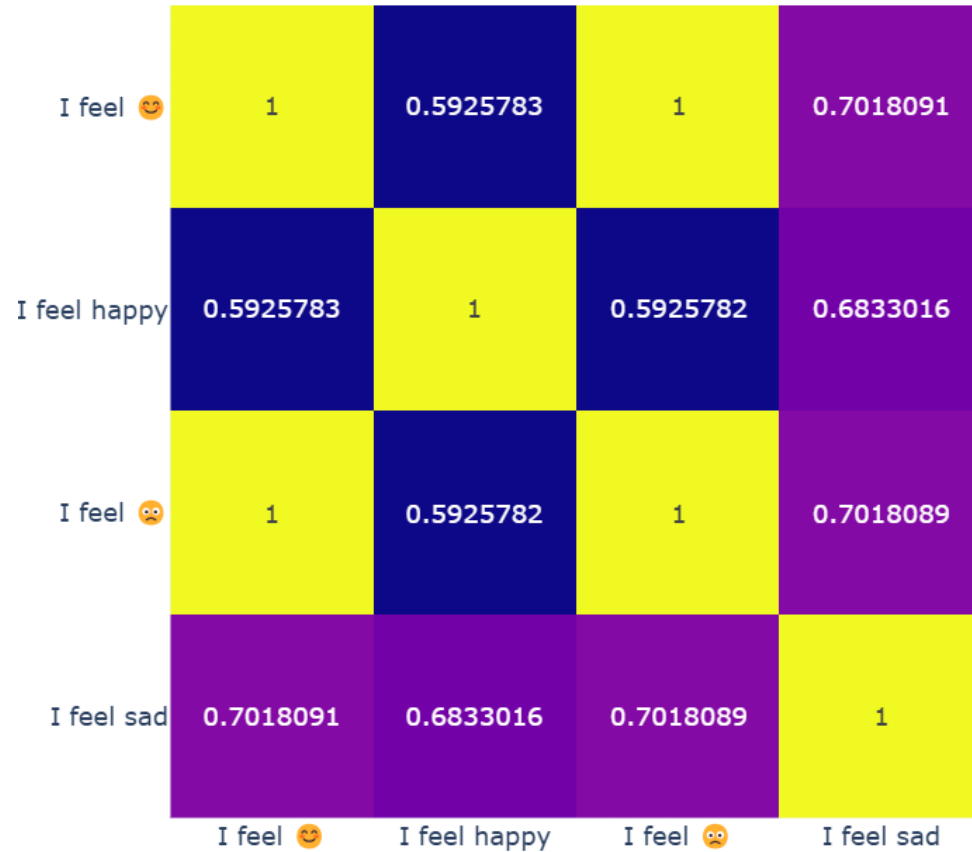


출처 : <https://weaviate.io/blog/multimodal-rag>

모델 카드 : <https://huggingface.co/openai/clip-vit-base-patch32>

실습 링크 : https://colab.research.google.com/drive/1hF_XvLhgP5UUjVlztLyGT8nn8kII_SnHB?usp=sharing

Tokenizer와 Embedding에 의한 유사도 비교



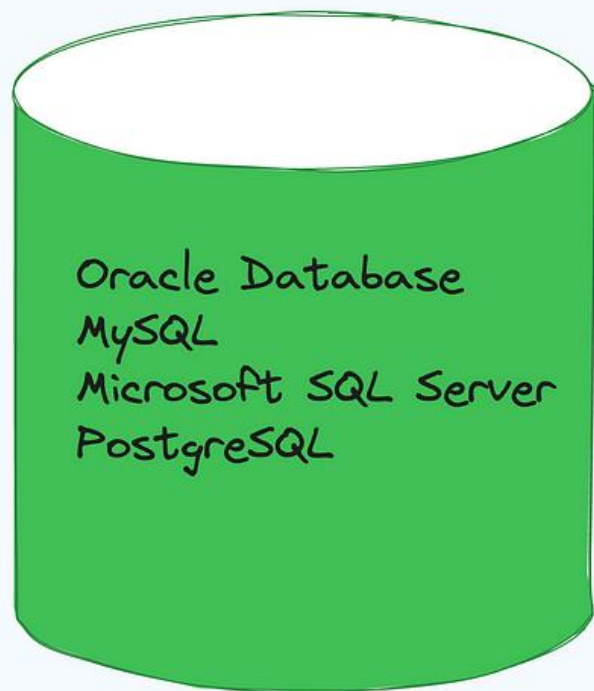
- Tokenizer 의 영향 : Unknown, 숫자, 날짜 , 오타

실습 링크 : <https://colab.research.google.com/drive/1q1y5g5QHmb2O6FYYjc3OEekMSyHSQo7f?usp=sharing>

벡터 데이터베이스

(ANN : Approximate Nearest Neighbors)

Databases



RDBMS

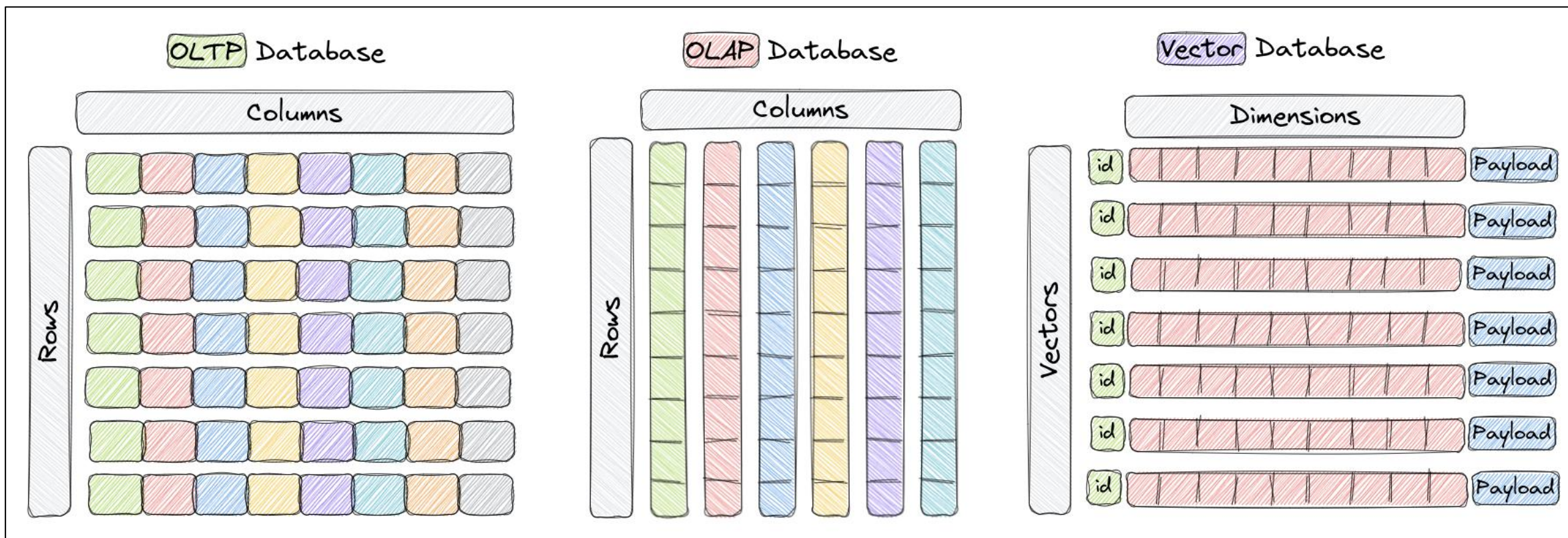


NOSQL



Vector

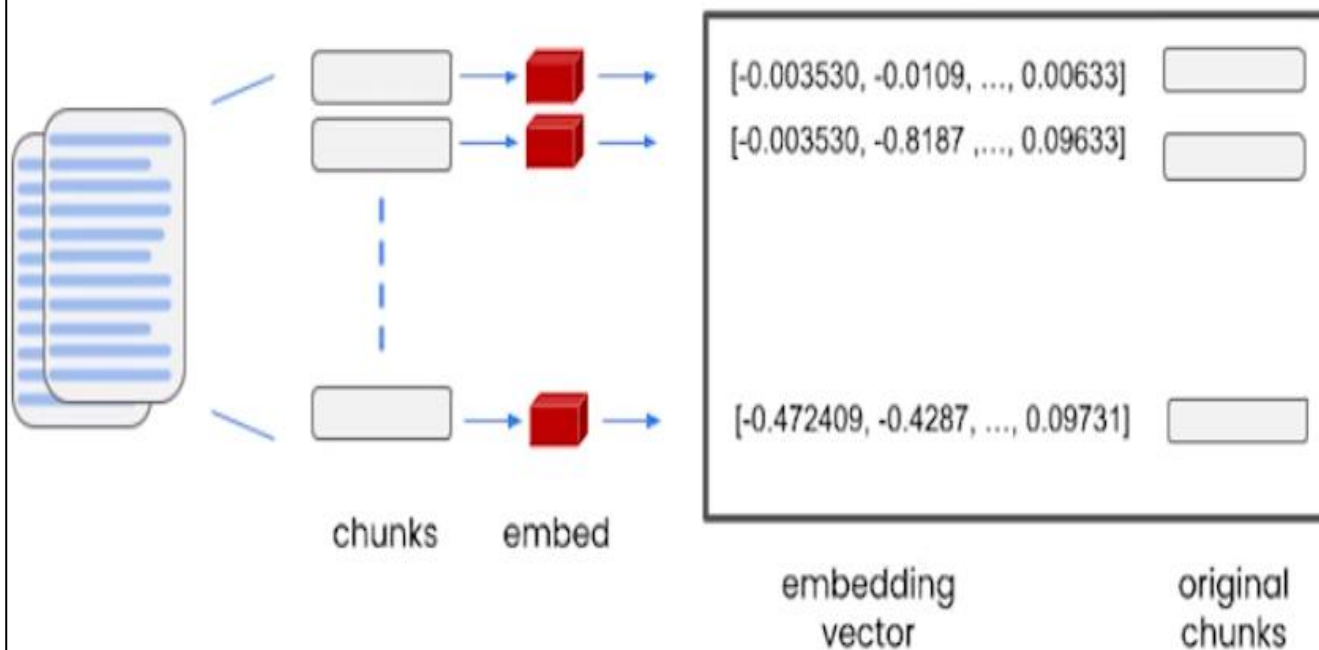
데이터베이스 : 대표적인 제품



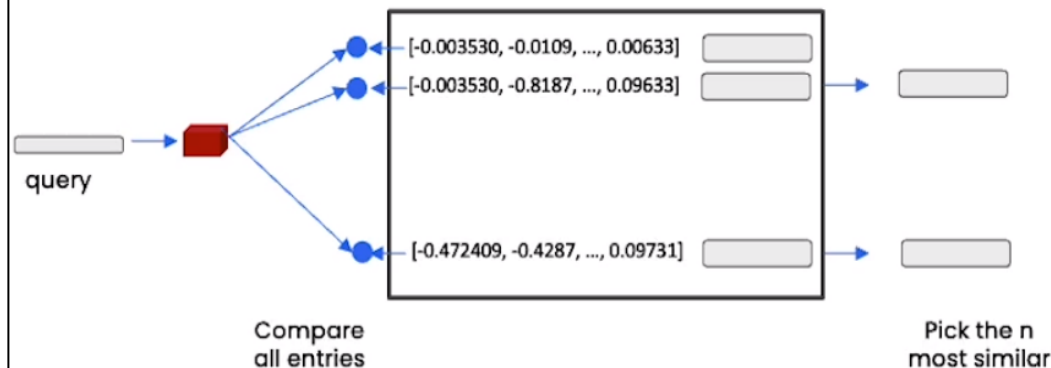
출처 : <https://qdrant.tech/documentation/overview/>

Vector Database

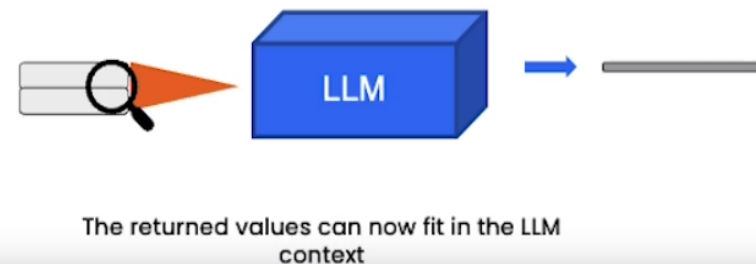
create

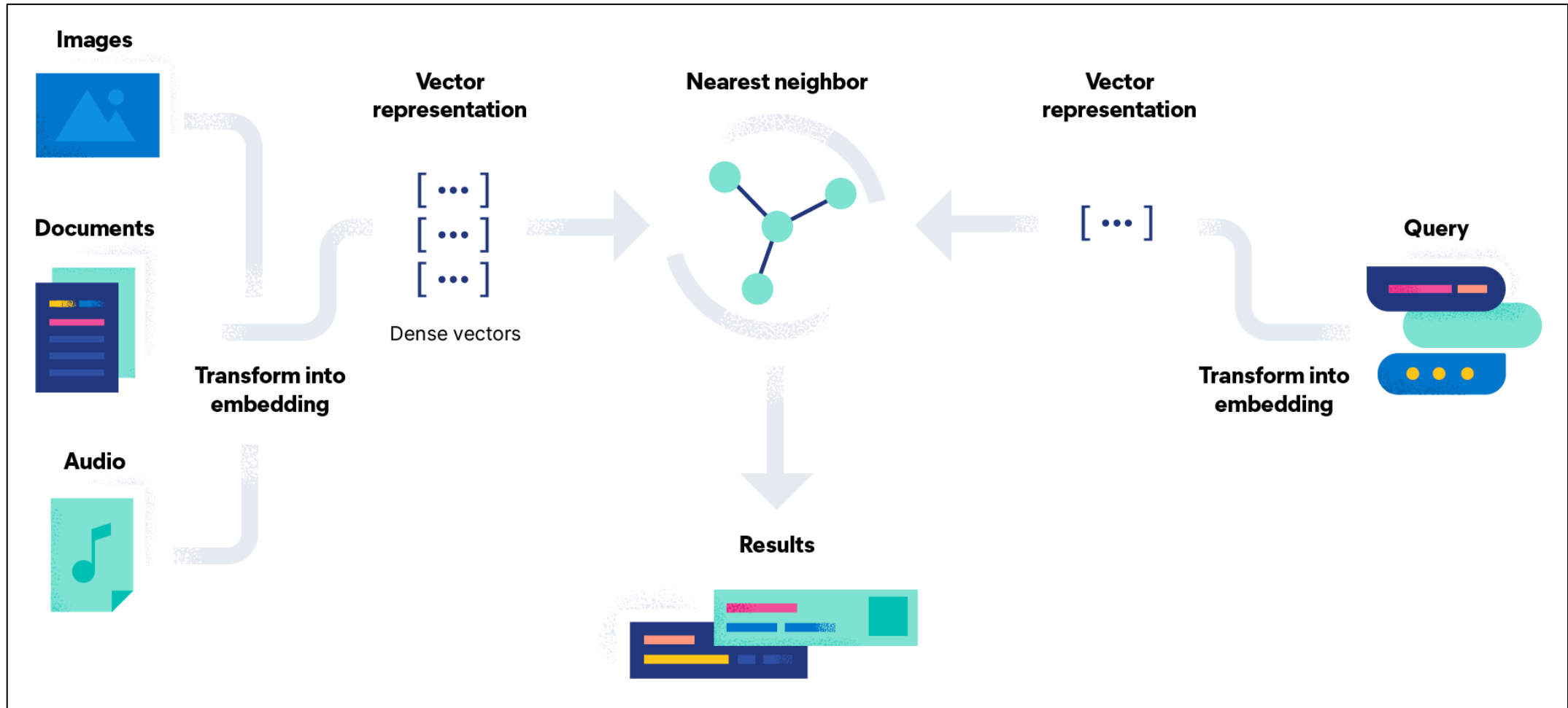


index



Process with Llm

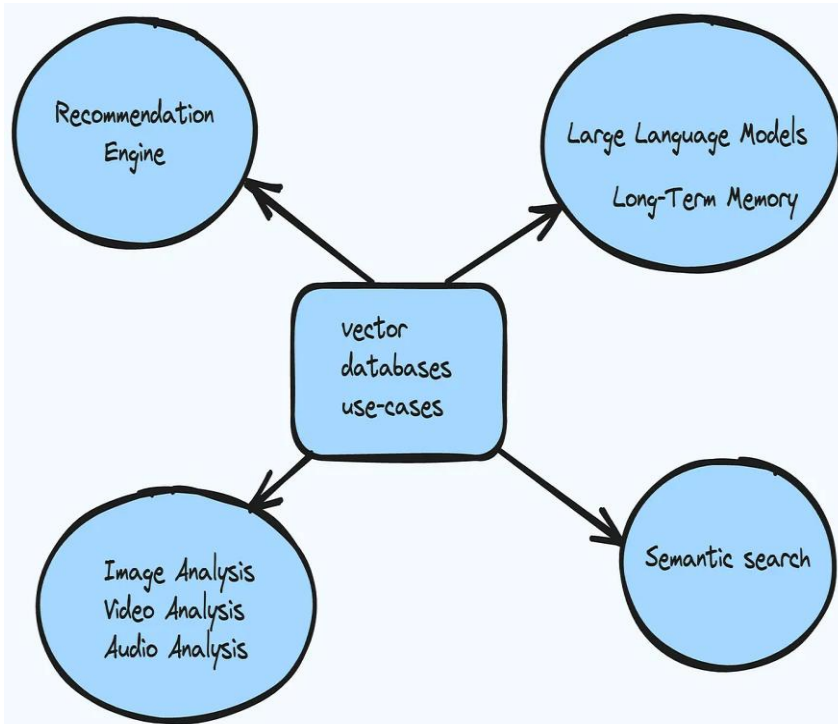




출처 : <https://www.elastic.co/kr/what-is/vector-database>

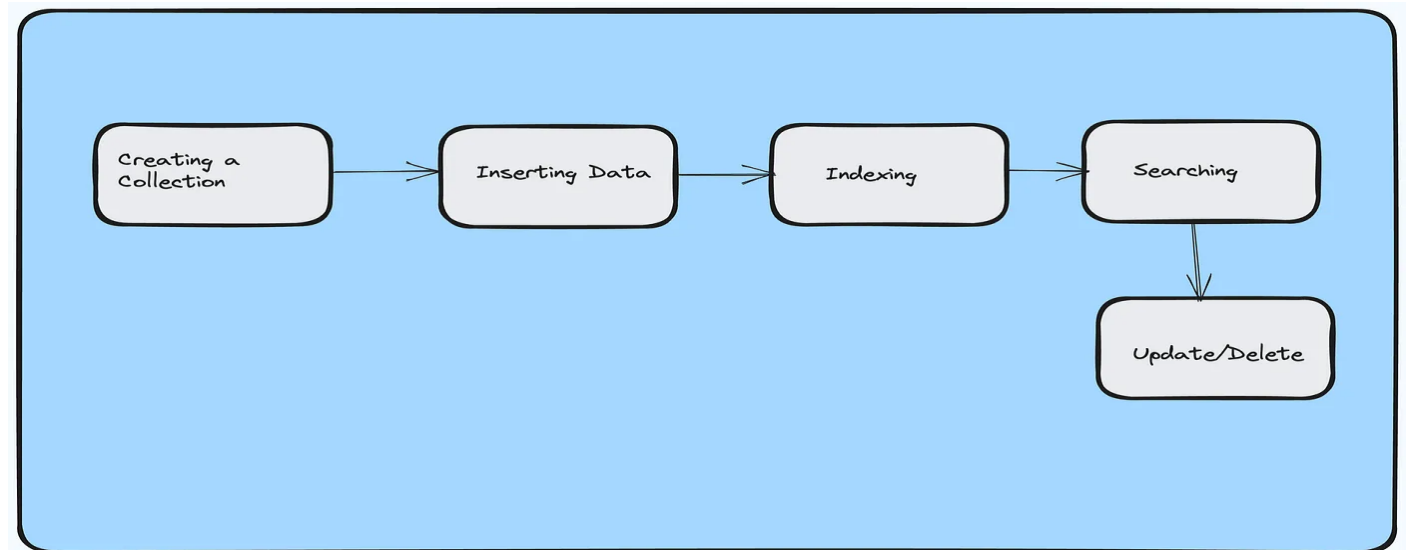
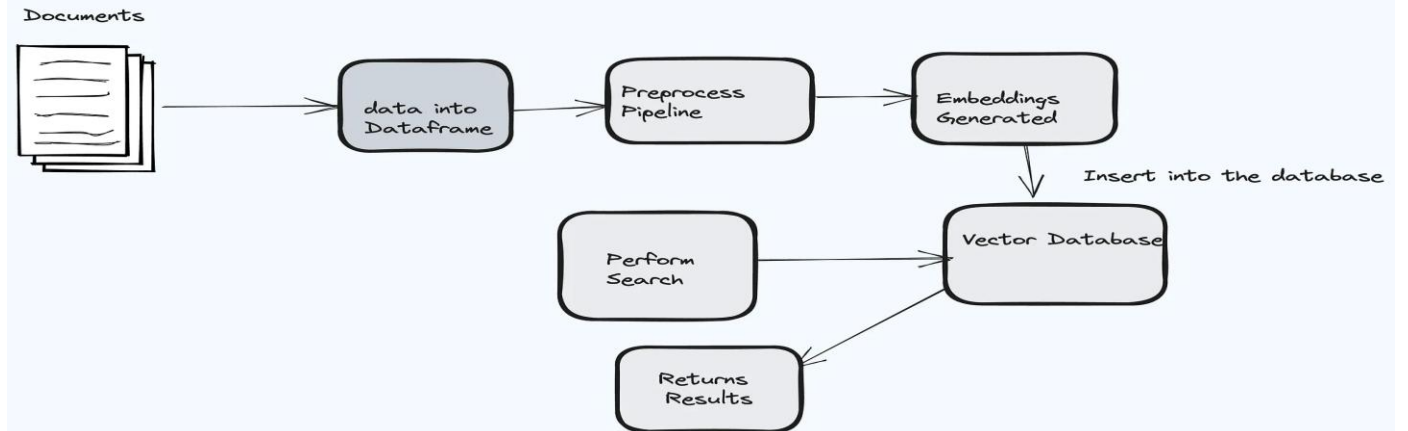
- 목표 : 대량의 벡터 모임에서 빠르고, 정확한 검색 제공 (빠른 인덱스 생성)

벡터 데이터베이스 : 활용 영역



Vector Databases Usecases

Vector Database Workflow for a Text Embeddings



벡터 데이터베이스 : 기능 및 대표적인 Vector DB



- 복잡한 비정형 데이터 처리
- 유사도 검색
- 확장성
- 실시간 처리
- 유연한 데이터 모델링
- 높은 연동성
- ML/AI 지원

☐	Vector Databases ▾	Python API ▾	Java API ▾	Other API's
1	Milvus	Python SDK	Java SDK	Go SDK, N
2	Pinecone	Python SDK	No	No
☐ ↗	Faiss (Facebook AI Similari...	Python SDK	No	C++
4	Weaviate	Python SDK	Java SDK	Go SDK
5	Qdrant	Python SDK	No	Rust SDK



swyx

@swyx · Follow



\$235m has been invested into Vector Databases in the past year:

- @qdrant_engine - \$7.5m Seed

- @tryChroma - \$18M Seed

- @weaviate_io - \$50m Series A

- @milvusio - \$60m Series B

- @Pinecone - \$100m Series B

For reference, MongoDB raised \$300m from start to \$1.2b IPO.

4:50 PM · Apr 28, 2023



247



Reply



Share

Read 14 replies

벡터 데이터베이스 : Open vs Commercial



Feature	Open Source (e.g., FAISS)	Commercial (e.g., Milvus)
Cost	Mostly free, with potential hosting or infrastructure costs	Tiered pricing, potential for steep costs with scale
Customizability	Full access to modify and adapt	Limited by API and feature set provided
Community Support	Large, active community; frequent updates	Depends on vendor; may be limited
Transparency	Full visibility into source code and algorithms	Often a black-box system



Enable the extension (do this once in each database where you want to use it)

```
CREATE EXTENSION vector;
```



Create a vector column with 3 dimensions

```
CREATE TABLE items (id bigserial PRIMARY KEY, embedding vector(3));
```



Insert vectors

```
INSERT INTO items (embedding) VALUES ('[1,2,3]'), ('[4,5,6]');
```



Get the nearest neighbors by L2 distance

```
SELECT * FROM items ORDER BY embedding <-> '[3,1,2]' LIMIT 5;
```



Also supports inner product (<#>), cosine distance (<=>), and L1 distance (<+> , added in 0.7.0)

Note: <#> returns the negative inner product since Postgres only supports ASC order index scans on operators

벡터 컬럼을 포함한 테이블 생성

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,  
  content TEXT,  
  author_id BIGINT,  
  embedding VECTOR(1538)  
);
```

HNSW 알고리즘으로 인덱스 생성 후,
코사인 유사도가 높은 5개 문서 조회

```
CREATE INDEX ON documents USING hnsw (embedding vector_cosine_ops);  
  
--find the closest 5 elements to a given query  
SELECT * FROM documents ORDER BY embedding <=> '[10.5, 11.0,...]' LIMIT 5;.
```

다른 테이블과 Join 기능

```
WITH matching_docs as (  
  --find 5 closest matches  
  SELECT *  
  FROM documents  
  ORDER BY embedding <=> '[10.5, 11.0,...]'  
  LIMIT 5  
)  
SELECT d.content, a.first_name, a.last_name  
FROM matching_docs d  
INNER JOIN author a ON (a.id = d.author_id).
```

ChromaDB 사용법 : 벡터 collection



```
import chromadb
# setup Chroma in-memory, for easy prototyping. Can add persistence easily!
client = chromadb.Client()

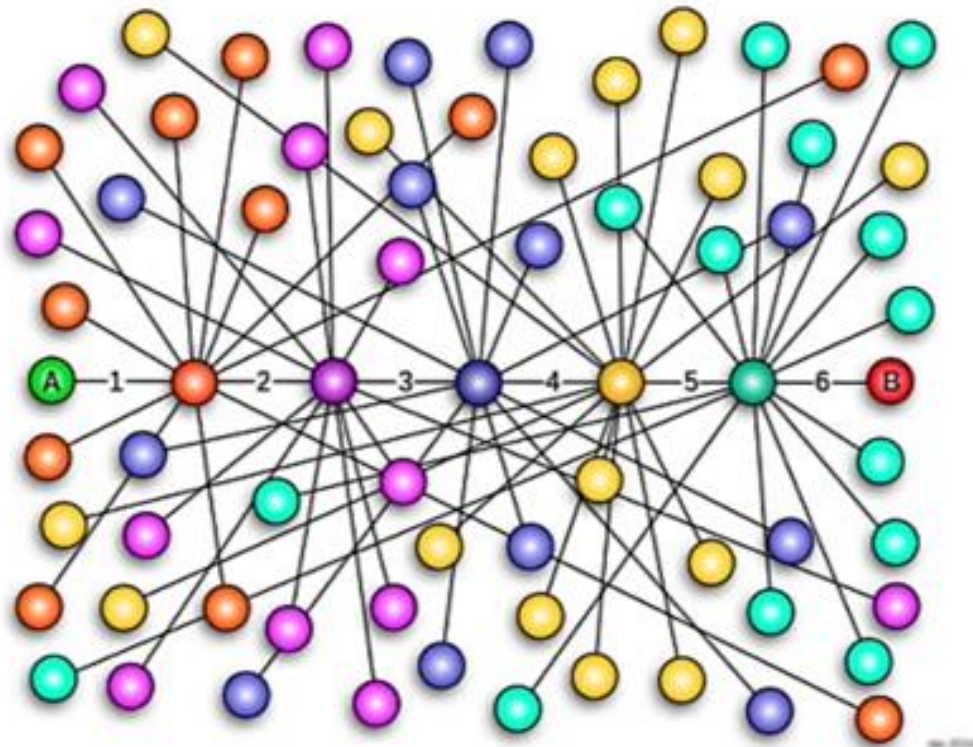
# Create collection. get_collection, get_or_create_collection, delete_collection also available!
my_db = client.create_collection("My_Test_Vector_DB")

# 임베딩 함수를 별도로 지정할 수도 있다.
# db = chroma_client.create_collection(name=name, embedding_function=embed_function)

# Add docs to the collection. Can also update and delete. Row-based API coming soon!
my_db.add(
    documents=docs, # we handle tokenization, embedding, and indexing automatically. You can skip
    metadatas=metas, # filter on these!
    ids=doc_ids, # unique for each doc
)
```

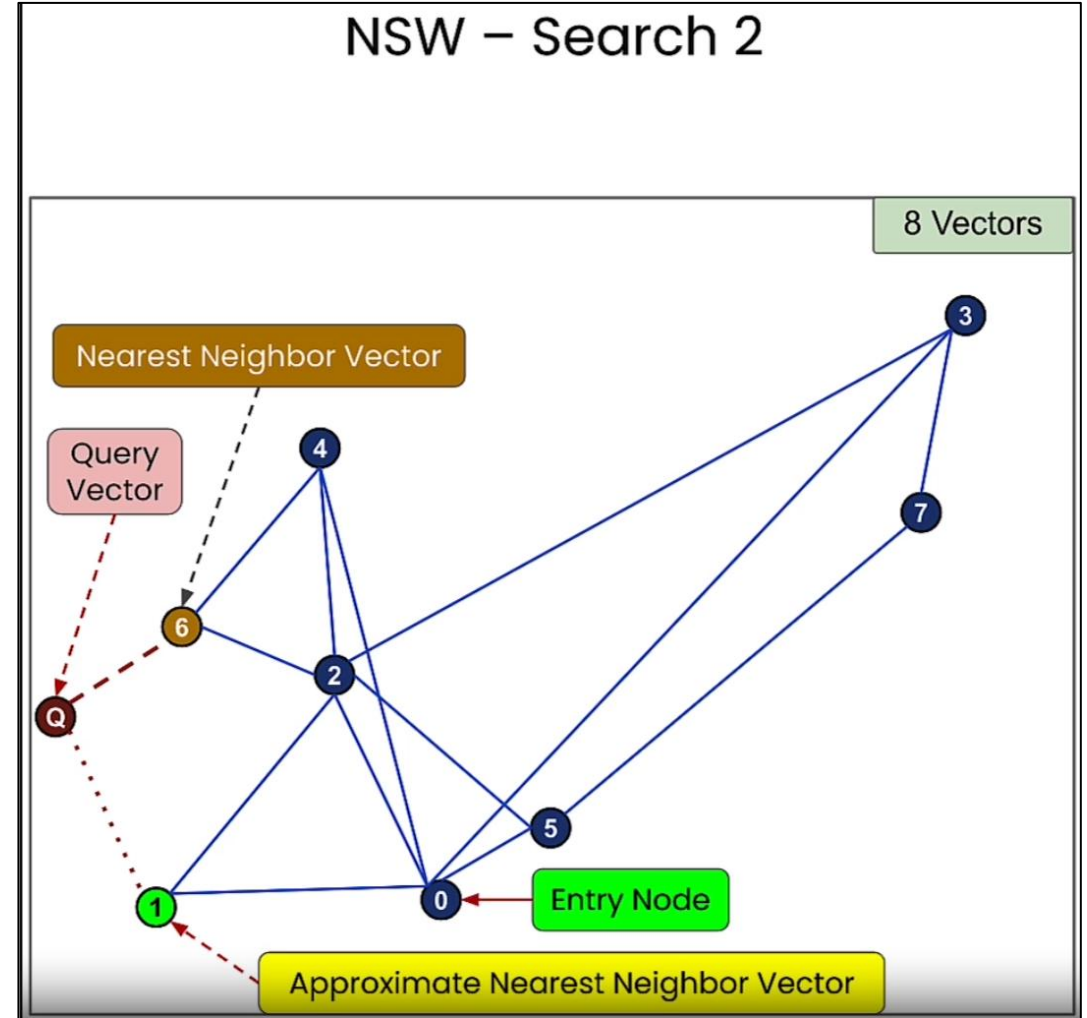
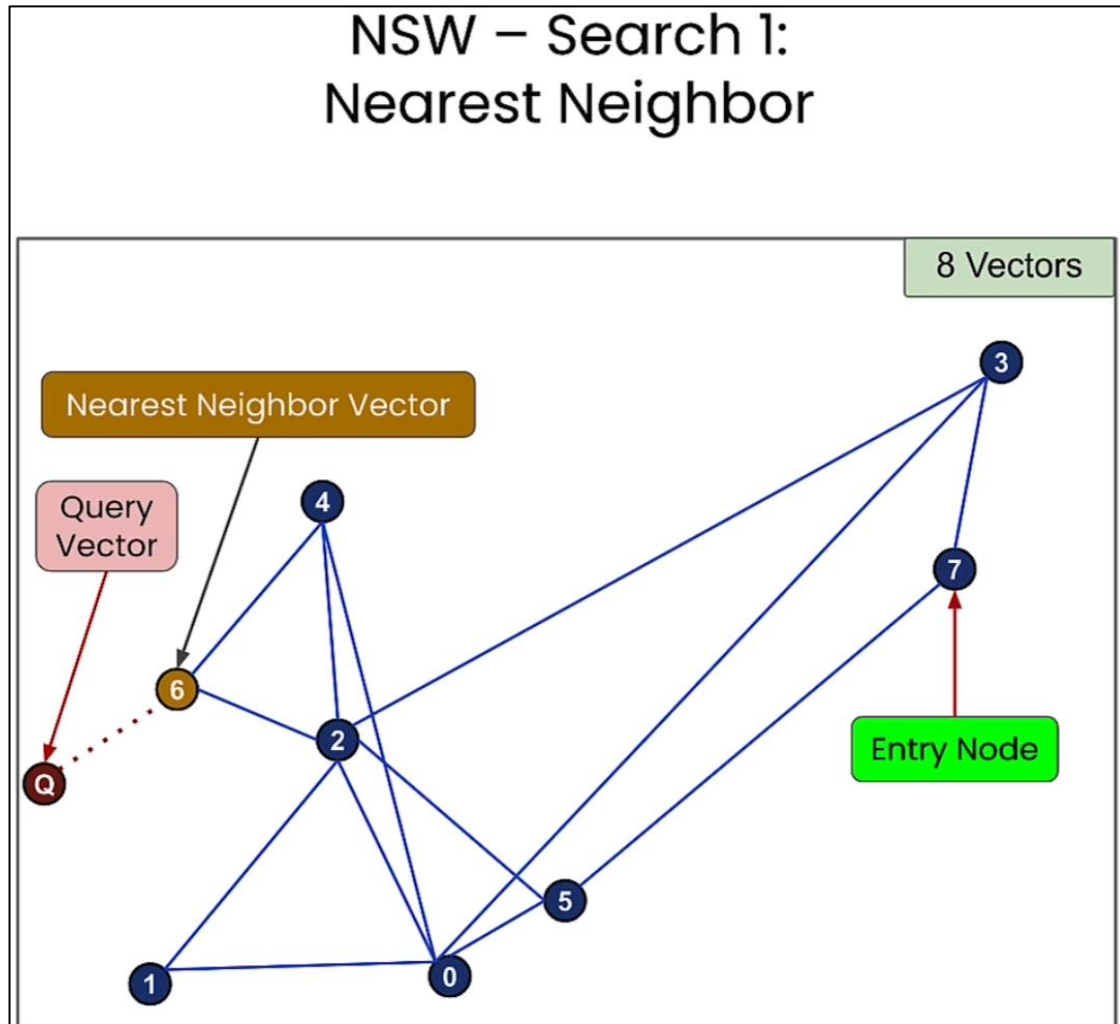
```
# Query/search 2 most similar results. You can also .get by id
results = my_db.query(
    query_texts=[str_query],
    n_results=2,
    # where={"metadata_field": "is_equal_to_this"}, # optional filter
    # where_document={"$contains": "search_string"} # optional filter
)
```

small world



- 인간 사회 네트워크 관찰 → 모든 사람이 밀접하게 연결됨
- 여섯 다리만 건너면 아는 사이 (Six Degrees of Separation)
 - 어떤 두 개인도 평균적으로 여섯 단계의 지인 관계를 통해 연결
- 높은 군집성 (High Clustering)
 - 노드들이 밀접한 그룹을 형성하며, 그룹 내 구성원들이 서로 연결되는 특징
- 짧은 경로 길이 (Short Path Lengths)
 - 높은 군집성이 있음에도 불구하고, 두 노드를 연결하는 데 필요한 단계 수가 적음

ANN : 대략 유사한 문서 찾기 - NSW 예



출처 : https://github.com/ksm26/vector-databases-embeddings-applications/blob/main/images/3_8.png

항상 가장 정확한 문서를 찾을 수는 없다

대표적인 ANN 라이브러리 : FAISS, ScaNN, DiskANN



FAISS : 많은 책의 처음 몇 페이지를 기억하는 사서. 속도와 정확성의 균형. 중간 규모의 컬렉션(수백만 권의 책)에 적합

ScaNN : 수십억 권의 책이 있는 도서관 사서. 주제와 스타일별로 책을 그룹화. 검색 범위를 그룹으로 좁힌 후 검색

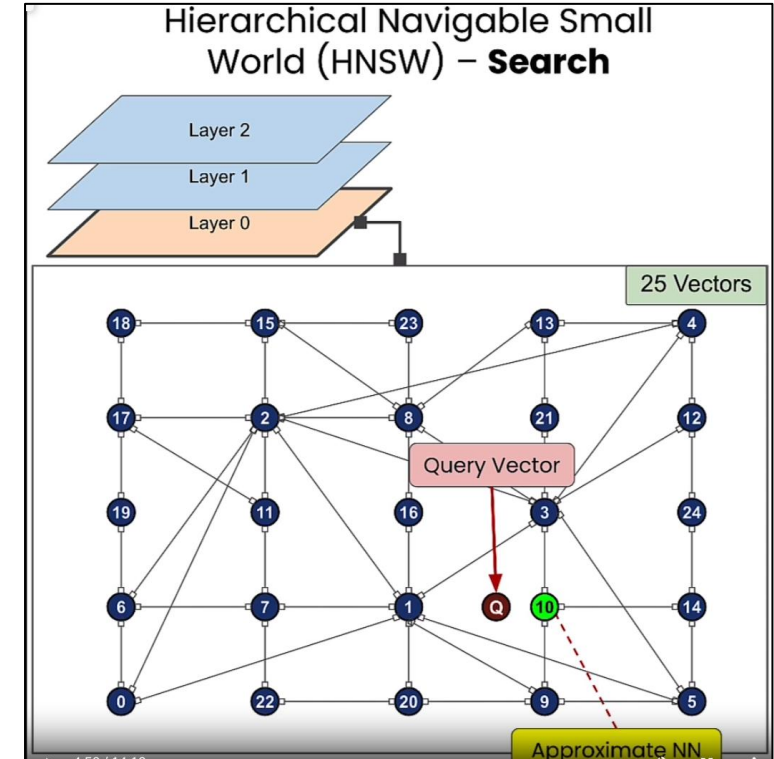
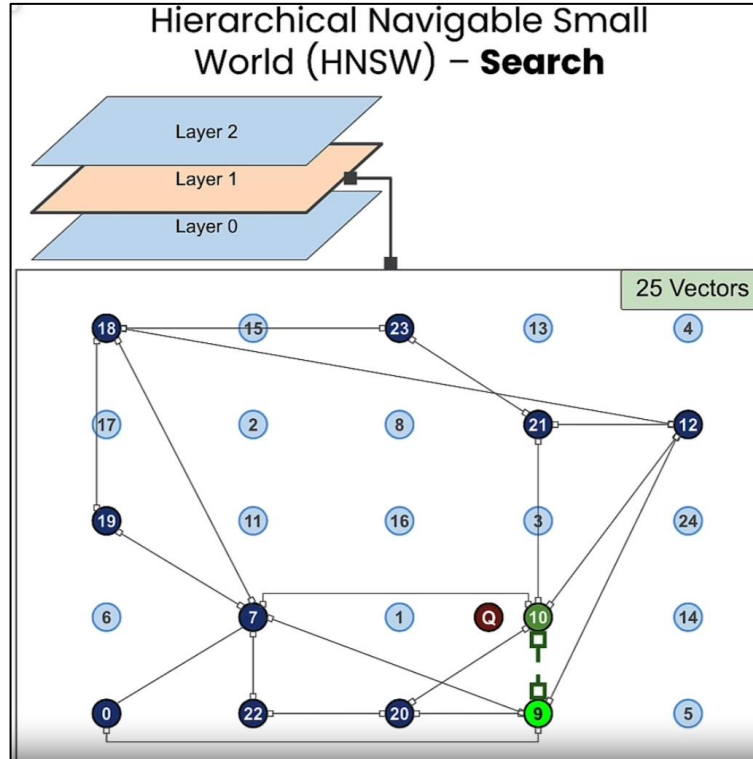
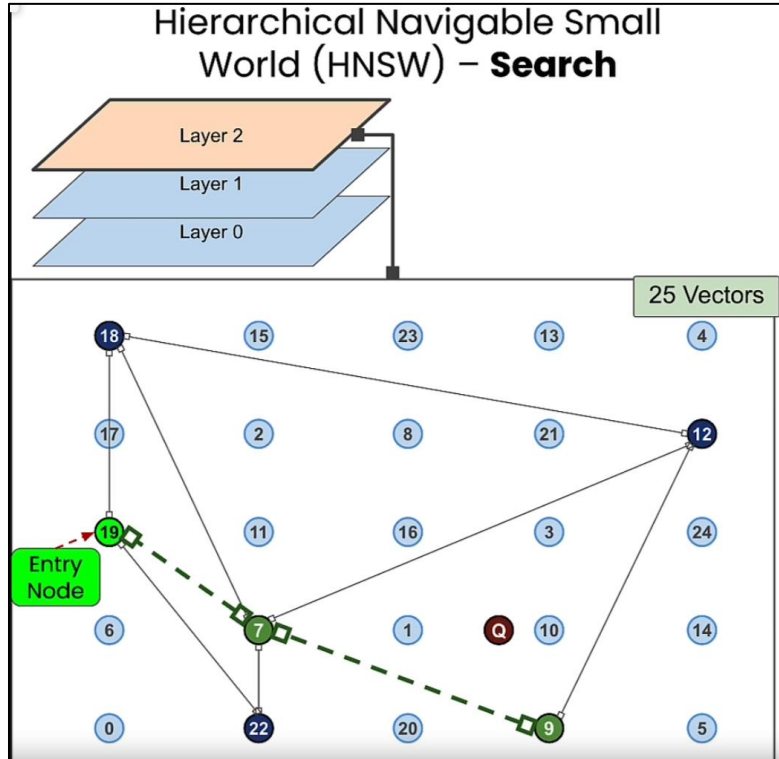
DiskNN : 자주 빌리는 책이 있는 작은 책장과 나머지를 위한 거대한 디지털 색인이 있는 사서.

대표적인 ANN 라이브러리 : FAISS, ScaNN, DiskANN



라이브러리	구현된 알고리즘	특징
FAISS	- Product Quantization (PQ)	벡터를 압축하여 메모리 사용량 감소 및 검색 속도 향상
	- Hierarchical Navigable Small World (HNSW)	그래프 기반 검색으로 높은 정확도와 빠른 검색 속도 제공
	- Inverted File Index (IVF)	대규모 데이터셋을 효율적으로 처리하기 위한 분할 기반 검색
	- Flat Index (IndexFlatL2)	정확한 검색을 위한 기본적인 인덱스 구조
ScaNN	- Tree-based Partitioning + Asymmetric Distance Computation (ADC)	트리 기반 파티셔닝과 비대칭 거리 계산으로 빠르고 정확한 근사 최근접 이웃(ANN) 검색
	- Optimized Quantization	벡터 양자화를 최적화하여 메모리 효율성 증가 및 속도 향상.
DiskANN	- Disk-based Hierarchical Navigable Small World (Disk-HNSW)	HNSW를 디스크 기반으로 확장하여 대규모 데이터셋 처리 가능
	- Graph-based Search	그래프 탐색을 활용하여 높은 성능과 확장성을 제공.

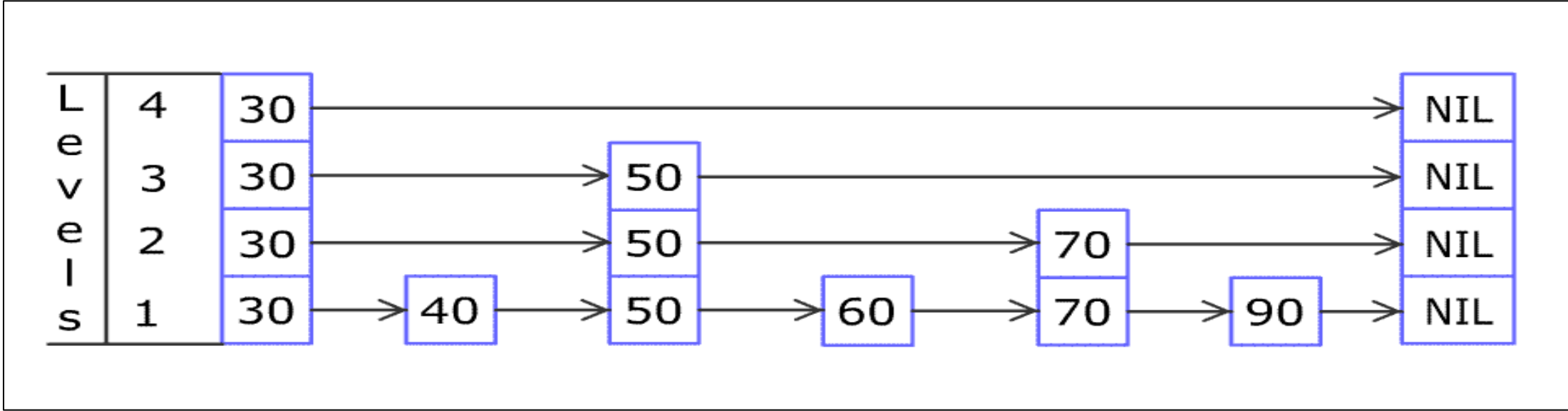
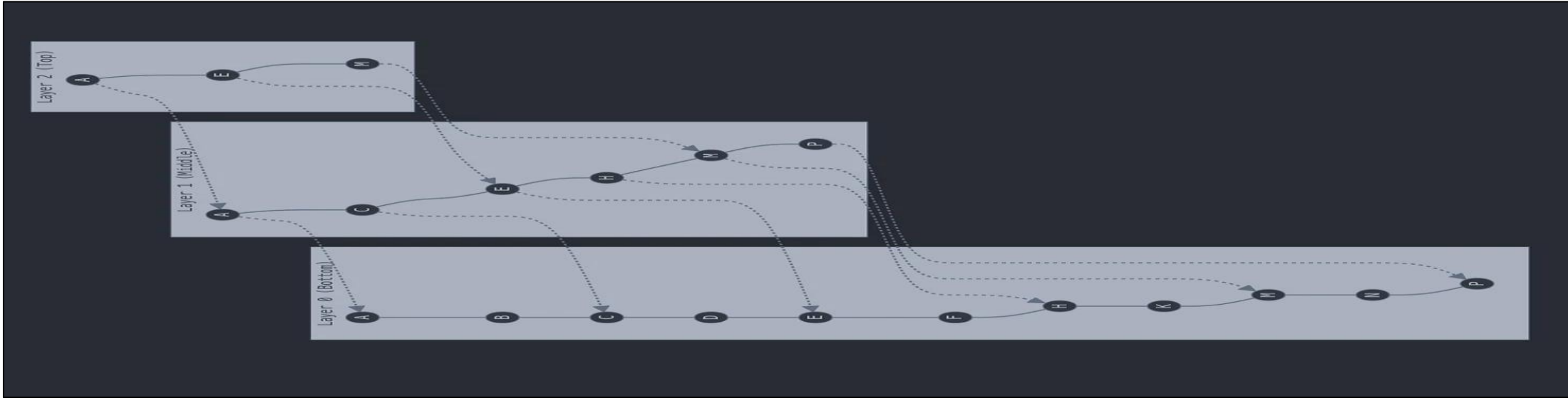
대표적인 알고리즘 : HNSW



출처 : https://github.com/ksm26/vector-databases-embeddings-applications/blob/main/images/3_8.png

실습 링크 : https://colab.research.google.com/drive/1-O_B_WzuU5vzlhjo6_AQ_1BUBtjygwsv?usp=sharing

대표적인 알고리즘 : HNSW



출처 : <https://phann123.medium.com/what-is-hnsw-hierarchical-navigable-small-world-49935be7fc05>

HNSW 수행 (HNSW Runtime)



•상위 레벨에서의 낮은 발견 확률

(Low likelihood in Higher Levels)

- HNSW 그래프의 상위 레벨에서 벡터가 발견될 확률이 지수적으로 감소

•로그 형태의 검색 시간 증가

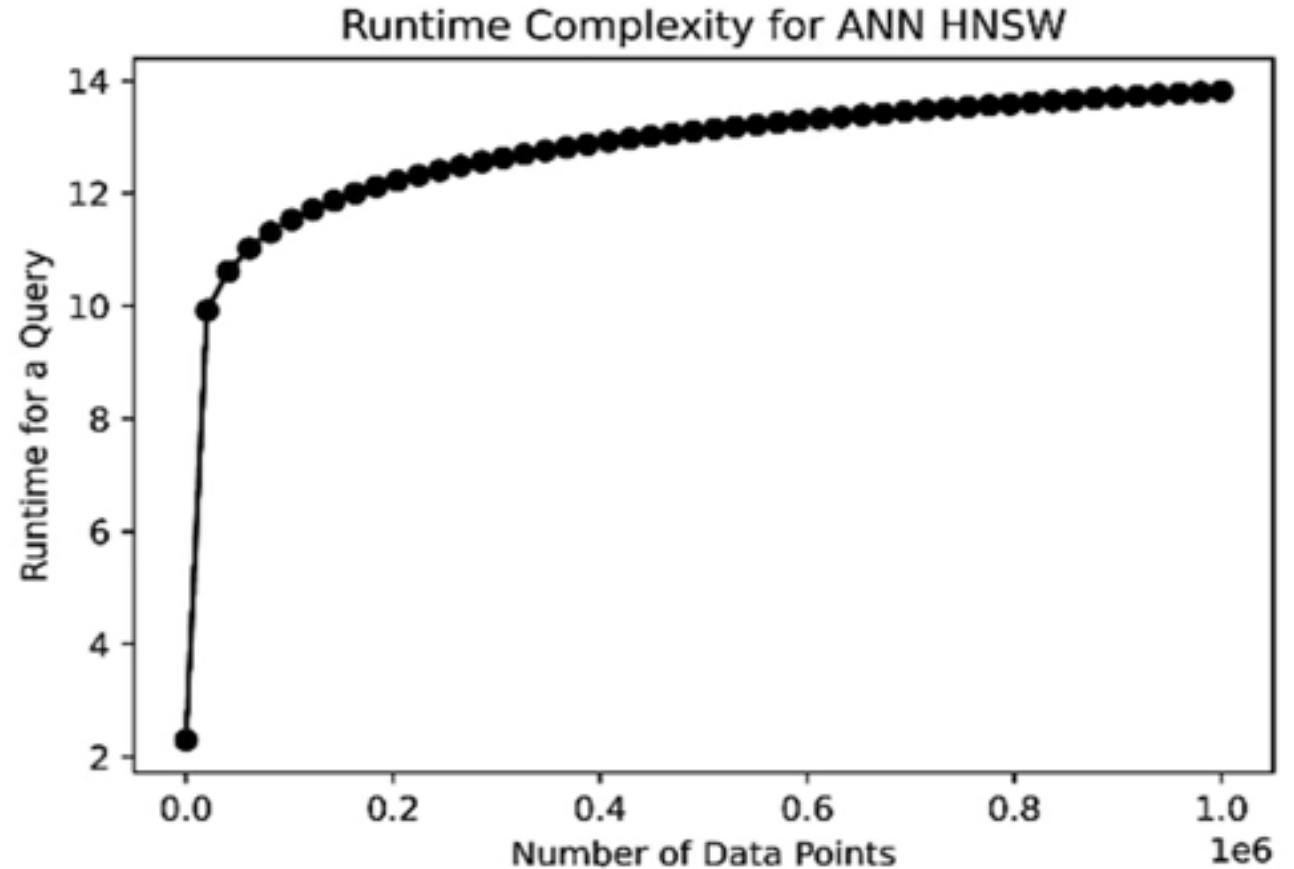
(Query time increases Logarithmically)

- 데이터 포인트 수가 증가할수록 벡터 검색을 위한 비교 횟수가 로그 형태로 증가

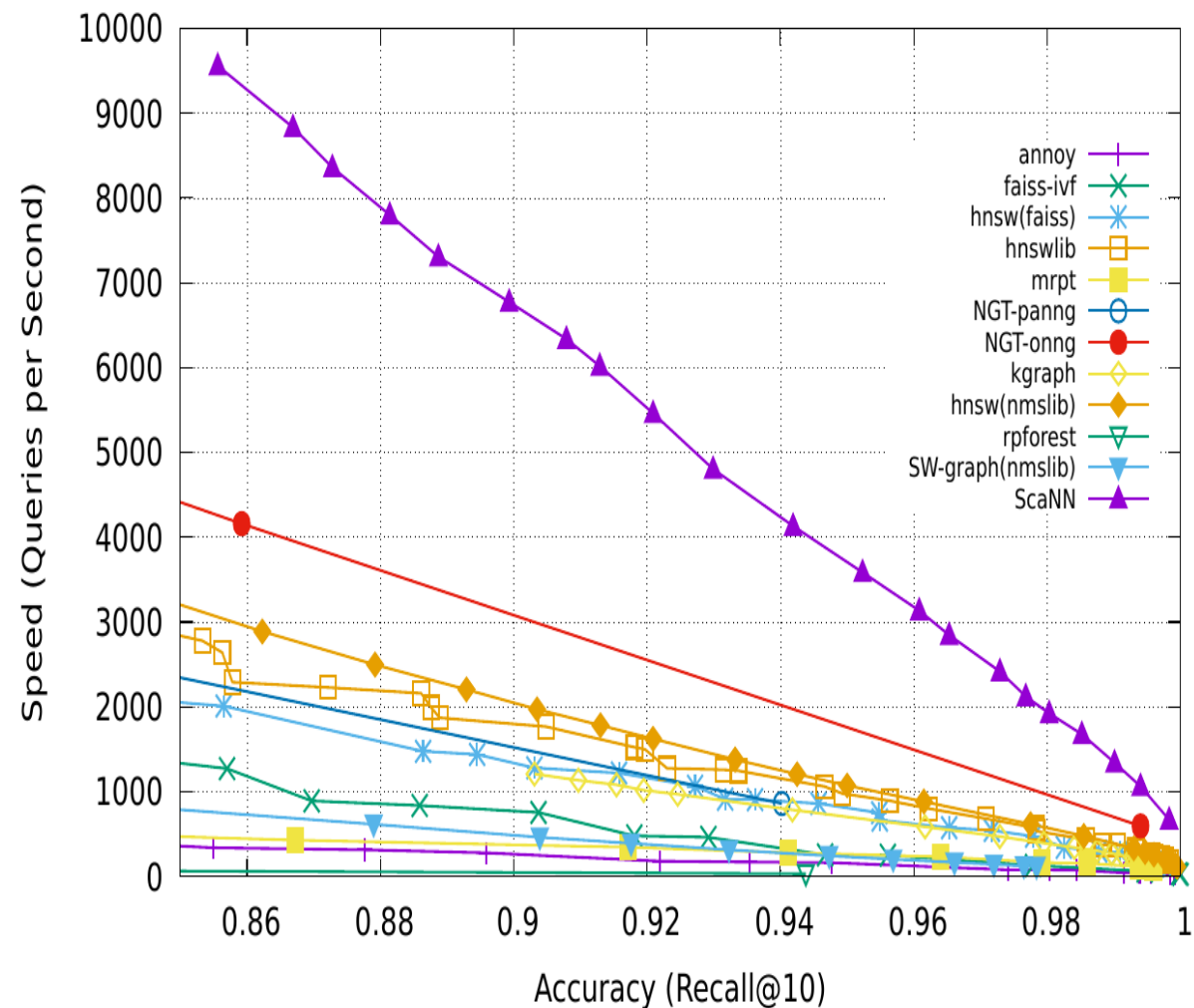
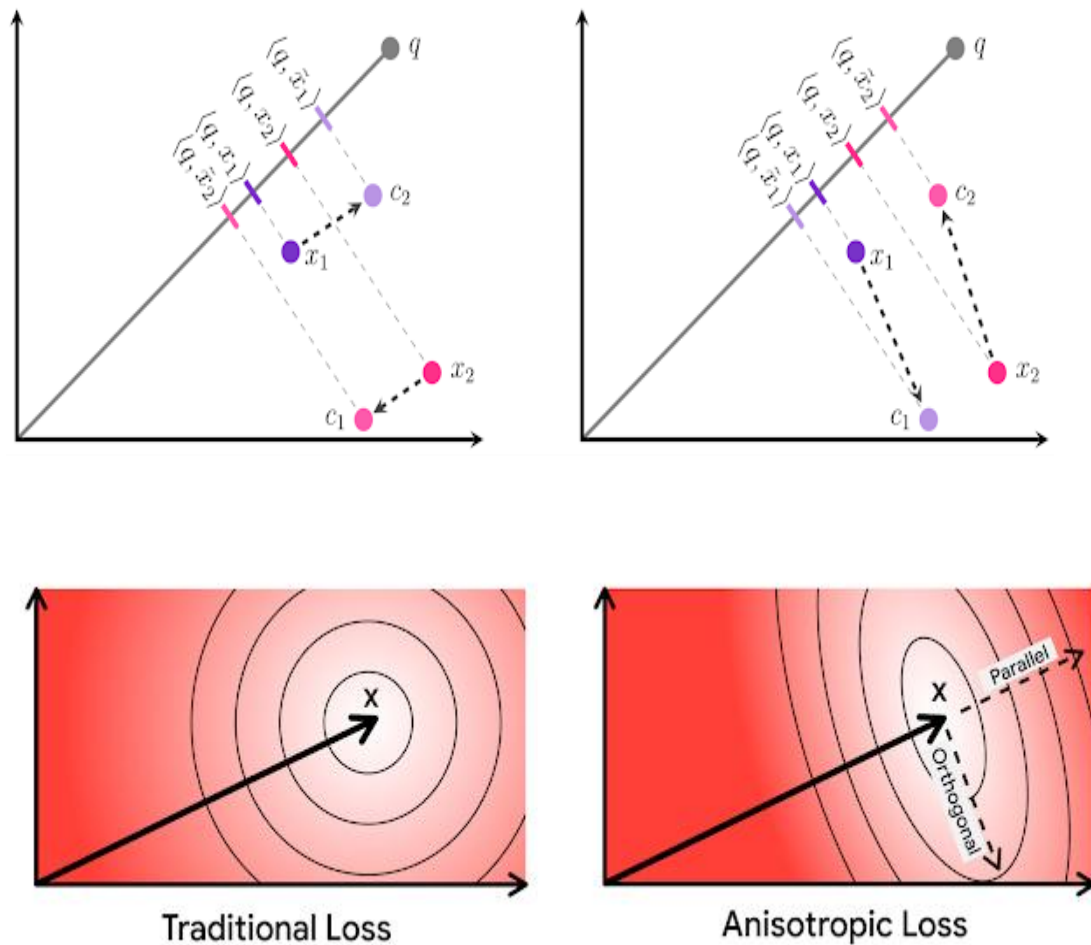
• $O(\log(N))$ 시간 복잡도

($O(\log(N))$ runtime complexity)

- 결과적으로 HNSW 알고리즘의 시간 복잡도는 $O(\log(N))$



대표적인 알고리즘 : ScaNN (Scalable Nearest Neighbors)



PQ



Recall

100%

50%

Speed

8.26ms → 1.49ms

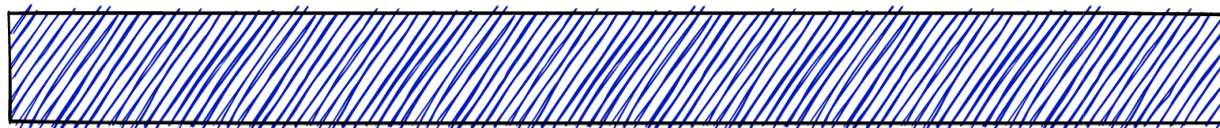
Memory

256MB → 6.5MB

- 벡터를 일정한 크기로 분할
- 부분을 대표하는 코드를 생성
- 코드북 생성
- 검색 벡터 분할 및 코드 검색
- 코드를 조합하여 벡터 생성
- 생성된 벡터를 기반으로 검색

Similarity Search

Product Quantization 101



Quantization basics

By default, **each dimension** of the vectors is represented by a **float32** value.

A **vector with 1536 dimensions**, e.g., OpenAI Ada embedding, **requires 6 kB of memory**.

#vectors	Memory usage
1	6 KB
1M	~6 GB
1B	~6 TB

Product Quantization

[	-0.25	-0.45	0.98	-0.62	[	-0.98	-0.76	0.08	0.57	]
[	0.16	0.71	0.82	0.20	[	0.79	0.67	-0.06	0.28	]
[	-0.81	0.45	-0.01	-0.17	[	0.72	-0.12	-0.89	-0.99	]
[	-0.30	-0.52	0.40	0.82	[	0.65	0.40	0.90	0.32	]



[	[	-0.25	-0.45	0.98	-0.62	]	[	-0.98	-0.76	0.08	0.57	]	]
[	[	0.16	0.71	0.82	0.20	]	[	0.79	0.67	-0.06	0.28	]	]
[	[	-0.81	0.45	-0.01	-0.17	]	[	0.72	-0.12	-0.89	-0.99	]	]
[	[	-0.30	-0.52	0.40	0.82	]	[	0.65	0.40	0.90	0.32	]	]

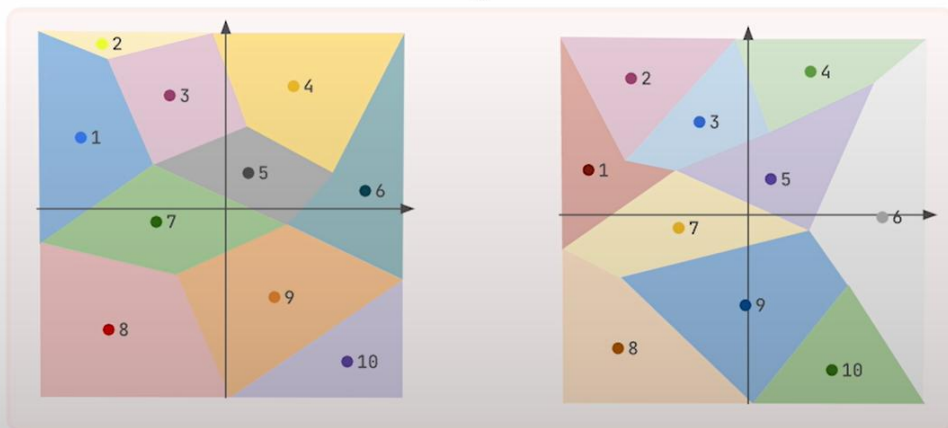
Original **vectors** are **divided into chunks**.

The higher the **number of chunks**, the lower the **compression rate**.

양자화(Quantization)

Product Quantization

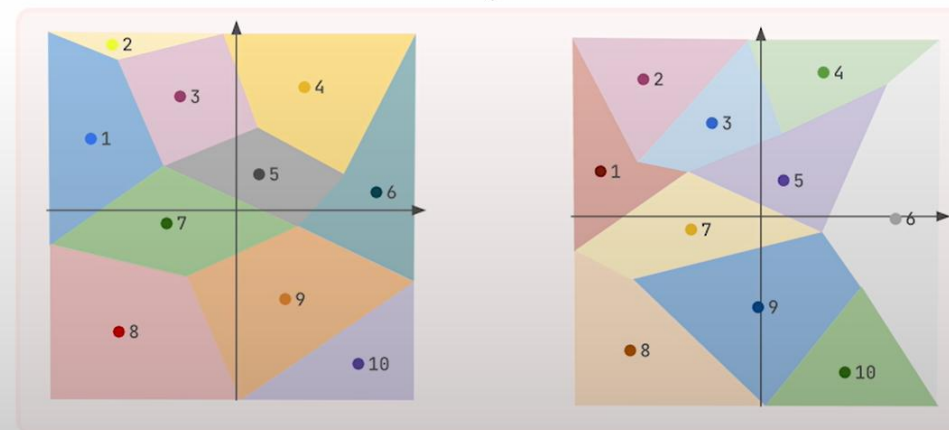
[	[	-0.25	-0.45	0.98	-0.62	]	[	-0.98	-0.76	0.08	0.57	]	]
[	[	0.16	0.71	0.82	0.20	]	[	0.79	0.67	-0.06	0.28	]	]
[	[	-0.81	0.45	-0.01	-0.17	]	[	0.72	-0.12	-0.89	-0.99	]	]
[	[	-0.30	-0.52	0.40	0.82	]	[	0.65	0.40	0.90	0.32	]	]



Every subvector might be now mapped to the closest centroid.

Product Quantization

[	[	-0.25	-0.45	0.98	-0.62	]	[	-0.98	-0.76	0.08	0.57	]	]
[	[	0.16	0.71	0.82	0.20	]	[	0.79	0.67	-0.06	0.28	]	]
[	[	-0.81	0.45	-0.01	-0.17	]	[	0.72	-0.12	-0.89	-0.99	]	]
[	[	-0.30	-0.52	0.40	0.82	]	[	0.65	0.40	0.90	0.32	]	]



[	[												]
[	[			3						7			]

Product Quantization impact

Product Quantization **increases the indexing time**, as we need to run clustering to find the centroids.

Its impact on the search latency has to be tested in a specific configuration.

	Full vectors	1D clusters	2D clusters	4D clusters	8D clusters
Mean precision	0.9837	0.9677	0.9143	0.8068	0.6618
Mean search time	2719 μ s	4134 μ s	2947 μ s	2175 μ s	2053 μ s
Compression	x1	x4	x8	x16	x32
Upload & indexing time	332 s	921 s	597 s	481 s	474 s

Dataset: arxiv-titles-384-angular-no-filters

Rescoring

When we search with quantized vectors, **many documents** may have the **same representation**.



Vector databases usually **keep the original vectors** on disk and load them **to rescore** the results.

[	[	3	]	[	7	]	]						
													
[	[	-0.25	-0.45	0.98	-0.62	]	[	-0.98	-0.76	0.08	0.57	]	]
[	[	-0.81	0.45	-0.01	-0.17	]	[	0.72	-0.12	-0.89	-0.99	]	]

Scalar Quantization

The simplest and the most obvious idea to reduce the memory requirements is to convert each 4-byte floating point number to a 1-byte integer.

[	-0.25	-0.45	0.98	-0.62	-0.98	-0.76	0.08	0.57	]
[	0.16	0.71	0.82	0.20	0.79	0.67	-0.06	0.28	]
[	-0.81	0.45	-0.01	-0.17	0.72	-0.12	-0.89	-0.99	]
[	-0.30	-0.52	0.40	0.82	0.65	0.40	0.90	0.32	]



[	-32	-58	128	-80	-127	-99	11	74	]
[	21	92	107	26	103	87	-8	37	]
[	-105	59	-1	-22	94	-15	-116	-128	]
[	-39	-67	52	107	85	52	117	42	]

Scalar Quantization impact

Scalar Quantization reduces both the **indexing and search time** but also comes with a **precision cost**.



	Full vectors	SQ enabled
Mean precision	0.989	0.986
Mean search time	0.0094 s	0.0037 s
Upload & indexing time	649 s	496 s

Dataset: arxiv-titles-384-angular-no-filters

Binary Quantization

To save memory even more, **converting floats into booleans** pushes the quantization to the limits.

[	-0.25	-0.45	0.98	-0.62	-0.98	-0.76	0.08	0.57	]
[	0.16	0.71	0.82	0.20	0.79	0.67	-0.06	0.28	]
[	-0.81	0.45	-0.01	-0.17	0.72	-0.12	-0.89	-0.99	]
[	-0.30	-0.52	0.40	0.82	0.65	0.40	0.90	0.32	]



[	0	0	1	0	0	0	1	1	]
[	1	1	1	1	1	1	0	1	]
[	0	1	0	0	1	0	0	0	]
[	0	0	1	1	1	1	1	1	]

```
from raxx import compare

compare(
    qrels=qrels,
    runs=[
        product_name_pq_run,
        product_name_pq_rescore_run,
        product_name_sq_run,
        product_name_sq_rescore_run,
        product_name_bq_run,
        product_name_bq_rescore_run,
    ],
    metrics=["precision@25"],
)
```

#	Model	P@25
---	-----	-----
a	product_name_pq	0.647
b	product_name_pq_rescore	0.827 ^{a e}
c	product_name_sq	0.994 ^{a b e f}
d	product_name_sq_rescore	0.994 ^{a b e f}
e	product_name_bq	0.784 ^a
f	product_name_bq_rescore	0.950 ^{a b e}

- 기본 임베딩 벡터 : float32 (4 바이트)
 - ✓ OpenAI Ada 임베딩 : 1535 차원 ➔ 6KB
 - ✓ 1M 데이터 : ~ 6GB, 1B 데이터 : ~6TB
- 벡터의 저장 공간, 인덱스 생성/검색 속도와 정확도에 영향
 - ✓ 검색 속도는 빨라지고, 정확도는 낮아짐
- Rescoring : 원래 벡터로 추가 검증
- 대표적인 양자화 방법
 - ✓ Product Quantization
 - ✓ Scalar Quantization
 - ✓ Binary Quantization

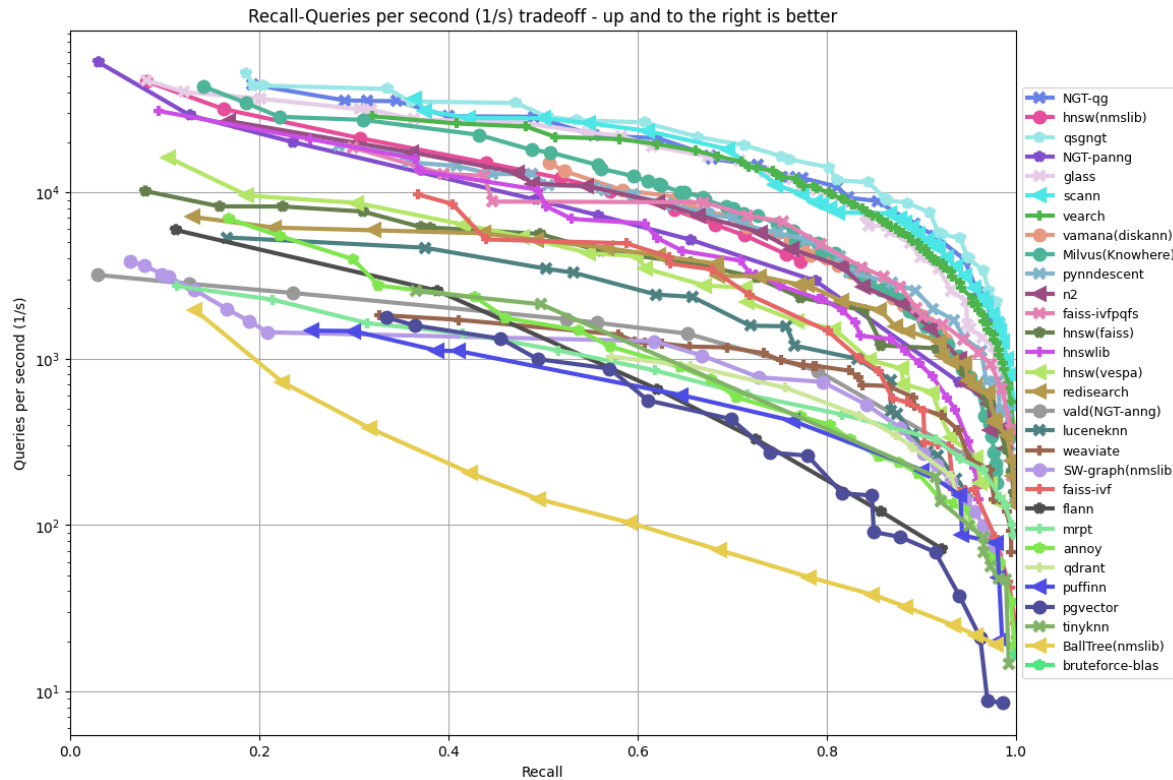
#	Model	P@25
---	-----	-----
a	product_name_pq	0.647
b	product_name_pq_rescore	0.827 ^{a e}
c	product_name_sq	0.994 ^{a b e f}
d	product_name_sq_rescore	0.994 ^{a b e f}
e	product_name_bq	0.784 ^a
f	product_name_bq_rescore	0.950 ^{a b e}

ANN(Approximate Nearest Neighbors) 벤치마크



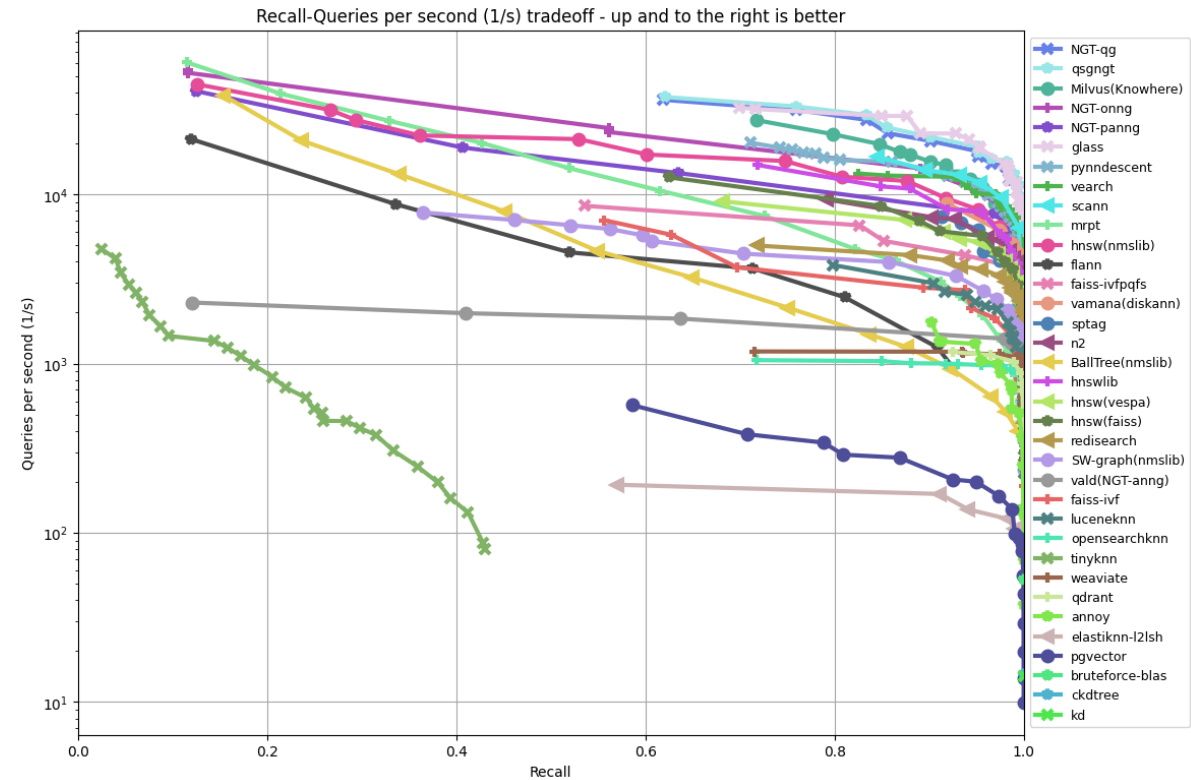
Distance: Angular

glove-100-angular (k = 10)



Distance: Euclidean

fashion-mnist-784-euclidean (k = 10)



출처 : <https://ann-benchmarks.com/>

벡터 데이터베이스 비교

Name	Free tier	Queries Per Second	Self-Host	Managed in Cloud	SOC-2	HIPAA	Open Source	License	BM25	Aggregations	Size of vectors dimension	Metadata Filtering	Time Based Metadata Filtering	Time-Series Compression	Hybrid Search	Website URL
Qdrant	Self-hosted is free	300 by ANN (Around 350 by FastEmbed)	Yes	Yes	Can be (Depending on hosting)	Can be (Depending on hosting)	Yes	Apache License 2.0	No But Similar	No	Qdrant does not have any hard limits	Yes	Somewhat (Need to convert time to an integer)	No	Yes (Sparse-Dense Vectors)	qdrant.tech
Weaviate	yes	518	Yes	Yes	Can be (Depending on hosting)	Can be (Depending on hosting)	Yes	Apache License 2.0	Yes	Yes	65535					
Pinecone	Yes	From Pinecone website (queries per second for 1M vectors of size 768; top_10): - s1 pod: 10 - p1 pod: 30 - p2 pod: 150	No	Yes	Yes	Yes	No	Commercial	Yes	No	20000	Yes	Somewhat (Need to convert date/time to integer in Unix time)	No	Yes (Sparse-Dense Vectors)	pinecone.io
Milvus	Yes	1,751	Yes	Yes	Yes	? (Depending on Hosting?)	Yes	Apache License 2.0	No	No	32768	Yes	Somewhat (Need to convert date/time to integer in Unix time)	No	No, they use the phrase "Hybrid Search", but it really means metadata filtering	milvus.io
ChromaDB	In memory of server	?	Yes	Not Yet	? (Depending on Hosting?)	? (Depending on Hosting?)	Yes	Apache License 2.0	No	No		Yes	Somewhat (Need to convert time to an integer)	No	query	chroma.com

벡터 데이터베이스 알고리즘과 SDK 비교

- 유사도 비교 방법

Vector Database	Distance Metrics	ANN Algorithms	Filtering	Post-Processing
Qdrant	Cosine, Dot Product, L2	HNSW	☑	☑
Pinecone	Cosine, Euclidean, Dot Product	Proprietary (Pinecone Graph Algorithm)	☑	☑
Milvus	Euclidean, Cosine, IP, L2, Hamming, Jaccard, Tanimoto	HNSW, IVF_FLAT, IVF_SQ8, IVF_PQ, RNSG, ANNOY	☑	☑
Chroma	Cosine, Euclidean, Dot Product	HNSW	☑	☑
Weaviate	Cosine	HNSW	☑	☑
Faiss	L2, Cosine, IP, L1, Linf	IVF, HNSW, IMI, PQ	✗	✗
Elasticsearch	Cosine, Dot Product, L1, L2	HNSW	☑	☑

- API 통합

Vector Database	Language SDKs	REST API	GraphQL API	GRPC API
Qdrant	Python, Rust	☑	✗	☑
Pinecone	Python, Node.js, Go, Rust	☑	✗	☑
Milvus	Python, Java, Go, C++, Node.js, RESTful	☑	✗	☑
Chroma	Python	☑	✗	✗
Weaviate	Python, Java, Go, JavaScript, .NET	☑	☑	☑
Faiss	C++, Python	✗	✗	☑
Elasticsearch	Java, Python, Go, Ruby, PHP, Rust, Perl	☑	✗	✗

알고리즘 별 벡터 인덱싱 튜닝 파라미터

알고리즘	주요 하이퍼파라미터	설명	알고리즘	주요 하이퍼파라미터	설명
HNSW (Hierarchical Navigable Small World)	M	한 노드당 연결할 이웃 수. 큰 값은 인덱스 크기를 증가시키지만 검색 정확도가 향상됨.	LSH (Locality Sensitive Hashing)	n_hash_tables	해시 테이블의 수. 많은 테이블은 검색 정확도를 높이지만 메모리 사용량이 증가함.
	ef_construction	인덱스 생성 시 고려할 이웃 수. 값이 크면 인덱스 생성 속도가 느려지지만 검색 품질이 좋아짐.		hash_table_size	해시 테이블의 크기. 크기가 커지면 충돌 가능성이 낮아져 정확도가 향상될 수 있음.
	ef_search	검색 시 고려할 이웃 수. 값이 클수록 검색 정확도가 높아지지만 속도는 느려짐.		distance_metric	유사도 측정 방식 (Hamming, Euclidean 등). 검색 성능과 자원 사용에 영향을 미침.
	distance_metric	벡터 간 거리 계산 방식 (L2, Cosine 등). 검색 성능에 영향을 미침.	Flat (Brute-force)	N/A	모든 벡터를 순차적으로 비교하므로 튜닝할 하이퍼파라미터가 없음. 정확도는 가장 높지만 속도는 가장 느림.
IVF (Inverted File Index)	nlist	벡터를 그룹화하는 클러스터의 수. 클러스터 수가 많을수록 검색 정확도가 증가하지만 메모리 사용량이 커짐.	Annoy (Approximate Nearest Neighbors Oh Yeah)	n_trees	트리의 개수. 트리가 많을수록 검색 정확도가 향상되지만 메모리 사용량과 인덱스 생성 시간이 증가함.
	nprobe	검색 시 탐색할 클러스터 수. 값이 클수록 정확도는 향상되지만 검색 속도는 느려짐.		search_k	검색 시 탐색할 후보 벡터 수. 값이 클수록 정확도가 높아지지만 검색 속도는 느려짐.
	metric_type	벡터 간 유사도를 계산하는 방식 (Euclidean, Inner Product 등). 검색 성능과 정확도에 영향을 미침.		distance_metric	벡터 간 거리 계산 방식 (Euclidean, Manhattan 등). 성능과 정확도에 영향을 미침.
PQ (Product Quantization)	code_size	각 벡터를 압축할 때 사용할 코드북 크기. 더 작은 크기는 더 빠른 검색을 가능하게 하지만 정확도가 낮아짐.	FLANN (Fast Library for Approximate Nearest Neighbors)	branching	KD-tree의 가지 분기 수. 분기가 많을수록 검색 성능은 높아지지만 인덱스 크기가 증가함.
	n_bits	각 서브 벡터를 인코딩할 비트 수. 값이 클수록 압축 손실이 적어지지만 메모리 사용량이 증가함.		iterations	인덱스를 생성할 때 반복할 횟수. 많은 반복은 인덱스 품질을 향상시키지만 시간이 더 소요됨.
	metric_type	벡터 유사도 측정 방식 (L2, Inner Product 등).		checks	검색 시 확인할 노드 수. 값이 클수록 검색 정확도는 향상되지만 속도는 느려짐.

ChromaDB 주요 파라미터

파라미터	설명	기본값	효과
`IS_PERSISTENT`	데이터베이스가 데이터를 영구적으로 저장할지 여부를 설정합니다.	`FALSE`	데이터를 디스크에 영구 저장하여 재시작 시에도 데이터 유지 가능.
`PERSIST_DIRECTORY`	데이터베이스가 데이터를 영구적으로 저장할 경로를 지정합니다.	`./chroma`	데이터가 저장될 경로를 설정하여, 디스크에 저장된 데이터를 관리할 수 있습니다.
`ALLOW_RESET`	데이터베이스 인덱스를 초기화(모든 데이터 삭제)할 수 있는지 여부를 설정합니다.	`FALSE`	인덱스를 초기화하여 모든 데이터를 삭제할 수 있는 기능을 허용합니다.
`hnsw:space`	벡터 간 거리 계산에 사용되는 메트릭을 설정합니다. 'l2', 'cosine', 'ip' 중 선택 가능.	'l2'	검색 정확도와 성능에 영향을 미치며, 선택한 메트릭에 따라 검색 결과가 달라질 수 있습니다 ①.
`hnsw:M`	HNSW 그래프에서 각 노드의 최대 연결 수를 설정합니다.	16	더 많은 연결은 검색 정확도를 높이지만 메모리 사용량과 검색 속도를 증가시킵니다 ①.
`hnsw:search_ef`	검색 시 이웃의 동적 리스트 크기를 설정하여 검색 정확도를 조정합니다.	10	더 높은 값은 검색 정확도를 높이지만 메모리 사용량을 증가시킵니다 ①.
`hnsw:construction_ef`	새로운 벡터 추가 시 탐색할 이웃의 수를 설정합니다.	100	값이 높을수록 더 정확한 결과를 제공하지만 메모리 사용량과 인덱스 생성 시간이 증가합니다 ① ②.

HNSW 튜닝 사례

M	색인 빌드 시간	응답시간	재현율
16	317초	56.6 밀리 초	76.1%
32	688초	97.3 밀리 초	92.4%
48	1166초	98.8 밀리 초	96.9%
60	1585초	119.4 밀리 초	98.6%

표1. M값에 따른 성능 변화 표

256 차원 랜덤 벡터 100만개, ef_construction = 500,
K = 1000, ef = 10000

ef_con	색인 빌드 시간	응답시간	재현율
500	688초	88.7 밀리 초	88.1%
1000	1149초	70.8 밀리 초	89.5%
2000	1942초	66.1 밀리 초	91.0%
3000	2681초	59.6 밀리 초	92.1%

표2. ef_construction 값에 따른 성능 변화 표

256 차원 랜덤 벡터 100만개, M = 32,
K = 1000, ef = 7000

ef	응답 시간	재현율
1000	14.8 밀리 초	53.3 %
3000	41.0 밀리 초	76.1 %
5000	75.8 밀리 초	85.0 %
7000	78.8 밀리 초	89.5 %
10000	103.9 밀리 초	93.4 %

표3. ef 값에 따른 성능 변화 표

256 차원 랜덤 벡터 100만 개, M = 32, ef_con = 1000
K = 1000

- 인덱스 생성 : M 과 ef_con 조정 ➔ 인덱스 생성 시간과 정확도의 trade-off
 - ✓ M : 인덱스 크기와 생성 시간에 영향
 - ✓ ef_con : 생성 시간에 영향
- 벡터 검색 : ef_search 조정 ➔ 검색 속도와 정확도의 trade-off
 - ✓ ef_search : 검색 시간에 영향

ChromaDB : 일반적인 설정

데이터셋 크기	M (연결할 최대 이웃 수)	ef_construction (그래프 생성 시 후보군의 크기)	search_ef (검색 시 후보군의 크기)
작은 데이터셋 (1k ~ 10k 노드)	5 ~ 10	50 ~ 100	10 ~ 50
중간 크기 데이터셋 (10k ~ 100k 노드)	10 ~ 20	100 ~ 200	50 ~ 200
큰 데이터셋 (100k ~ 수백만 노드)	20 ~ 50	200 ~ 500	200 ~ 1000

- 높은 M, ef_construction, search_ef : 탐색 정확도 및 그래프 생성 품질 향상, 메모리와 계산 비용 증가.
- 낮은 M, ef_construction, search_ef : 탐색 속도 향상, 정확도 저하 가능.
- 데이터셋 크기가 커질수록 해당 파라미터들을 더 크게 설정
- 메모리와 처리 시간에 대한 트레이드오프를 고려

Vector 데이터베이스

- ChromaDB 실습

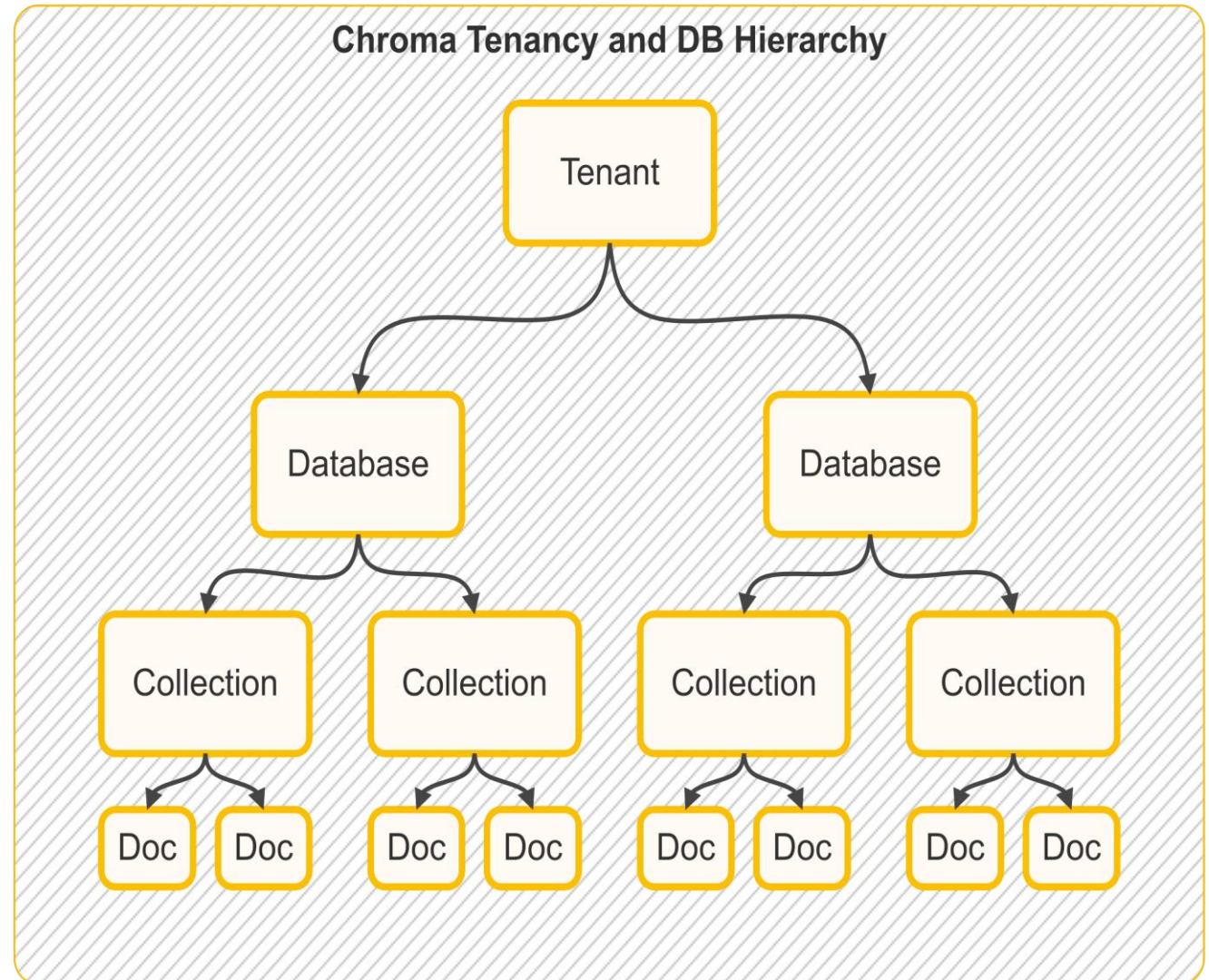
ChromaDB

- 2022년: Chroma 프로젝트 시작. 생성형 AI와 LLM의 성장에 따라 벡터 데이터베이스의 필요성에 대응해 개발 착수.
- 2023년: ChromaDB 오픈소스 출시. **LangChain** 및 **OpenAI API와 통합**되며 생성형 AI 애플리케이션에서 주목받음.
- 2023년: 대규모 데이터셋 처리 및 실시간 검색 기능 강화, 커뮤니티 성장과 GitHub에서 활발한 활동 진행.
- 2024년 이후 (예상): 실시간 데이터 처리, 멀티모달 데이터 지원 등 기능 확장과 클라우드 기반 확장성 강화 전망.

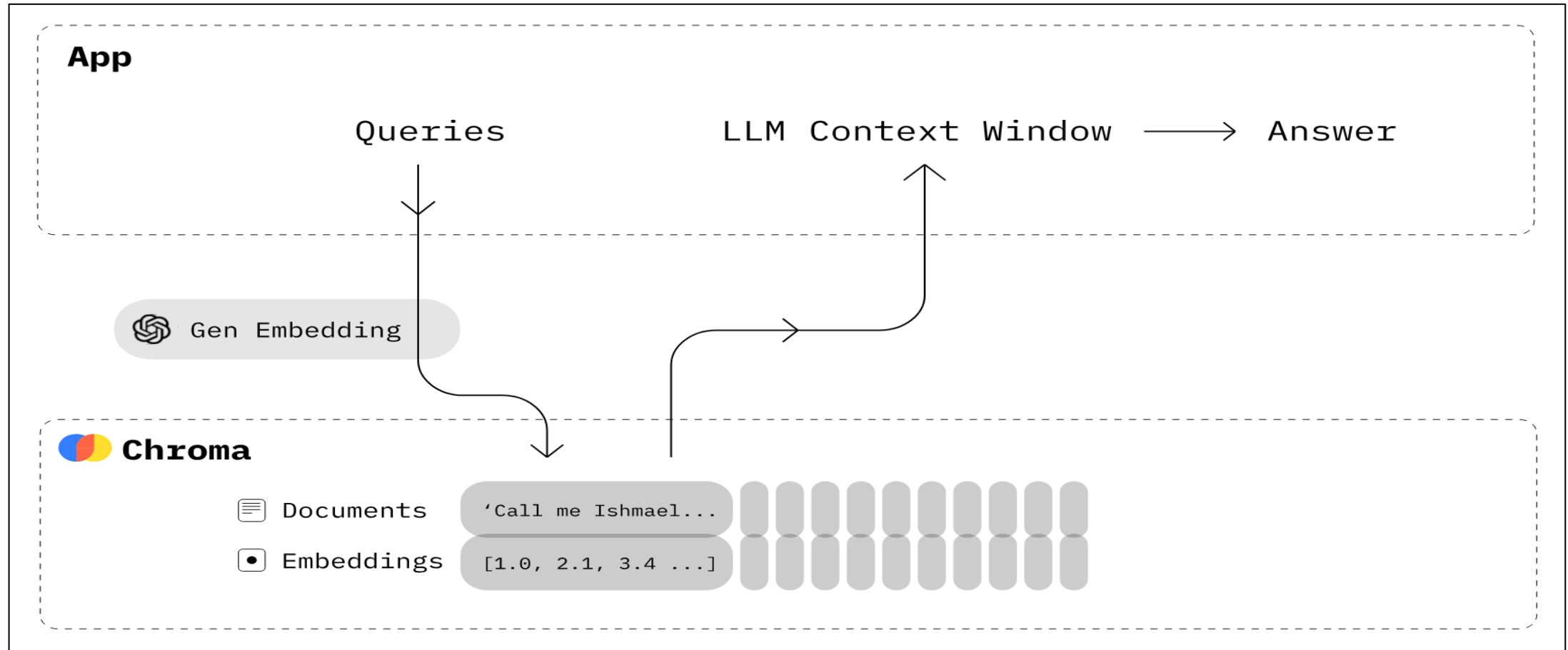
특징	설명
빠른 성능	임베딩 저장 및 검색이 매우 빠르고 효율적
파이썬 네이티브	파이썬으로 개발되어 파이썬 생태계와의 통합이 용이
다양한 백엔드	SQLite, PostgreSQL 등 여러 백엔드 시스템 지원
멀티테넌시	기업 환경에 적합한 멀티테넌트 아키텍처 지원
메타데이터 필터링	강력한 메타데이터 필터링 기능 제공
REST API	API를 통한 접근 가능
로컬 실행	로컬 환경에서 실행 가능하여 개발/테스트 용이
오픈소스	무료로 사용 가능한 오픈소스 라이선스

ChromaDB 구성

- Tenant (테넌트)
 - ✓ 가장 상위 수준의 격리 단위
 - ✓ 다수의 데이터베이스
 - ✓ 조직이나 사용자 그룹 분리
- Database (데이터베이스)
 - ✓ 테넌트 내에 논리적 컨테이너
 - ✓ 독립적인 collection 포함
- Collection (컬렉션)
 - ✓ 임베딩과 메타데이터를 저장하는 기본 단위
 - ✓ 유사한 특성을 가진 문서나 데이터를 그룹화
 - ✓ 고유한 스키마와 설정
 - ✓ 벡터 검색, 필터링, CRUD 작업 수행



ChromaDB 실습

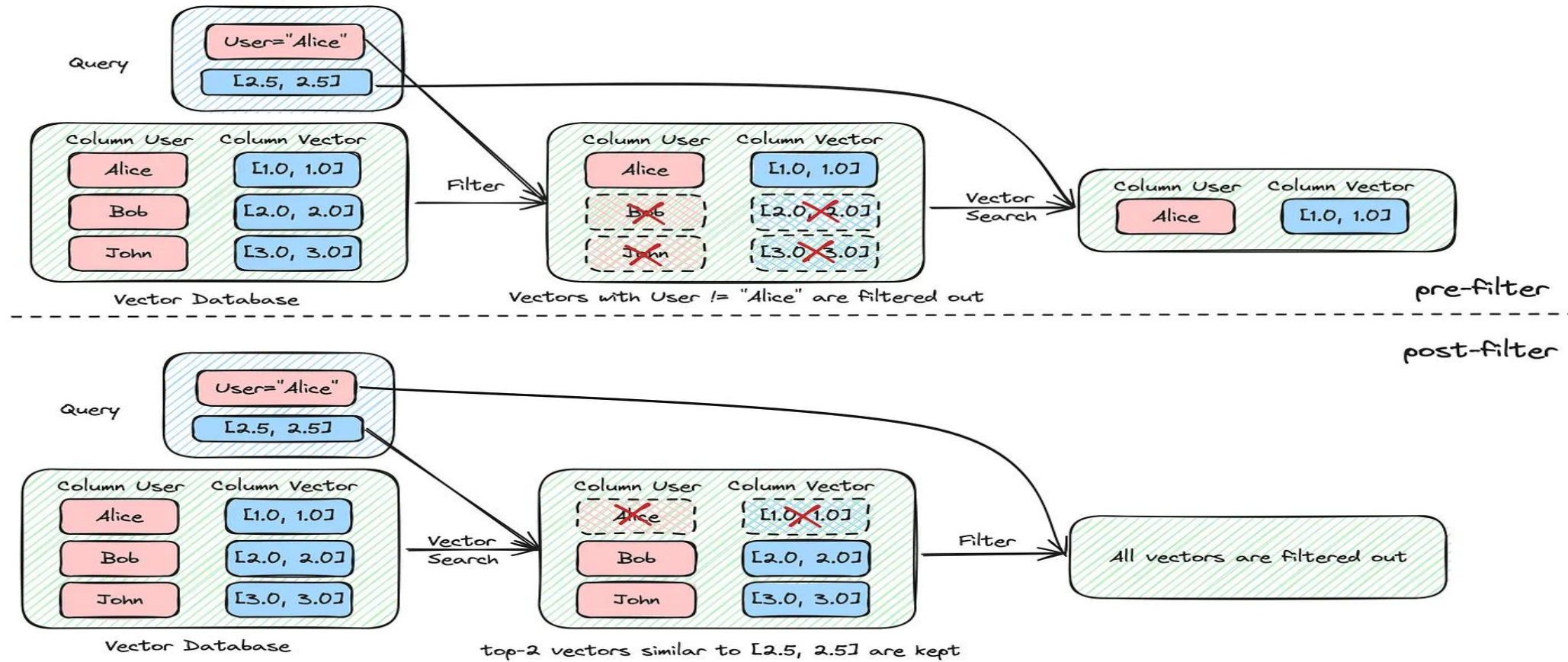


출처 : <https://opentutorials.org/module/6369>

ChromaDB + Embedding 실습

실습 : https://colab.research.google.com/drive/1Qlv8Te_U-X0C7YFvfxiW2doddVSoP7p?usp=sharing

ChromaDB 실습



출처 : <https://medium.com/@myscale/optimizing-filtered-vector-search-in-myscale-77675aaa849c>

ChromaDB + 필터 실습

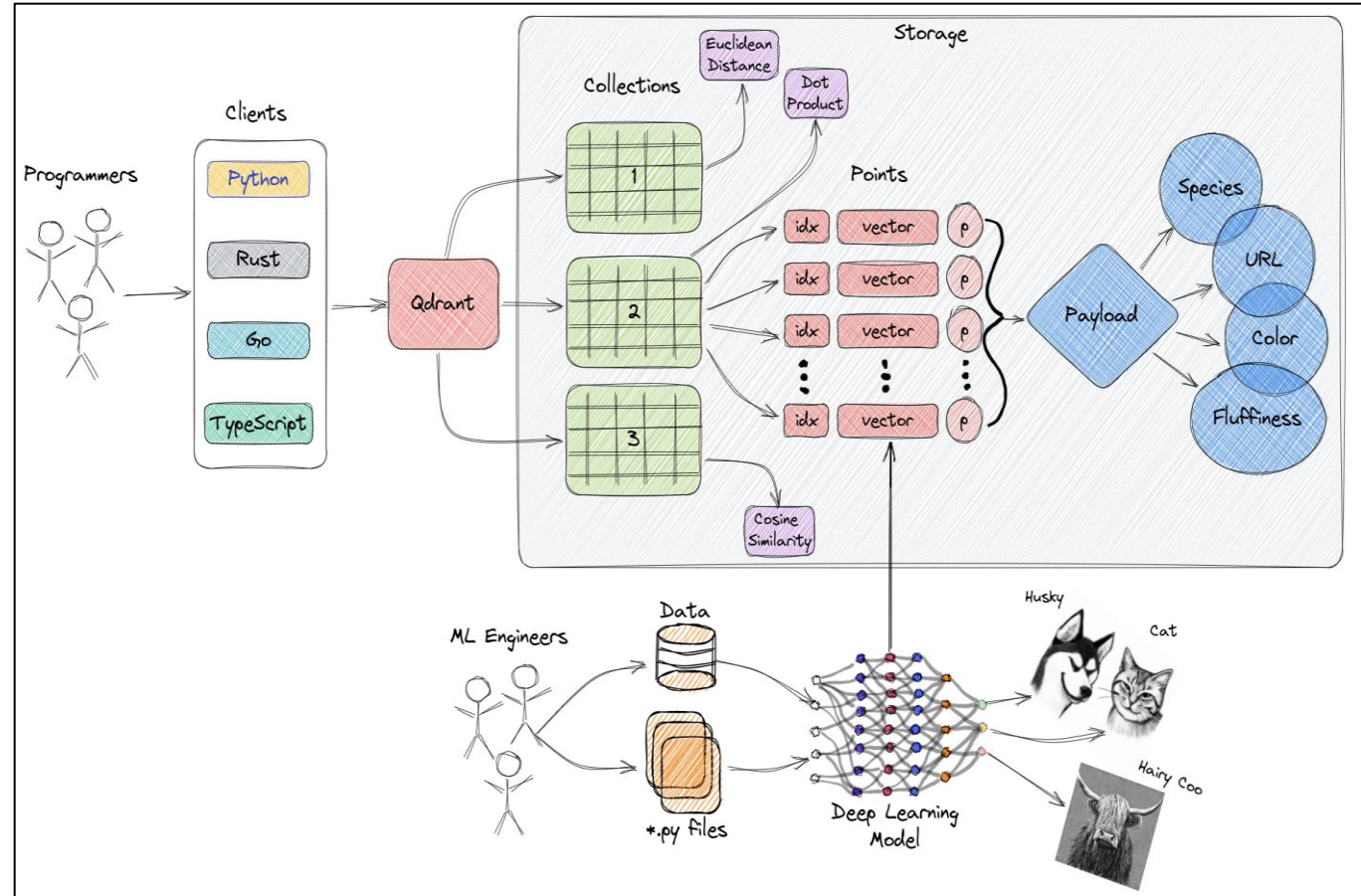
실습 : <https://colab.research.google.com/drive/1isvHKC4s6dZHxEhZVZEEX8J1natTYlvF?usp=sharing>

Vector 데이터베이스

- Qdrant 실습

Qdrant 실습

- Collection
 - ✓ 벡터 데이터를 저장하는 독립된 논리적 컨테이너
 - ✓ 데이터 분할 및 관리를 위한 단위
- Point
 - ✓ 고유 ID를 가진 벡터 및 관련 메타데이터
 - ✓ 벡터 값, 페이로드 및 다양한 메타데이터 포함
- Vector
 - ✓ 이미지, 텍스트, 오디오 등이 임베딩 된 벡터
 - ✓ 유사성 검색 및 군집화에 사용
- Payload
 - ✓ 벡터에 연결된 메타데이터로 검색에 사용 가능
 - ✓ 키-값 쌍 형식으로 저장
- Indexing
 - ✓ 검색 성능 향상을 위해 벡터 데이터 인덱싱
 - ✓ 다양한 인덱싱 알고리즘 사용 가능
- Similarity Search
 - ✓ 주어진 벡터와 가장 유사한 벡터 찾기
 - ✓ 거리 메트릭(L2 거리 등)을 사용하여 유사도 계산



출처 : <https://qdrant.tech/documentation/overview/>

Tokenizer to Attention 결과

실습 : https://colab.research.google.com/drive/1hv1PHd55p_wK49ACVj4hh9nZ_Cc_8boD?usp=sharing

Vector 데이터베이스

- Weaviate 실습

Weaviate

- 2018년: 네덜란드의 'Semi Technologies' 에서 비정형 데이터 관리를 위한 Weaviate 프로젝트 시작
- 2019년: 첫 오픈소스 벡터 검색 데이터베이스 출시. NLP와 벡터 저장 중심.
- 2022년: Weaviate의 성능과 확장성 강화. 대규모 데이터셋 처리 및 새로운 인덱스 기술 도입.
- 2023년: 커뮤니티 성장 . RAG(검색 증강 생성) 기술과 통합된 애플리케이션 개발 활성화.
- 2024년: 다중 테넌시 기능 개선, 실시간 업데이트 및 멀티모달 데이터 지원 확대

특징	설명
확장성	수평적 확장이 가능한 분산 아키텍처 지원
GraphQL API	직관적인 GraphQL 인터페이스 제공
실시간 벡터화	데이터 입력 시 자동 벡터화 지원
다중 샤딩	대규모 데이터셋 처리를 위한 샤딩 지원
CRUD 작업	벡터와 객체에 대한 완전한 CRUD 작업 지원
멀티 테넌시	기업용 멀티 테넌트 지원
보안 기능	인증, 권한 관리 등 엔터프라이즈급 보안 제공
다양한 인덱싱 지원	여러 벡터 인덱스 백엔드 지원 (HNSW, LSH 등)

Weaviate 구성

•클래스(Class): 데이터의 카테고리 또는 타입

예: "Book", "Movie", "Product" 등.

•속성(Properties): 클래스 안에 저장되는 데이터의 세부 항목

예: 제목, 저자, 출판연도 등.

•데이터 타입(DataType): 각 속성에 저장되는데이터의 유형

예: 문자열(string), 숫자(int), 벡터(vector) 등

json

```
{
  "class": "Book",
  "properties": {
    "title": "1984",
    "author": "George Orwell",
    "publicationYear": 1949,
    "embedding": [0.12, 0.85, 0.33, 0.44]
  }
}
```

json

📄 복사 ✎ 편집

```
{
  "classes": [
    {
      "class": "Book",
      "description": "A collection of written works",
      "properties": [
        {
          "name": "title",
          "dataType": ["string"],
          "description": "The title of the book"
        },
        {
          "name": "author",
          "dataType": ["string"],
          "description": "The author of the book"
        },
        {
          "name": "publicationYear",
          "dataType": ["int"],
          "description": "The year the book was published"
        },
        {
          "name": "embedding",
          "dataType": ["vector"],
          "description": "Vector representation of the book's content"
        }
      ]
    }
  ]
}
```

Weaviate 실습

1. with_near_text

- **설명:** 텍스트 기반으로 데이터의 유사성을 평가.
- **사용 예:** 특정 개념과 의미적으로 유사한 데이터를 검색.
- **옵션 필드:**
 - `concepts`: 텍스트 벡터화의 기준이 되는 단어나 문장.
 - `distance`: (선택 사항) 결과와의 최대 거리.
 - `certainty`: (선택 사항) 검색 결과의 최소 유사도 (0~1 사이의 값).

예시:

python

복사 편집

```
.with_near_text({
    "concepts": ["biology", "organisms"],
    "certainty": 0.8
})
```

2. with_near_vector

- **설명:** 벡터 데이터를 직접 사용해 유사성을 평가.
- **사용 예:** 특정 벡터와 가장 가까운 데이터를 검색.
- **옵션 필드:**
 - `vector`: 유사성 비교에 사용할 벡터 값.
 - `distance`: (선택 사항) 최대 허용 거리.
 - `certainty`: (선택 사항) 최소 유사도.

예시:

python

```
.with_near_vector({
    "vector": [0.12, 0.56, 0.78],
    "distance": 0.1
})
```

3. with_near_object

- **설명:** 기존 Weaviate 객체의 ID를 기준으로 유사한 데이터를 검색.
- **사용 예:** 특정 객체와 유사한 데이터를 찾고 싶을 때.
- **옵션 필드:**
 - `id`: 기준이 되는 객체의 UUID.
 - `distance`: (선택 사항) 최대 허용 거리.
 - `certainty`: (선택 사항) 최소 유사도.

예시:

python

```
.with_near_object({
    "id": "123e4567-e89b-12d3-a456-426614174000"
})
```

4. with_near_image

- **설명:** 이미지 데이터를 기준으로 유사한 이미지를 검색.
- **사용 예:** 업로드된 이미지와 유사한 이미지를 찾고 싶을 때.
- **옵션 필드:**
 - `image`: Base64로 인코딩된 이미지 데이터.
 - `distance`: (선택 사항) 최대 허용 거리.
 - `certainty`: (선택 사항) 최소 유사도.

예시:

python

```
.with_near_image({
    "image": "data:image/jpeg;base64,/9j/4AAQSkZ..."
})
```

Weaviate CRUD

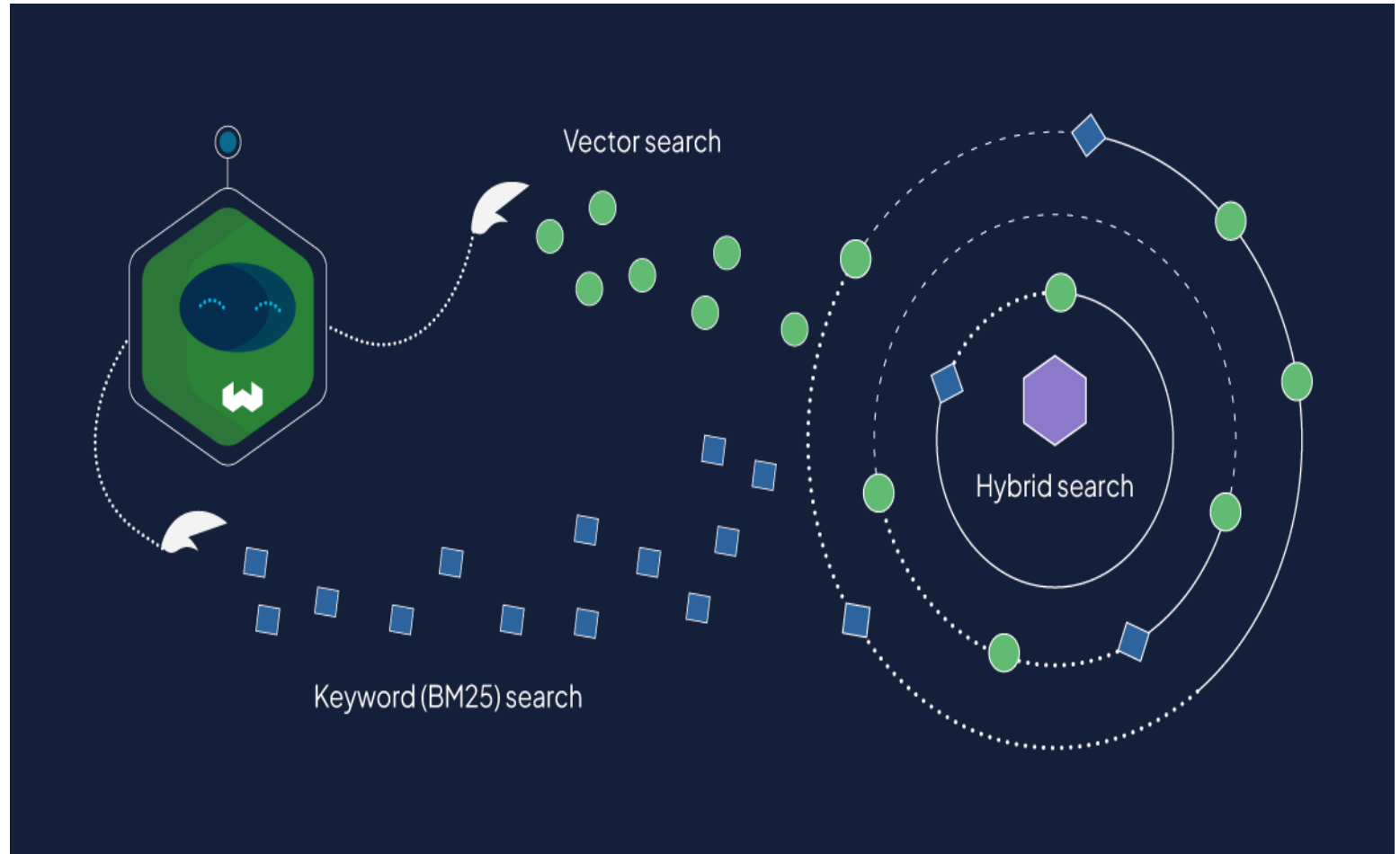
실습 링크 : <https://colab.research.google.com/drive/1LEi1jMTwhNv2VDa4IBS7UTiVWrQAltWq?usp=sharing>

Weaviate 실습

```
response = (  
    client.query  
        .get("JeopardyQuestion", ["question", "answer"])  
        .with_hybrid(query="food", alpha=0.5)  
        .with_limit(5)  
        .do()  
)
```

```
response = (  
    client.query  
        .get("JeopardyQuestion", ["question", "answer"])  
        .with_near_text({"concepts": ["food"]})  
        .with_limit(5)  
        .do()  
)
```

```
response = (  
    client.query  
        .get("JeopardyQuestion", ["question", "answer"])  
        .with_bm25(query="food")  
        .with_limit(5)  
        .do()  
)
```



출처 : <https://weaviate.io/blog/hybrid-search-fusion-algorithms>

Weaviate 하이브리드 검색

실습 : https://colab.research.google.com/drive/1VJnj1egMGILGrW7TSFBYAlD_bcDwyQL?usp=sharing

RAG 실습

웹 문서와 LLM 연동 : 지시 + 컨텍스트(Context)

- 웹 문서를 Context로 제공하여 LLM 답변 요청
 - ✓ 입력된 내용과 연관된 문서 검색
(google, 네이버, ...)
 - ✓ 검색된 내용을 가져오기
 - ✓ 해당 문서 첨부하여 LLM 답변 요청
- 검색 정확도에 따라 답변 정확도가 달라짐

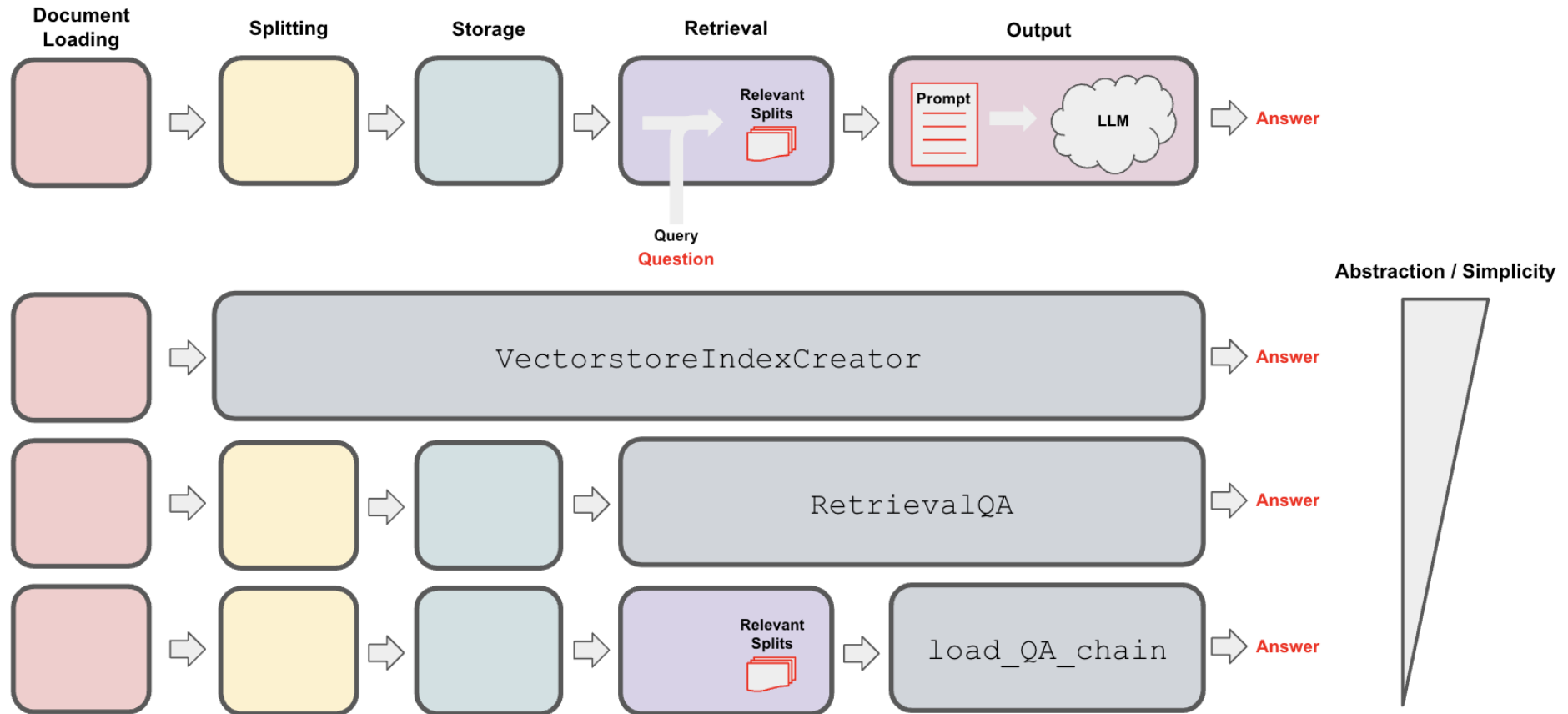
지시 : 프롬프트

컨텍스트 : 웹 문서

웹 조회 기반 생성(RAG) = 프롬프트 + 웹 검색



LangChain Retrieval QA Chain 추상화 단계



RAG : 문서 검색 기반 생성 (RAG : Retrieval Augmented Generation)

초록마을, AI가 상품 찾아준다

| 마이크로소프트 GPT-4 검색엔진 장착

유통 | 입력 : 2023/08/10 08:38

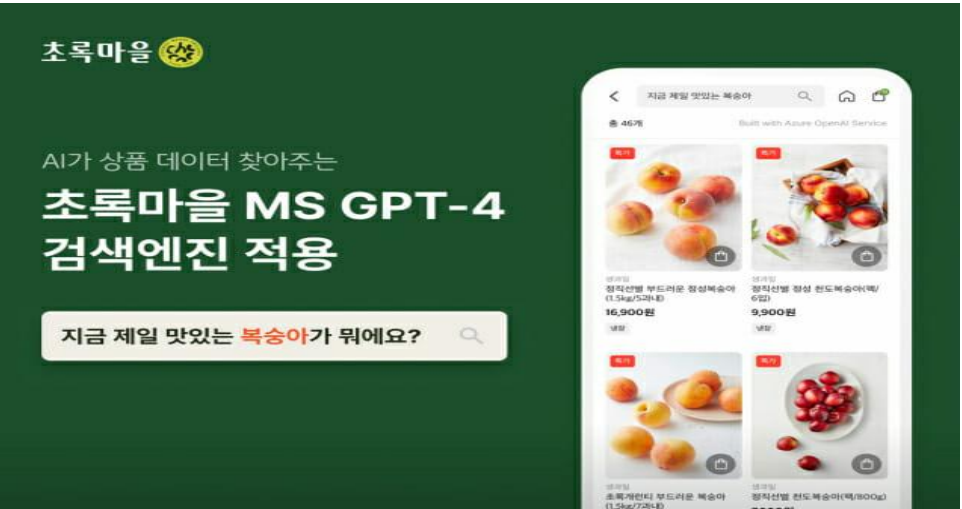
안희정 기자 |   기자 페이지 구독  기자의 다른기사 보기     

[웹비나] ROHM | EEPROM의 기초와 특징, 성능을 최대화시킬 수 있는 테크닉 등을 소개합니다!

D2C 푸드테크 스타트업 정육각이 마이크로소프트의 애저(Azure) 오픈AI GPT-4를 적용한 검색엔진을 자체 개발하고 친환경 유기농 전문 초록마을의 모바일 앱에 전격 도입했다고 10일 밝혔다.

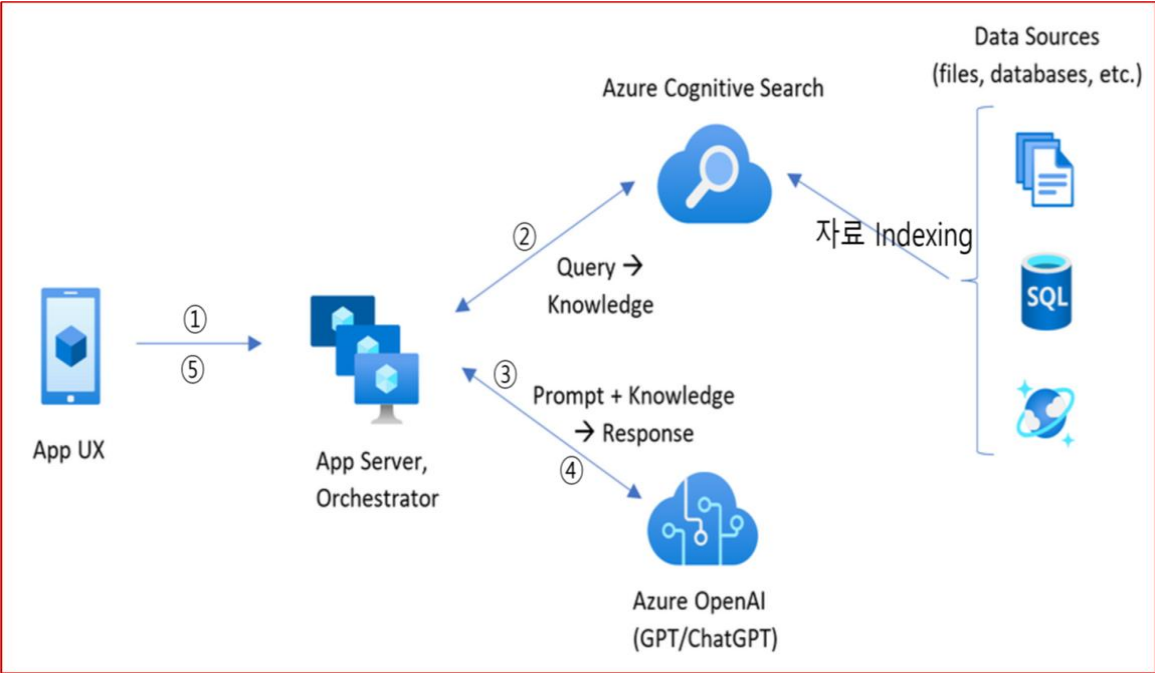
지난달 초 모바일 앱을 네이티브 앱 방식으로 전면 재개발한 데 이어 고객 편의 극대화에 주력한다는 복안이다.

새롭게 적용한 검색엔진은 학습한 검색 패턴을 바탕으로 고객의 의도를 파악하고 관심사가 반영된 개인 맞춤형 결과를 추천해준다. 가령 미역국을 입력하면 초록마을 내 완제품을 포함해 국거리용 한우, 바지락살, 국물용 멸치 등의 부재료 및 간장과 같은 고관여 상품까지 노출하는 식이다.



새로운 검색엔진은 정육각 개발팀과 마이크로소프트가 협업한 결과물로 구상부터 적용까지 채 한 달이 걸리지 않았다. 약 2만 개가 넘는 초록마을 상품마스터의 전처리 데이터 생성 및 정리에 GPT-4를 적용하는 것을 구상한 후 마이크로소프트의 전문가 지원 프로그램인 패스트트랙을 통해 한국, 호주의 엔지니어들과 접근 방식을 논의했다.

24년 된 초록마을이 빠르게 검색엔진을 갈아끼울 수 있었던 배경에는 클라우드 기반으로 경영 환경을 전면 전환하면서 지난 달 초 도입을 완료한 마이크로소프트 애저가 있다. 애저 CosmosDB로 이전한 기존 상품 정보에 GPT-4로 생성한 원천 데이터를 결합하고 애저 Cognitive Search를 이용하는 등 애저 생태계 내에서 매끄럽고 신속하게 새로운 기능을 구현했다.



기업 데이터와 LLM 연동 : 지시 + 컨텍스트(Context)

기업 데이터를 Context로 제공하자.

- ✓ 토큰 제한 : **데이터가 크면??** → 쪼개자 (split)
 - ✓ 쪼개지면 필요한 **문서를 어떻게 찾을까?** → 질문과 유사한 문서를 찾자 (embedding 비교)
 - 질문 : **무엇을 임베딩**하면 답변에 필요한 **문서를 정확히** 찾을까?
 - ✓ 문서가 많으면 **찾는 속도가 늦지 않을까?** → **인덱싱(indexing)**을 하자.
 - **벡터 데이터베이스(Vector Database)** : 인덱싱된 벡터 관리 시스템
- 지시 : 프롬프트



컨텍스트 : 벡터 데이터베이스 내의 문서

내부 문서 조회 기반 생성(RAG) = 프롬프트 + 벡터데이터베이스

RAG : 기업 데이터와 LLM 연동

- 준비 사항

- ✓ 입력 데이터 : 종합소득세 안내

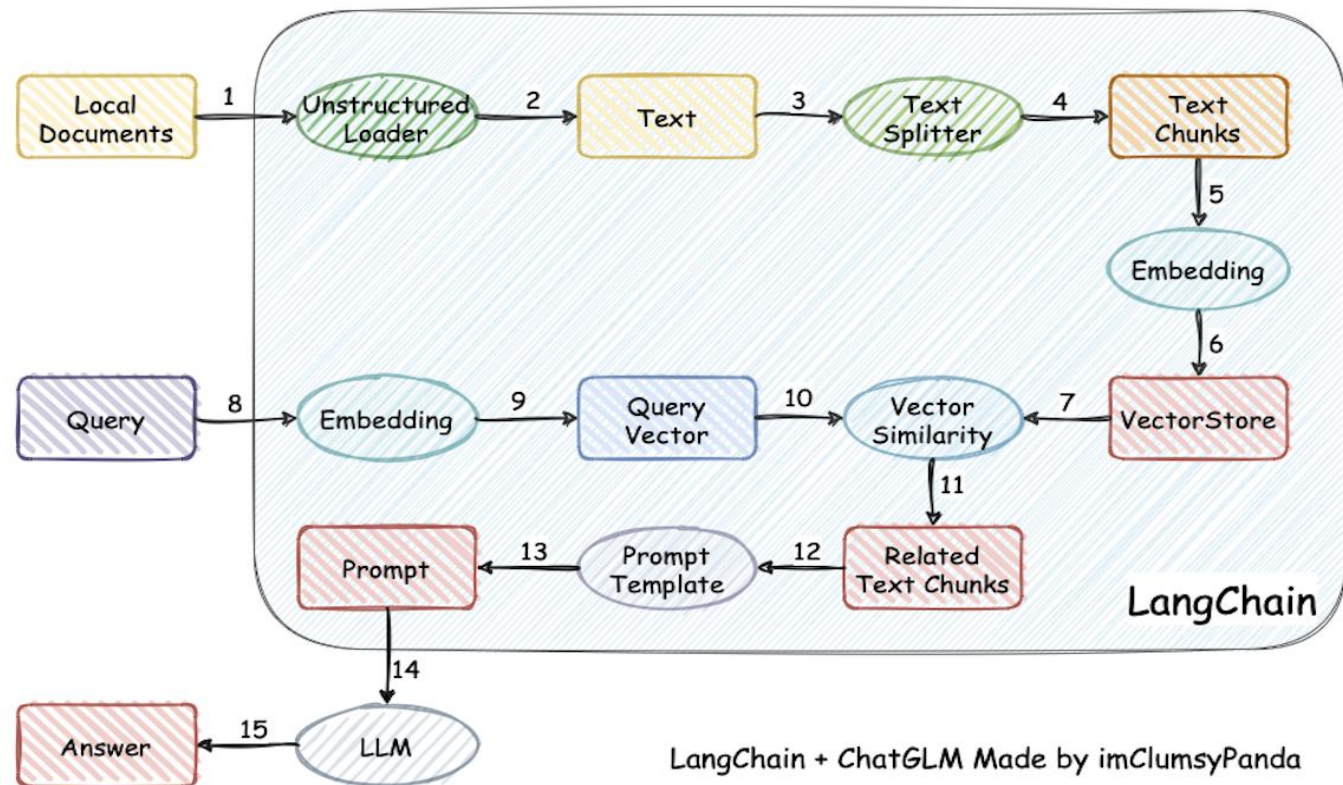
[국세청>국세신고안내>개인신고안내>종합소득세>기본정보>종합소득세 개요 \(nts.go.kr\)](https://nts.go.kr)

- ✓ KMS : FAISS (Facebook AI Similarity Search)

- 기존 자료 탐색을 위한 Index 생성

- ✓ Orchestration Layer : LangChain

- OpenAI, FAISS 연동



KMS와 챗GPT 연동 – 종합소득세 안내

실습 : https://colab.research.google.com/drive/1IMiiNi1m_lvfb1MFjMOYz_swhicFoj8S?usp=sharing

RAG : 테스트 결과

```
2 #####
3 # question = "무신고 가산세 얼마야?"
4 # question = "종합소득세 확정신고 안해도 되는 경우는?" # 오류도 발생
5 # question = "수출 부진 및 산불 피해자 종합소득세 납부 기간은?" #
6 # question = "납부기한 연장 대상자는?" # 표 이해는 어려운 듯
7 # question = "전문사업자는 복식부기 대상인가?"
8 question = "종합소득세 납부 기간은?" # 정보 요약이 상대적으로 좋은 듯
9
10 print(chatbot.run(
11     | prompt.format(query=question)
12 ))
```

종합소득세 납부 기간은 다음해 5월 1일부터 5월 31일입니다. 성실신고확인서 제출자는 6월 30일까지 납부 가능합니다.

```
2 #####
3 # question = "무신고 가산세 얼마야?"
4 # question = "종합소득세 확정신고 안해도 되는 경우는?" # 오류도 발생
5 # question = "수출 부진 및 산불 피해자 종합소득세 납부 기간은?" #
6 # question = "납부기한 연장 대상자는?" # 표 이해는 어려운 듯
7 question = "전문사업자는 복식부기 대상인가?"
8 # question = "종합소득세 납부 기간은?" # 정보 요약이 상대적으로 좋은 듯
9
10 print(chatbot.run(
11     | prompt.format(query=question)
12 ))
```

Yes, 전문사업자 is subject to 복식부기.

RAG : 테스트 결과

```
[13] 1
      2 #####
      3 # question = "무신고 가산세 얼마야?"
      4 question = "종합소득세 확정신고 안해도 되는 경우는?" # 오류도 발생
      5 # question = "수출 부진 및 산불 피해자 종합소득세 납부 기간은?" #
      6 # question = "납부기한 연장 대상자는?" # 표 이해는 어려운 듯
      7 # question = "전문사업자는 복식부기 대상인가?"
      8 # question = "종합소득세 납부 기간은?" # 정보 요약이 상대적으로 좋은 듯
      9
     10 print(chatbot.run(
     11     | prompt.format(query=question)
     12 ))
```

1. 근로소득만 있는 경우
2. 원천징수 의무가 없는 근로소득 또는 퇴직소득이 있는 경우
3. 연말정산을 하지 않은 경우
4. 직전 과세기간의 수입금액이 7,500만원 미만이고, 다른 소득이 없는 보험모집인·방문판매원 계약배달 판매원의 사업소득으로서 소속회사에서 연말정산을 한 경우
5. 퇴직소득과 연말정산대상 사업소득만 있는 경우
6. 비과세 또는 분리과세되는 소득만 있는 경우
7. 연 300만원 이하인 기타소득이 있는 자로서 분리과세를 원하는 경우 등

오류 ~~~ : Hallucination
어떻게 줄일 것인가?

✓ 다음의 경우에 해당되면 종합소득세를 확정신고하지 않아도 됩니다.

- 근로소득만 있는 자로서 연말정산을 한 경우
 - 다만 다음에 해당하는 경우는 확정신고하여야 합니다.
 - 2인 이상으로부터 받는 근로소득·공적연금소득·퇴직소득 또는 연말정산대상 사업소득세를 납부함으로써 확정신고 납부할 세액이 없는 경우 제외)
 - 원천징수 의무가 없는 근로소득 또는 퇴직소득이 있는 경우(납세조합이 연말정산이 규정에 따라 소득세를 납부한 경우 제외)
 - 연말정산을 하지 아니한 경우

S대 학칙 챗봇

```
##%
CHAT_PROMPT_TEMPLATE = ChatPromptTemplate.from_messages(
    [
        SystemMessagePromptTemplate.from_template(
            """
            Answer as thoroughly and truthfully as possible with the following context.
            Find the answer step by step with proofs.
            Answer in Korean with source.
            Use the following format.
            답변 :
                {Answer here}
            출처 :
                {Source fine name here}
            """
        ),
        MessagesPlaceholder(variable_name="history"),
        HumanMessagePromptTemplate.from_template("{input}"),
    ]
)

##%

# S대 챗봇 클래스 정의

class SKK_Talk :

    def __init__(self, dir_base) :

        self.dir_base = dir_base
        os.chdir(dir_base)
        print("Working Directory === ", os.getcwd())
```

```
IPython Console
Console I/O X
In [43]: runfile('C:/MyData/위데이터랩_자료/제품_개발/JJK_1000문(2023Jul)/test_streaming.py', wdir='C:/MyData/위데이터랩_자료/제품_개발/JJK_1000문(2023Jul)')

질문 == 등록금 반환 기준 알려줘
Answer =====
답변 :
등록금 반환 기준은 대학의 학칙에 따라 결정됩니다. 일반적으로 등록금은 과오납으로 인한 경우나 관계법령에 특별한 규정이 있는 경우를 제외하고는 반환되지 않습니다. 그러나 학칙에 따라 일부 경우에는 등록금을 반환할 수 있습니다.

학칙에 따르면 등록금 반환의 대상은 다음과 같습니다.
1. 법령에 의해 입학이 불가능하거나 학업을 계속할 수 없는 경우
2. 입학 허가를 받은 자가 입학 포기를 표시한 경우
3. 재학 중인 자가 자퇴를 표시한 경우
4. 휴학 중인 자가 복학하지 않아 제적된 경우
5. 등록한 자가 학기 중에 휴학하는 경우. 단, 병역법상 복무 의무, 임신, 출산, 육아, 창업 등의 사유로 휴학하는 경우에는 등록금을 반환하지 않고 복학 시 등록금을 면제할 수 있습니다.
6. 본인의 질병, 사망, 천재지변 또는 기타 부득이한 사유로 입학을 하지 않게 되거나 학업을 계속하지 않게 된 경우

또한, 계절수업 수강료의 반환 기준도 있습니다. 계절수업 수강료를 납부한 후 본인 사망, 자퇴, 제적, 질병, 천재지변 또는 기타 부득이한 사유로 계절수업에 참가할 수 없는 경우에는 이미 납부한 수강료를 반환할 수 있습니다.

이러한 등록금 반환 기준은 학칙에 따라 상세한 내용이 정해져 있으며, 학교마다 차이가 있을 수 있습니다. 따라서 등록금 반환에 관한 자세한 내용은 해당 대학의 학칙을 참고하시기 바랍니다.

출처 :
- 대학 학칙 (해당 대학의 학칙 파일)
관련 파일 =====
```

EZIS ChatDBA

www.chatdba.wedatalab.com

 안녕하세요 🙋 저는 EZIS ChatDBA입니다.
데이터베이스 관리시스템(DBMS)에 관한 질문을 해주세요.
[예: MySQL과 Oracle Database의 차이를 알려줘.]

 inner조인과 outer조인의 차이에 대해 알려줘.

 내부 조인은 두 테이블 사이의 공통 필드를 기반으로 두 테이블을 결합하는 조인 유형입니다. 쿼리에 지정된 기준과 일치하는 행만 반환합니다. 반면, 외부 조인은 두 테이블 사이의 공통 필드를 기반으로 두 테이블을 결합하지만, 다른 테이블에 일치하는 항목이 없더라도 한 테이블의 모든 행을 반환하는 조인 유형입니다. 외부 조인에는 왼쪽 외부 조인, 오른쪽 외부 조인, 완전 외부 조인의 세 가지 유형이 있습니다. 왼쪽 외부 조인은 오른쪽 테이블에 일치하는 항목이 없더라도 왼쪽 테이블의 모든 행을 반환합니다. 오른쪽 외부 조인은 왼쪽 테이블에 일치하는 항목이 없더라도 오른쪽 테이블의 모든 행을 반환합니다. 완전 외부 조인은 두 테이블 중 하나에 일치하는 항목이 없더라도 두 테이블의 모든 행을 반환합니다.

 ORA-00001 에러에 대해 알려줘.

 ORA-00001 오류는 고유 제약 조건 위반을 나타내는 Oracle 오류 메시지입니다. 이는 데이터베이스에 삽입되는 데이터가 기본 키나 고유 인덱스와 같은 고유 제약 조건을 위반한다는 의미입니다. 이 오류는 오류 원인에 대한 추가 정보를 제공할 수 있는 Oracle 로그를 확인하여 추가로 디버깅할 수 있습니다. 문제를 해결하려면 삽입되는 데이터의 순서를 조정해야 할 수도 있습니다. 그래도 문제가 해결되지 않으면 Oracle에 문의하여 추가 지원을 받는 것이 좋습니다.

 블랙핑크 멤버에 대해 알려줘.

 죄송합니다. DBMS 관련 지식에 대해 질문해주세요. 더 나은 답변을 제공해드릴 수 있습니다.

질문을 입력해주세요.



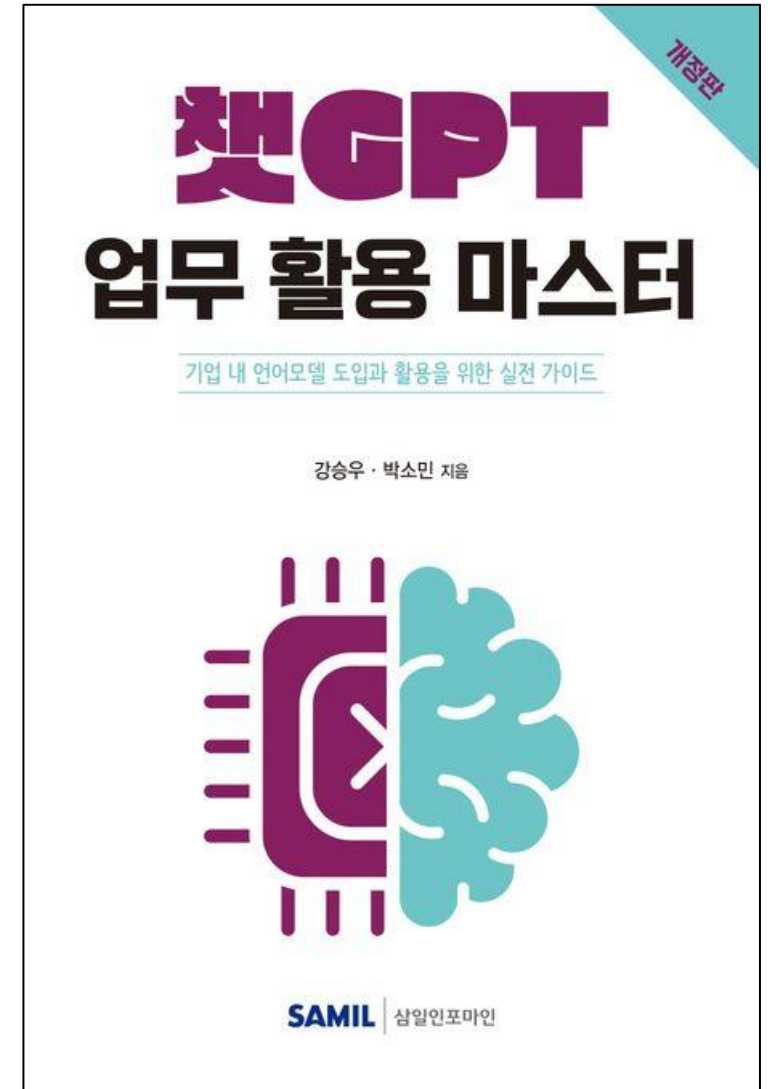
Response in English ☐

EZIS ChatDBA는 Oracle Community ASK TOM과 StackOverflow의 데이터를 기반으로 답변합니다. 해당 데이터가 없을 경우, 구글 검색 결과를 포함합니다. 따라서 사실과 다른 정보를 제공할 수 있습니다.

요약

요약

- RAG = 지시 + 컨텍스트(Context)
 - ✓ 데이터베이스 지식 : SQL
 - ✓ 일반 문서 지식 : 임베딩 후 유사도 비교, 키워드 탐색
 - ✓ 웹 문서/뉴스 지식 : 웹 검색
 - ✓ CSV 파일 : Pandas
- 벡터 데이터베이스 = 임베딩 + 인덱싱
 - ✓ 임베딩 (embedding) : 의미론적 비교
 - 임베딩 방법(모델)에 따른 조회 정확도 차이
 - 임베딩 데이터 처리에 따른 조회 정확도 차이
 - ✓ 인덱싱 (indexing) : (정확하고) 빠른 검색
 - 인덱싱 방법에 따른 조회 정확도와 속도 차이



검색 기반 생성 (RAG) = NLU + NLG

- LLM의 언어 이해 능력에 기반한 지식 주입
- 새로운 지식을 훈련 없이 주입
- LLM의 환각(Hallucination) 감소

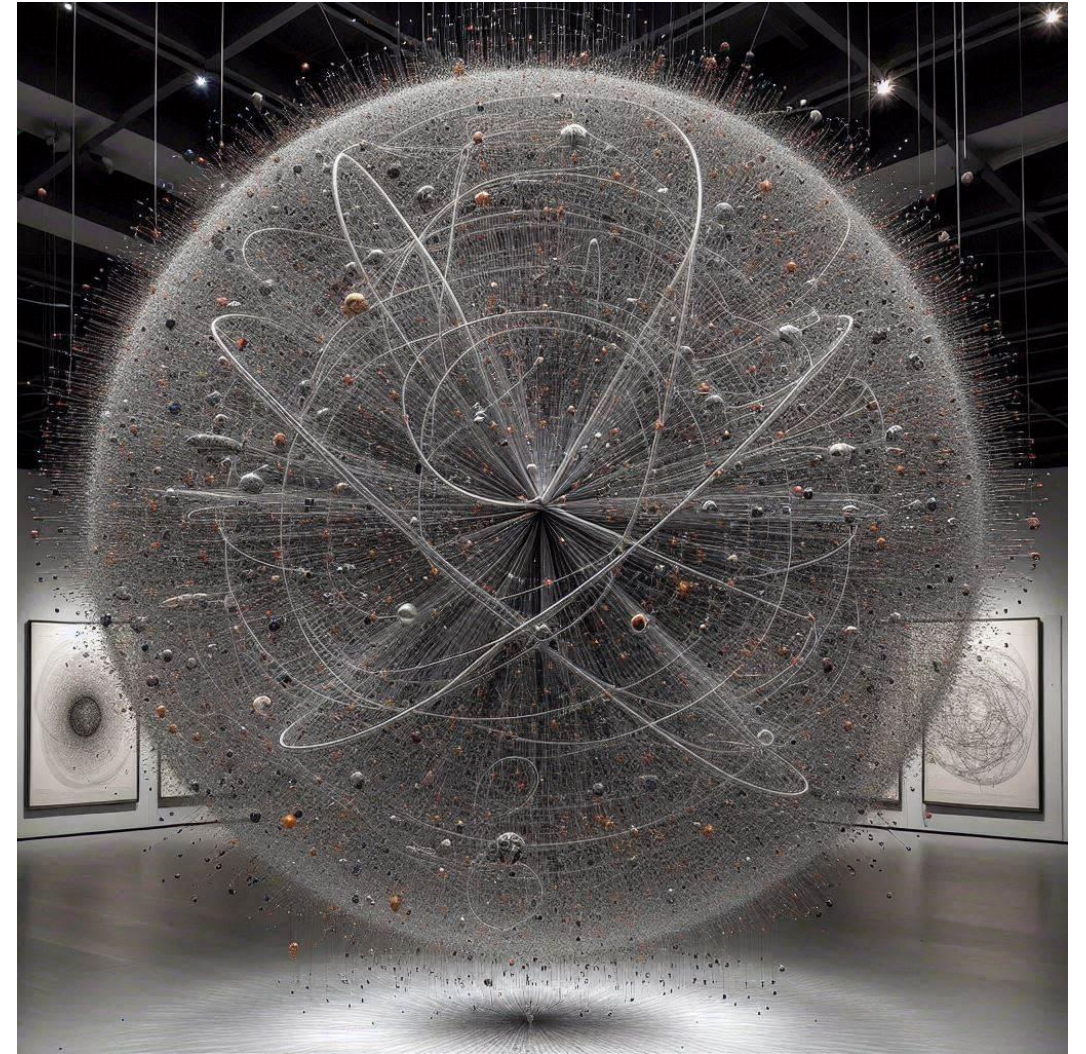
임베딩 : 정보를 (사람이 생각하는) 의미를 담은 수치화

- 임베딩 공간에서의 언어, 의미, 형태(소리, 글, 그림) 의 상대적 거리
- **Multi-Modal 임베딩** : 다양한 모드(글, 그림, 소리)의 정보의 연관관계 설정

벡터 데이터베이스

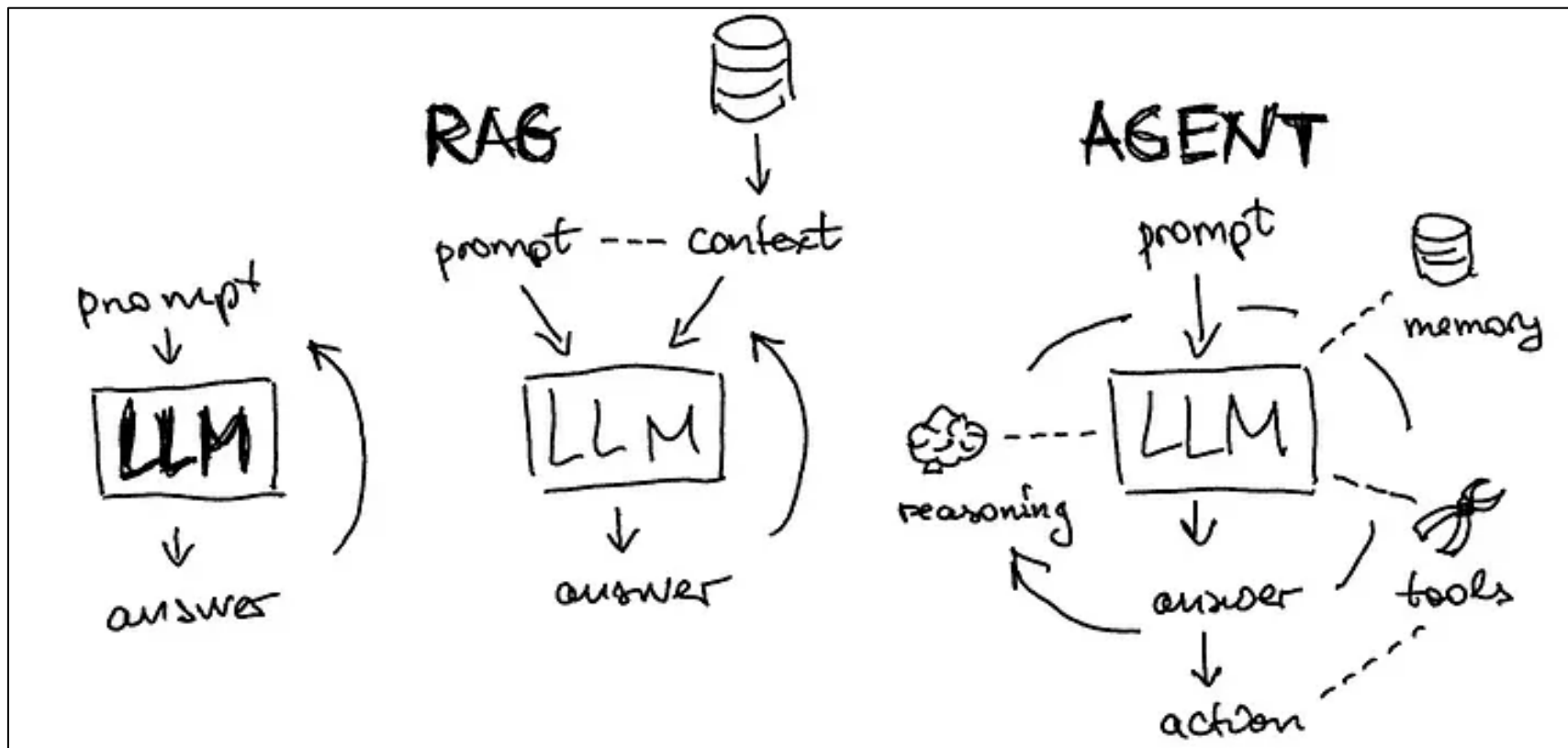
- 벡터화(수치화)된 지식 저장소
- 비정형 데이터 유사성 검색
- 비정형 데이터의 관계를 보관
- 비정형 데이터 간의 정보 융합 활성화

➔ AI 기반의 새로운 르네상스



벡터들 사이의 거리가 벡터들 사이의 관계를 나타내는 공간
< Copilot 생성 이미지 >

다양해지는 LLM 애플리케이션 : AI Agent 의 등장



- 다양한 Tool을 장착한 자율적(Autonomous) 판단을 하는 에이전트(Agent)
- 데이터 검색 뿐 아니라, 생성/변경/삭제 기능 추가

감사합니다.